

Componentes/ReasonForLostSale/STDReasonForLostSale.PRW

374 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STDREASONFORLOSTSALE.CH"
4
5 Static      oModel  := Nil          // Model
6
7 //-----
8 /*{Protheus.doc} STDRFDGetModel
9 Cria estrutura de dados do Model
10
11 @param
12 @return oModel      Retorna estrutura de dados do Model
13 @obs
14 @sample
15 */
16 //-----
17 Static Function STDRLSGetModel()
18
19 Local oMasterStr      := Nil          // Estrutura Master
20 Local oGridStr        := Nil          // Estrutura Grid
21
22 If oModel == Nil
23   /*
24     Monta estrutura de tabela
25   */
26   oMasterStr := CreateMasterStruct()
27   /*
28
29   Monta estrutura de grid da tabela de Motivo de Desconto (MDU)
30   */
31   oGridStr := CreateGridStruct()
32
33   /*
34     Instacia Objeto
35   */
36   oModel := MPFormModel():New( 'MBR' )
37
38   oModel:AddFields( "MBR_MASTER" , Nil , oMasterStr )
39
40   oModel:AddGrid( "MBR_GRID"      ,      "MBR_MASTER"      , oGridStr )
41
42   oModel:SetRelation( "MBR_GRID" , { { "MBR_FILIAL", 'xFilial( "MBR" )' } , { "MBR_ITEM", "MBR_ITEM" } } , 'MBR_FILIAL + MBR_ITEM' )
43
44   oModel:SetPrimaryKey( { "MBR_FILIAL" , "MBR_ITEM" } )
45
46   oModel:GetModel( "MBR_GRID" ):SetOptional( .T. )
47
48   oModel:SetOperation( 3 )
49
50   oModel:Activate()
51
52 EndIf

```

```

53
54 Return oModel
55
56
57 //-----
58 /*{Protheus.doc} CreateMasterStruct
59 Cria estrutura do Model Master
60
61 @param
62 @return oStruct      Retorna estrutura do Model Master
63 @obs
64 @sample
65 */
66 //-----
67 Static Function CreateMasterStruct()
68
69 Local oStruct      := FWFormModelStruct().New()    // Estrutura
70 Local cX2Unico     := "MBR_FILIAL+MBR_MOTIVO+MBR_CODIGO+MBR_NUMORC+MBR_DOC+MBR_SERIE+MBR_PROD+MBR_ITEM+MBR_PDV" //X2_UNICO da tabela MBR
71
72 If ExistFunc('FWX2Unico')
73     cX2Unico := FWX2Unico("MBR")
74 EndIf
75
76 oStruct:AddTable(
77     FWX2CHAVE(),
78     StrTokArr( cX2Unico, '+' ),
79     FWX2Nome("MBR")
80 );
81 oStruct:AddField(
82     STR0001,
83     STR0001,
84     "MBR_FILIAL",
85     "C",
86     FWSizeFilial(),
87     0,
88     Nil,
89     Nil,
90     Nil,
91     Nil,
92     Nil,
93     NIL,
94     NIL,
95 );
96
97 oStruct:AddField(
98     STR0002,
99     STR0002,
100    "MBR_ITEM",
101    "C",
102    TamSX3("MBR_ITEM")[1],
103    0,
104    Nil

```

```

105 Nil , ; // [08] Code-block de validacao When do campo
106 Nil , ; // [09] Lista de valores permitido do campo
107 Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
108 Nil , ; // [11] Code-block de inicializacao do campo
109 NIL , ; // [12] Indica se trata-se de um campo chave
110 NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
111 ) // [14] Indica se o campo e virtual
112
113 Return oStruct
114
115
116 //-----
117 /*{Protheus.doc} CreateGridStruct
118 Cria estrutura do Model Grid
119
120 @param
121 @return oStruct Retorna estrutura do Model Grid
122 @obs
123 @sample
124 */
125 //-----
126 Static Function CreateGridStruct()
127
128 Local oStruct := FWFormModelStruct():New() // Estrutura
129 Local cX2Unico := "MBR_FILIAL+MBR_MOTIVO+MBR_CODIGO+MBR_NUMORC+MBR_DOC+MBR_SERIE+MBR_PROD+MBR_ITEM+MBR_PDV" //X2_UNICO da tabela MBR
130
131 If ExistFunc('FWX2Unico')
132 cX2Unico := FWX2Unico("MBR")
133 EndIf
134
135 oStruct:AddTable(
136 FWX2CHAVE() , ; // [01] Alias da tabela
137 StrTokArr( cX2Unico, '+' ) , ; // [02] Array com os campos que correspondem a primary key
138 FWX2Nome("MBR") ) // [03] Descrição da tabela
139
140 //Carrega informacoes de campos
141 DbSelectArea("SX3")
142 SX3->(DbSetOrder(1))
143 SX3->(DbSeek("MBR"))
144
145 While !SX3->(EOF()) .AND. SX3->X3_ARQUIVO == "MBR"
146
147 oStruct:AddField(
148 AllTrim( X3Titulo() ) , ; // [01] Titulo do campo
149 AllTrim( X3Descric() ) , ; // [02] ToolTip do campo
150 AllTrim( SX3->X3_CAMPO ) , ; // [03] Id do Field
151 SX3->X3_TIPO , ; // [04] Tipo do campo
152 SX3->X3_TAMANHO , ; // [05] Tamanho do campo
153 SX3->X3_DECIMAL , ; // [06] Decimal do campo
154 Nil , ; // [07] Code-block de validacao do campo
155 Nil , ; // [08] Code-block de validacao When do campo
156 StrTokArr( AllTrim( X3CBox() ),',' ) , ; // [09] Lista de valores permitido do campo

```

```

157 Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
158 Nil , ; // [11] Code-block de inicializacao do campo
159 NIL , ; // [12] Indica se trata-se de um campo chave
160 NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
161 ( SX3->X3_CONTEXT == 'V' ) ) // [14] Indica se o campo e virtual
162
163 SX3->(DbSkip())
164
165 EndDo
166
167 Return oStruct
168
169
170 //-----
171 /*{Protheus.doc} STDAddReason
172 Function Motivo de Desconto
173
174 @param nItemLine Linha do Item na Venda
175 @param cItemCod Codigo do Item
176 @param nAmount Quantidade
177 @param cCodCli Código do Cliente
178 @param cStoreCli Loja do Cliente
179 @param cSeller Codigo do Vendedor
180 @param dDate Data
181 @param cCodReason Codigo do Motivo de Venda Perdida
182 @param cJustify String contendo justificativa de perda
183 @param cL1_Num caractere, valor do campo SL2->L2_NUM
184 @param cL1_Serie caractere, valor do campo SL2->L2_SERIE
185 @param cL1_Doc caractere, valor do campo SL2->L2_DOC
186
187 @return lRet Retorna se adicionou no Grid
188 @obs
189 @sample
190 */
191 //-----
192 Function STDAddLostSale( nItemLine , cItemCod , nAmount , cCodCli , ;
193 cStoreCli , cSeller , dDate , cCodReason , ;
194 cJustify , cL1_Num , cL1_Serie , cL1_Doc )
195
196 Local oModelGrid := Nil // Componente Grid do Motivo de Venda Perdida
197 Local aBusca := {} // array para busca do Item na MBR
198 Local lFirstItem := .F. // Indica se primeiro item
199 Local lRet := .F. // Retorna se adicionou motivo de desconto
200 Local nLine := 0 // Quantidade de linhas do grid
201
202 Default nItemLine := 0
203 Default cItemCod := ""
204 Default nAmount := 1
205 Default cCodCli := ""
206 Default cStoreCli := ""
207 Default cSeller := ""
208 Default dDate := CToD("")

```

```

209 Default cCodReason      := ""
210 Default cJustify        := ""
211 Default cL1_Num         := ""
212 Default cL1_Serie       := ""
213 Default cL1_Doc         := ""
214
215 ParamType 0 Var nItemLine AS Numeric      Default 0
216 ParamType 1 Var cItemCod  AS Character   Default ""
217 ParamType 2 Var nAmount   AS Numeric      Default 1
218 ParamType 3 Var cCodCli   AS Character   Default ""
219 ParamType 4 Var cStoreCli AS Character   Default ""
220 ParamType 5 Var cSeller   AS Character   Default ""
221 ParamType 6 Var dDate     AS Date        Default CToD("")
222 ParamType 7 Var cCodReason AS Character   Default ""
223 ParamType 8 Var cJustify  AS Character   Default ""
224 ParamType 9 Var cL1_Num  AS Character   Default ""
225 ParamType 10 Var cL1_Serie AS Character   Default ""
226 ParamType 11 Var cL1_Doc AS Character   Default ""
227
228
229 If oModel == Nil
230     STDRLSGetModel() // Cria o Model
231 EndIf
232
233 oModelGrid := oModel:GetModel("MBR_GRID")
234
235 AAdd( aBusca, {"MBR_ITEM", CValToChar(nItemLine)} )
236
237 If oModelGrid:SeekLine( aBusca )
238     If !( oModelGrid:IsDeleted() )
239         oModelGrid:DeleteLine()
240     EndIf
241 EndIf
242
243 lFirstItem := ( oModelGrid:Length() == 1 ) .AND. Empty(oModelGrid:GetValue("MBR_ITEM"))
244
245 If lFirstItem
246     lRet := .T.
247 Else
248     nLine := ( oModelGrid:Length() ) + 1
249     lRet := oModelGrid:AddLine() == nLine
250 EndIf
251
252 If lRet
253
254     oModelGrid:SetValue( "MBR_FILIAL" , xFilial("MBR") )
255     oModelGrid:SetValue( "MBR_CODIGO" , GetSxENum("MBR","MBR_CODIGO" ) )
256     oModelGrid:SetValue( "MBR_ITEM" , CValToChar(nItemLine) )
257     oModelGrid:SetValue( "MBR_PROD" , cItemCod )
258     oModelGrid:SetValue( "MBR_QUANT" , nAmount )
259     oModelGrid:SetValue( "MBR_CODCLI" , cCodCli )
260     oModelGrid:SetValue( "MBR_VEND" , cSeller )

```

```

261 oModelGrid:SetValue( "MBR_LOJA" , cStoreCli )
262 oModelGrid:SetValue( "MBR_MOTIVO" , cCodReason )
263 oModelGrid:SetValue( "MBR_EMISSA" , dDate )
264 oModelGrid:SetValue( "MBR_SITUA" , "00" )
265 oModelGrid:SetValue( "MBR_ORIGEM" , "POS" )
266 oModelGrid:SetValue( "MBR_PDV" , STFGetStation("PDV") )
267 oModelGrid:SetValue( "MBR_ESTACA" , STFGetStation("CODIGO") )
268 oModelGrid:SetValue( "MBR_NUMORC" , Iif(!Empty(cL1_Num) , cL1_Num , STDGPBasket("SL1","L1_NUM" )) )
269 oModelGrid:SetValue( "MBR_DOC" , Iif(!Empty(cL1_Doc) , cL1_Doc , STDGPBasket("SL1","L1_DOC" )) )
270 oModelGrid:SetValue( "MBR_SERIE" , Iif(!Empty(cL1_Serie) , cL1_Serie , STDGPBasket("SL1","L1_SERIE" )) )
271 oModelGrid:SetValue( "MBR_JUSTIF" , cJustify )
272
273
274 EndIf
275
276 Return lRet
277
278
279 //-----
280 /*/{Protheus.doc} STDRLSFree
281 Limpa variáveis
282
283 @param
284 @return Nil
285 @obs
286 @sample
287 */
288 //-----
289 Function STDRLSFree()
290
291 If oModel <> Nil
292     FreeObj(oModel)
293 EndIf
294
295 Return Nil
296
297
298 //-----
299 /*/{Protheus.doc} STDRFDGet
300 Busca Motivos de Desconto Cadastrados
301
302 @param
303 @return aReasons Retorna motivos de desconto cadastrados
304 @obs
305 @sample
306 */
307 //-----
308 Function STDRFLSGet()
309
310 Local aReasons := {} // Retorno funcao
311 Local aArea := GetArea() // Armazena area corrente
312

```

```

313 DbSelectArea("MBQ")
314 DbSetOrder(1) // MDT_FILIAL + MDT_CODIGO
315 //MBQ->(DbGoTop()) //TODO
316 If MBQ->(DbSeek( xFilial("MBQ") )) // Busca aproximada
317     While !EOF() .AND. xFilial("MBQ") == MBQ->MBQ_FILIAL
318         AADD( aReasons , { MBQ->MBQ_CODVEP , MBQ->MBQ_DSCVEP } )
319     MBQ->(DbSkip())
320 EndDo
321 EndIf
322
323 RestArea(aArea)
324
325 Return(aReasons)
326
327
328 //-----
329 /*{Protheus.doc} STDRecLostSale
330 Grava Motivos de Venda Perdida
331
332 @return Nil
333 @obs
334 @sample
335 */
336 //-----
337 Function STDRecLostSale()
338
339 Local   oModelGrid      := NIL           // Componente Grid MBR
340 Local   aArea            := GetArea()     // Armazena alias corrente
341 Local   nI               := 0            // Contador
342 Local   nJ               := 0            // Contador
343 Local   aStruct          := {}           // Armazena estrutura de dados do modelo
344
345 If oModel <> NIL
346
347     oModelGrid := oModel:GetModel("MBR_GRID")
348     aStruct     := oModelGrid:GetStruct():aFields
349
350     DbSelectArea("MBR")
351
352     For nI := 1 To oModelGrid:Length()
353
354         RecLock("MBR",.T.)
355
356         oModelGrid:GoLine(nI)
357
358         For nJ := 1 To Len(aStruct)
359             REPLACE MBR->&(aStruct[nJ][3]) WITH oModelGrid:GetValue(aStruct[nJ][3])
360         Next nJ
361
362         MsUnlock()
363
364         STFSLICreate( Nil , "UP" , "UP" , "NOVO" , ;

```

```
365      Nil      , Nil      , Nil      , Nil      , ;
366      "MBR"    , MBR->(RecNo()) , Nil      )
367
368      Next nI
369
370 EndIf
371
372 RestArea(aArea)
373
374 Return Nil
```

Componentes/ReasonForLostSale/STWReasonForLostSale.PRW

187 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWREASONFORLOSTSALE.CH"
4
5
6 //-----
7 /*{Protheus.doc} STWRsnLstSl
8 Motivo de Venda Perdida
9
10 @param  nItemLine      Linha do Item na Venda
11 @param  oReasons       Objeto motivo de venda perdida
12 @param  cJustific      , caractere, texto da justificativa de cancelamento
13 @return Nil
14 @obs
15 @sample
16 */
17 //-----
18 Function STWRsnLstSl( nItemLine , oReasons, cJustific )
19
20 Local cItemCod      := ""           //Codigo Produto
21 Local nAmount       := 1           //Quantidade
22 Local cCodCli       := ""          //Codigo do Cliente
23 Local cStoreCli     := ""          //Loja do Cliente
24 Local cSeller       := ""          //Codigo Vendedor
25 Local dDate         := dDataBase   //Data da nao venda
26 Local nPosSel       := 0           //Posicao do objeto selecionado
27 Local cCodReason    := ""          //Codigo do motivo
28 Local aRsnCanIt     := {}          //array para receber o objeto oReasons e justificativa
29
30 Default nItemLine   := 0           //Linha do Item na Venda
31 Default oReasons    := Nil         //objeto motivo da venda
32 Default cJustific   := ""          //justificativa
33
34 ParamType 0 Var nItemLine As Numeric Default 0
35
36 If SuperGetMV ("MV_LJMVPE",,.F.) .AND. oReasons <> Nil // Usa Movimento de Venda Perdida?
37
38   If !IsInCallStack("STWRsnLCanCup")
39
40     aRsnCanIt := IIF(Existfunc("STIGTRsnLstIt"), STIGTRsnLstIt(),{})
41
42     If Len(aRsnCanIt)> 0
43       oReasons := aRsnCanIt[1] // objeto motivo de perda da venda
44       cJustific := aRsnCanIt[2] // justificativa
45     Endif
46
47   Endif
48
49   nPosSel := oReasons:GetPos()
50   If nPosSel > 0
51     cCodReason := SubStr(oReasons:aItems[nPosSel],1,TamSX3("MBR_MOTIVO")[1])
52   EndIf

```

```

53
54
55   If nItemLine > 0 .AND. !Empty(cCodReason)
56
57       cItemCod      := STBTaxRet( nItemLine , "IT_PRODUTO" )
58       nAmount       := STBTaxRet( nItemLine , "IT_QUANT" )
59       cSeller        := STDGPBasket("SL2","L2_VEND",nItemLine)
60       cCodCli        := STDGPBasket("SL1","L1_CLIENTE")
61       cStoreCli      := STDGPBasket("SL1","L1_LOJA")
62
63       STDAddLostSale(   nItemLine   , cItemCod   , nAmount       , cCodCli       , ;
64                       cStoreCli    , cSeller      , dDate          , cCodReason    , ;
65                       cJustific )
66
67
68   EndIf
69
70   IIF(Existfunc("STISTRsnlstIt"), STISTRsnlstIt(),Nil)
71
72 EndIf
73
74 Return Nil
75 //-----
76 /*{Protheus.doc} STWRsnLCanCup
77 Gravação de Motivo de Venda Perdida para Cancelamento de Cupom de Venda
78
79 @param cL1_Filial, caractere, filial
80 @param cL1_Num   , caractere, numero da venda
81 @return Nil
82 @obs
83 @sample
84 */
85 //-----
86 Function STWRsnLCanCup(cL1_Filial, cL1_Num)
87
88 Local nQtdtem      := STDPBLength("SL2")
89 Local nX           := 1
90 Local aReasons     := {}
91
92 Default cL1_Num    := ""
93 Default cL1_Filial := ""
94
95 If SuperGetMV ("MV_LJMVPE",,.F.) //motivo da venda perdida
96
97     aReasons := Iif(ExistFunc("STIGTRsnCanCup"), STIGTRsnCanCup(), {})
98
99     if Len(aReasons) == 2 .and. !Empty(aReasons[1]) .AND. !Empty(aReasons[2])
100
101         If !Empty(cL1_Filial) .AND. !Empty(cL1_Num)//gravação de motivo da perda de uma venda finalizada
102
103             STWRsnlstVF(aReasons[1],aReasons[2], cL1_Filial, cL1_Num )
104

```

```

105         Else //gravação de motivo da perda de uma venda em progresso
106
107             For nX := 1 to nQtdtem
108                 STWRsnLstSL( nX , aReasons[1],aReasons[2] )
109             Next nX
110
111         Endif
112     Endif
113
114     STDRecLostSale()
115
116     Iif(ExistFunc("STISTRsnCanCup"), STISTRsnCanCup(), Nil)
117
118 Endif
119
120 Return
121
122
123 /*/{Protheus.doc} STWRsnlstVF
124     Rotina para gravar motivo de perda da venda de um cupom finalizado
125
126     @type Function
127     @param oReasons , object , contem lista de motivo de perda de venda
128     @param cJustific , caractere, texto complementar do motivo da perda de venda
129     @param cL1_Filial, caractere, filial
130     @param cL1_Num , caractere, numero da venda
131
132     @example
133     (examples)
134     @see (links_or_references)
135     /*/
136 Static Function STWRsnlstVF( oReasons,cJustific,cL1_Filial, cL1_Num )
137 Local nPosSel      := 0
138 Local cCodReason   := ""
139
140 nPosSel := oReasons:GetPos()
141
142 If nPosSel > 0
143     cCodReason := SubStr(oReasons:aItems[nPosSel],1,TamSX3("MBR_MOTIVO")[1])
144 EndIf
145
146 SL2->(dbSetOrder(1)) //L2_FILIAL+L2_NUM+L2_ITEM+L2_PRODUTO
147 SL2->(dbSeek(cL1_Filial + cL1_Num))
148
149 While SL2->(!EOF()) .And. SL2->L2_FILIAL == cL1_Filial .And. SL2->L2_NUM == cL1_Num
150     STDAddLostSale( Val(SL2->L2_ITEM), SL2->L2_PRODUTO , SL2->L2_QUANT , SL1->L1_CLIENTE , ;
151         SL1->L1_LOJA , SL2->L2_VEND , SL2->L2_EMISSAO , cCodReason , ;
152         cJustific , SL2->L2_NUM , SL2->L2_SERIE , SL2->L2_DOC )
153
154     SL2->(dbSkip())
155 EndDo
156

```

```
157 Return
158
159
160 /*/{Protheus.doc} STWVldRsnInf
161     (Rotina para certificar se o Motivo da Perda de Venda foi informado)
162     @type Function
163     @param oReasons, object, objeto TList do Motivo da Perda de Venda
164     @param cJustific, caractere, justificativa
165     @return lRet, lógico, .T. se foi informado OU não está parametrizado
166 */
167 Function STWVldRsnInf(oReasons, cJustific)
168 Local lRet      := .T.
169 Local nPosSel   := 0
170
171 Default oReasons := Nil
172 Default cJustific := ""
173
174 If !Empty(oReasons)
175     nPosSel := oReasons:GetPos()
176     If Empty(nPosSel)
177         STFMessage("STWVldRsnInf", "POPUP", STR0001)//"É necessário selecionar o motivo de cancelamento.
178         STFShowMessage("STWVldRsnInf")
179         lRet:= .F.
180     ElseIf Empty(cJustific)
181         STFMessage("STWVldRsnInf", "POPUP", STR0002)//"É necessário informar a justificativa de cancelamento.
182         STFShowMessage("STWVldRsnInf")
183         lRet:= .F.
184     Endif
185 Endif
186
187 Return lRet
```

Componentes/ReceiptTitle/STDRecSelCust.prw

170 linhas após remoção de metadados de autoria/data

```

1 #include 'totvs.ch'
2 #include 'STDRecSelCust.ch'
3
4 Static aCustSele := {}
5
6 //-----
7 /*{Protheus.doc} STDRSelCus()
8 Retorna relacao de cadastro de cliente pela busca por CGC
9
10 @param cCGC - recebe CPF ou CNPJ do cliente para busca de cadastro
11 @return aRet
12 @obs
13 @sample
14 */
15 //-----
16 Function STDRSelCus(cCGC)
17
18 Local aAreaSA1 := SA1->(GetArea())
19 Local aRet := {}
20 Local nMVLJPEATU := SuperGetMV("MV_LJPEATU",,1) // Pesquisa retaguarda 0=Desabilitado, faz pesquisa Local. 1=Pesquisa retaguarda se falhar local. 2=Pesquisa somente
    retaguarda
21 Local aRetCli := {} // Retorno da busca pelo cliente na retaguarda
22 Local nX := 1 // Contador do For
23
24 DbSelectArea("SA1")
25 SA1->(DbSetOrder(3)) //FILIAL + CGC
26
27 // verifica primeiro na Retaguarda
28 If nMVLJPEATU <> 0
29
30     STFMessage(ProcName(),"RUN",STR0001) //"Pesquisando cliente no Servidor. Aguarde..."
31     STFShowMessage(ProcName())
32     LjGrvLog( "Recebimento de Titulo", "Pesquisando cliente na Retaguarda", cCGC )
33     CursorWait()
34
35     aRetCli := STBGetCust(xFilial("SA1"),cCGC)
36
37     If ValType(aRetCli) != "A"
38         LjGrvLog( "Recebimento de Titulo", "Não conseguiu Pesquisar na retaguarda ", cCGC )
39     ElseIf Len(aRetCli) > 0
40
41         For nX :=1 To Len(aRetCli)
42             AADD(aRet, aRetCli[nX][1] + " / " + aRetCli[nX][2] + " / " + aRetCli[nX][3] + " / " +;
43                 Iif(Len(aRetCli[nX])>4,aRetCli[nX][5],"") )
44         Next nX
45
46     EndIf
47
48     CursorArrow()
49
50 ElseIf SA1->(DbSeek(xFilial("SA1")+AllTrim(cCGC))) .AND. nMVLJPEATU <> 2 // Se nao achar na Retaguarda busca no PDV
51

```

```

52   LjGrvLog( "Recebimento de Titulo", "Pesquisando cliente no PDV ", cCGC )
53   While SA1->(!EOF()) .AND. SA1->A1_FILIAL+AllTrim(SA1->A1_CGC) == xFilial("SA1")+AllTrim(cCGC)
54       AADD(aRet, SA1->A1_COD + " / " + SA1->A1_LOJA + " / " + SA1->A1_NOME + " / " + SA1->A1_CGC )
55       SA1->( dbSkip() )
56   EndDo
57
58
59 EndIf
60
61 STDSCustSele( aRet )
62
63 RestArea(aAreaSA1)
64
65 Return aRet
66
67 //-----
68 /*{Protheus.doc} STDSCustSele()
69 Responsável por Atualizar os valores de aCustSele
70
71 @param   aCadCli - Array com Codigo e Loja do Cliente que atualizará o array estático aCustSele
72 @obs
73 @sample
74 */
75 //-----
76 Function STDSCustSele( aCadCli )
77
78 Default aCadCli := {}
79
80 aCustSele := aCadCli
81
82 Return
83
84 //-----
85 /*{Protheus.doc} STDGCustTt()
86 Retorna array com codigo e loja do cliente
87
88 @param   aCadLoj array com Codigo e Loja do Cliente
89 @return  aRet
90 @obs
91 @sample
92 */
93 //-----
94 Function STDGCustTt(aCodLoj)
95
96 Local aRet      := {}
97 Local aAuxCodLoj := {}
98
99 Default aCodLoj := {}
100
101 aAuxCodLoj := STDStr2Arr( aCodLoj, " / " )
102 AADD(aRet, PadR(aAuxCodLoj[01],TamSX3( "A1_COD" ) [1], " "))
103 AADD(aRet, PadR(aAuxCodLoj[02],TamSX3( "A1_LOJA" ) [1], " "))

```

```

104
105 Return aRet
106
107 //-----
108 /*{Protheus.doc} STDGCustSele()
109 Responsável por retornar um array com o valor de aCustSele
110
111 @return  aCustSele
112 @obs
113 @sample
114 */
115 //-----
116 Function STDGCustSele()
117 Return aCustSele
118
119 //-----
120 /*{Protheus.doc} STDCallCustTit()
121 Responsável por Chamar a tela se seleção de título conforme a loja do cliente que foi selecionada
122
123 @param    nPos    - Posicao do Cliente selecionado no array aCadCli
124 @return
125 @obs
126 @sample
127 */
128 //-----
129 Function STDCallCustTit(nPos)
130
131 local aAux := {}
132 Local aCadCli := STDGCustSele() // Recupera conteudo a aCadCli
133
134 If Len(aCadCli) > 0
135     aAux := STDGCustTt(aCadCli[nPos])
136 EndIf
137
138 STIFindTit( aAux )
139
140 Return
141
142 //-----
143 /*{Protheus.doc} STDStr2Arr
144 Retorna array com código e loja do cliente com espaços em branco no código
145
146 @param    cText, Caracter, String matriz com as informações que serão separadas
147 @param    cToken, Caracter, Separador de conteúdo
148 @return    Array, Retorna informações sobre os clientes buscado
149 */
150 //-----
151
152 Function STDStr2Arr(cText,cToken)
153
154 Local aRet      := {} //Variavel de Retorno
155 Local nEnd      := 1  //Variavel de controle para fim da String

```

```
156 Local nStart      := 1      //Variavel de controle para inicio da String
157
158 Default cText      := ""
159 Default cToken      := ";"
160
161 While nEnd < Len(cText) .And. nEnd > 0
162     nEnd := At(cToken,cText,nStart)
163     If nEnd == 0
164         nEnd := Len(cText)
165     EndIf
166     aAdd(aRet,substr(cText,nStart,nEnd - nStart ))
167     nStart := nEnd + Len(cToken)
168 End
169
170 Return aRet
```

Componentes/ReceiptTitle/STDReceiptTitle.prw

113 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "STPOS.CH"
3 #INCLUDE "AUTODEF.CH"
4 #INCLUDE "MSOBJECT.CH"
5
6 //-----
7 /*{Protheus.doc} STDRecSE5
8 Função responsável por criar os registros na tabela SE5 para serem exibidos na conferência de
9 caixa ao encerrar o caixa.
10
11 @param      aTit      - Lista com todos os títulos selecionados e baixados.
12             aForm      - Lista com todas as formas de pgto utilizada na baixa.
13             lEstorno    - é estorno? (.T. = estorno/ .F. = baixa)
14 @return      Nil
15 @obs
16 @sample
17 */
18 //-----
19 Function STDRecSE5(aTit, aForm, lEstorno, lOffLine)
20 Local nI      := 1
21 Local cCashier := xNumCaixa()
22 Local cNumMov  := STDNumMov()
23 Local lE5FORMAPG:= SE5->(FieldPos("E5_FORMAPG")) > 0
24 Local lE5ORIGEM := SE5->(FieldPos("E5_ORIGEM")) > 0
25 Local lE5NUMMOV := SE5->(FieldPos("E5_NUMMOV")) > 0
26 Local cNatRece := IIf( ExistFunc("LjMExePara"), LjMExeParam("MV_NATRECE"), &(SuperGetMV("MV_NATRECE")))
27 Local nOpcConf := SuperGetmv("MV_LJOPCON",,2) //1-Conferência em uma só forma (Recebimentos); 2-Conferência por forma de pagamento
28
29 Default aTit      := {}
30 Default aForm      := {}
31 Default lEstorno    := .F.
32 Default lOffLine    := .F. //Processo offline
33
34 LjGrvLog( "Recebimento_Titulo", "Executando STDRecSE5. MV_NATRECE:", cNatRece)
35 LjGrvLog( "Recebimento_Titulo", "Executando STDRecSE5. MV_LJOPCON:", nOpcConf)
36
37 SA6->(dbSetOrder(2))
38 SA6->(dbSeek(xFilial("SA6") + Upper(cCashier)))
39
40 //Foi serpado a geração dos registros na tabela SE5 para que seja possível na conferência de caixa
41 //exibir os recebimentos ou por formas de pagamentos ou por apenas uma forma de pagamento (esta última
42 //ficando igual a red.z)
43 //Este SE5 não deverá subir para a retaguarda.
44 If nOpcConf == 2 .And. lE5FORMAPG
45
46     For nI := 1 To Len(aForm)
47
48         If Reclock("SE5",.T.)
49             REPLACE SE5->E5_FILIAL WITH xFilial("SE5")
50             REPLACE SE5->E5_DATA WITH dDataBase
51             REPLACE SE5->E5_TIPO WITH "FI"
52             REPLACE SE5->E5_BANCO WITH cCashier

```

```

53      REPLACE SE5->E5_AGENCIA WITH SA6->A6_AGENCIA
54      REPLACE SE5->E5_CONTA   WITH SA6->A6_NUMCON
55      REPLACE SE5->E5_RECPEG WITH Iif(lEstorno , "P", "R")
56      REPLACE SE5->E5_HISTOR WITH Iif(lEstorno,"Cancel de baixa","Receb Titulo")+IIF(lOffLine, "OffLine - ", " - ") +aForm[nI][3]+"/"+aForm[nI][4]+"/"+aForm[nI][5]
57      REPLACE SE5->E5_TIPODOC WITH Iif(lEstorno,"ES","VL")
58      If lE5FORMAPG
59          REPLACE SE5->E5_FORMAPG WITH AllTrim(aForm[nI][1])
60      EndIf
61      If lE5ORIGEM
62          REPLACE SE5->E5_ORIGEM WITH "LOJXREC"
63      EndIf
64      REPLACE SE5->E5_VALOR   WITH aForm[nI][2]
65      REPLACE SE5->E5_DTDIGIT WITH dDataBase
66      REPLACE SE5->E5_DTDISPO WITH SE5->E5_DATA
67      REPLACE SE5->E5_NATUREZ WITH cNatRece
68      If lE5NUMMOV
69          Replace SE5->E5_NUMMOV WITH cNumMov
70      EndIf
71      REPLACE SE5->E5_PREFIXO WITH aForm[nI][3]
72      REPLACE SE5->E5_NUMERO  WITH aForm[nI][4]
73      SE5->(dbCommit())
74      SE5->(MsUnLock())
75  EndIf
76  Next
77
78 Else
79
80  For nI := 1 To Len(aTit)
81
82      If Reclock("SE5",.T.)
83          REPLACE SE5->E5_FILIAL WITH xFilial("SE5")
84          REPLACE SE5->E5_DATA   WITH dDataBase
85          REPLACE SE5->E5_TIPO   WITH "FI"
86          REPLACE SE5->E5_BANCO  WITH cCashier
87          REPLACE SE5->E5_AGENCIA WITH SA6->A6_AGENCIA
88          REPLACE SE5->E5_CONTA   WITH SA6->A6_NUMCON
89          REPLACE SE5->E5_RECPEG WITH Iif(lEstorno , "P", "R")
90          REPLACE SE5->E5_HISTOR WITH Iif(lEstorno,"Cancel de baixa","Receb Titulo")+IIF(lOffLine, "OffLine - ", " - ") +aTit[nI][2]+"/"+aTit[nI][3]+"/"+aTit[nI][4]
91          REPLACE SE5->E5_TIPODOC WITH Iif(lEstorno,"ES","VL")
92          If lE5FORMAPG .And. Len(aForm) > 0 // Colocamos esse tratamento, pois na retaguarda mesmo com o parâmetro igual a 1 estava realizando a gravação deste campo.
93              REPLACE SE5->E5_FORMAPG WITH AllTrim(aForm[nI][1])
94          EndIf
95          If lE5ORIGEM
96              REPLACE SE5->E5_ORIGEM WITH "LOJXREC"
97          EndIf
98          REPLACE SE5->E5_VALOR   WITH aTit[nI][10]
99          REPLACE SE5->E5_DTDIGIT WITH dDataBase
100         REPLACE SE5->E5_DTDISPO WITH SE5->E5_DATA
101         REPLACE SE5->E5_NATUREZ WITH cNatRece
102         If lE5NUMMOV
103             Replace SE5->E5_NUMMOV WITH cNumMov
104         EndIf

```

```
105         REPLACE SE5->E5_PREFIXO WITH aTit[nI][2]
106         REPLACE SE5->E5_NUMERO  WITH aTit[nI][3]
107         SE5->(dbCommit())
108         SE5->(MsUnLock())
109     EndIf
110 Next
111 EndIf
112
113 Return Nil
```

Componentes/ReceiptTitle/STWReceiptTitle.PRW

2466 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "STPOS.CH"
3 #INCLUDE "STWRECEIPTTITLE.CH"
4 #INCLUDE "AUTODEF.CH"
5 #INCLUDE "MSOBJECT.CH"
6
7 Static cCodCliTit      := ""
8 Static cCodLojaTit    := ""
9
10 Function STWReceTit()
11 Return; // Funcao apenas para poder compilar no robo
12
13 Class STWReceiptTitle
14
15     Data cOperation      // OP_BAIXA("1") - Baixa de titulos / OP_ESTOR("2") - Estorno de baixa
16     Data cPrefix        // Prefixo do titulo
17     Data cNumber        // Numero do titulo
18     Data cParcel        // Parcela do titulo
19     Data aListTitles    // Array com a lista de titulos em aberto
20     Data oCustomer      // Cliente a receber
21     Data oPayment       // Objeto de controle de pagamento
22     Data lIsContingency // Indica se esta trabalhando por medida de contingencia (sem conexao com a retaguarda)
23     Data nValueContingency // Valor de pagamento (somente para medida de contingencia)
24     Data cCode          // Codigo
25     Data cBranch        // Loja
26     Data cName          //Nome do Cliente
27     Data cCnpjCpf       // Cnpj/Cpf
28     Data nTotNcc        // Valor de NCC utilizada (para estorno)
29     Data dDtRec         //Data do Recebimento
30     Data cHrRec         // Hora do Recebimento
31
32
33     // Metodos do controle de recebimento
34     Method STWReceiptTitle( cCode , cBranch, cCnpjCpf, cPrefix ,;
35                             cNumber , cParcel , cName )
36
37     Method LoadTitles()
38     Method DropTitles(oMdlGrd)
39     Method GetTotal(lExtrato)
40     Method GetTotNCC()
41     Method SetTotNCC()
42     Method Print(oMdlGrd)
43     Method GetReceiptArray(lExtrato)
44     Method ExtractPrint()
45     Method GetContingencyMode()
46     Method SetContingencyMode(lValue)
47     Method GetCustomer()
48     Method SetCustomer(oValue)
49     Method GetPayment()
50     Method SetPayment(oValue)
51     Method GetReverseMode()
52     Method SetReverseMode(lValue)
53     Method CountParcels()

```

```

53 Method ReverseDropTitles()
54 Method GetValueContingency()
55 Method SetValueContingency(nValue)
56 Method VldCustomer(cCodCli, cLojCli, cCPF)
57
58 // Metodos de parcelas
59 Method GetParcSelected(nI)
60 Method SetParcSelected(nI, lValue)
61 Method GetParcInterest(nI)
62 Method SetParcInterest(nI, nValue)
63 Method GetParcDiscount(nI)
64 Method GetParcIncrease(nI)
65 Method SetParcDiscount(nI, nValue)
66 Method GetParcMulct(nI)
67 Method SetParcMulct(nI, nValue)
68 Method GetParcTotal(nI)
69 Method SetParcTotal(nI, nValue)
70 Method ParcRecalculateTotal(nI)
71 Method GetParcOrig(nI)
72 Method GetParcValor(nI)
73
74 EndClass
75
76
77 //-----
78 /*{Protheus.doc} STWReceiptTitle
79 Objeto responsavel pelo controle de recebimento de títulos / estornos de recebimentos
80 @return Objeto STBReceiptTitle - Controle de recebimentos
81 @obs
82 @sample
83 */
84 //-----
85 Method STWReceiptTitle( cCode , cBranch, cCnpjCpf, cPrefix ,;
86                        cNumber, cParcel, cName
87                        ) Class STWReceiptTitle
88
89 Default cCode      := ""
90 Default cBranch    := ""
91 Default cCnpjCpf   := ""
92 Default cPrefix    := ""
93 Default cNumber    := ""
94 Default cParcel    := ""
95 Default cName      := ""
96
97 Self:cOperation    := OP_BAIXA
98
99 //Dados da parcela
100 Self:cPrefix       := cPrefix
101 Self:cNumber       := cNumber
102 Self:cParcel       := cParcel
103
104 //Dados do cliente
105 Self:cCode         := cCode

```

```

105 Self:cBranch      := cBranch
106 Self:cCnpjCpf     := cCnpjCpf
107
108 Self:aListTitles   := {}
109 Self:nValueContingency := 0
110 Self:lIsContingency := .F.
111 Self:nTotNcc       := 0
112 Self:cName         := cName
113
114
115 Self:dDtRec := Ctod("") //Data do Recebimento
116 Self:cHrRec := ""      // Hora do Recebimento
117
118
119 Return Self
120
121
122 //-----
123 /*{Protheus.doc} CountParcels
124 Retorna a quantidade de parcelas
125 @return Retorna a quantidade de parcelas
126 @obs
127 @sample
128 */
129 //-----
130 Method CountParcels() Class STWReceiptTitle
131
132 Local nRetorno := 0 // Retorno da funcao
133
134 If Len(Self:aListTitles) >= 4
135     nRetorno := Len(Self:aListTitles[4])
136 EndIf
137
138 Return nRetorno
139
140
141 //-----
142 /*{Protheus.doc} GetParcSelected
143 Verifica a selecao da parcela
144 @param nI Posicao da parcela no array aListTitles
145 @return Lista de Titulos
146 @obs
147 @sample
148 */
149 //-----
150 Method GetParcSelected(nI) Class STWReceiptTitle
151 Local lValor := .F.
152
153 Default nI := 0
154
155 If nI > 0
156     lValor := Self:aListTitles[4][nI][TIT_SELE]

```

```

157 EndIf
158
159 Return lValor
160
161
162 //-----
163 /*{Protheus.doc} SetParcSelected
164 Altera a selecao da parcela
165 @param  nI      Posicao da parcela no array aListTitles
166 @param  lValue Novo valor
167 @return Nil
168 @obs
169 @sample
170 */
171 //-----
172 Method SetParcSelected(nI, lValue) Class STWReceiptTitle
173
174 Default nI      := 0
175 Default lValue   := .F.
176
177 If nI > 0 .AND. ValType(Self:aListTitles[4][nI][TIT_SELE]) == "L"
178     Self:aListTitles[4][nI][TIT_SELE] := lValue
179 EndIf
180
181 Return Nil
182
183
184 //-----
185 /*{Protheus.doc} SetParcInterest
186 Altera o valor de Juros da parcela
187 @param  nI      Posicao da parcela no array aListTitles
188 @param  nValue Novo valor
189 @return Nil
190 @obs
191 @sample
192 */
193 //-----
194 Method SetParcInterest(nI, nValue) Class STWReceiptTitle
195
196 Default nI      := 0
197 Default nValue   := 0
198
199 If nI > 0
200     Self:aListTitles[4][nI][TIT_JURO] := nValue
201 EndIf
202
203 Return Nil
204
205
206 //-----
207 /*{Protheus.doc} GetParcInterest
208 Retorna o valor de Juros da parcela

```

```

209 @param  nI      Posicao da parcela no array aListTitles
210 @return  Valor de Juros da parcela
211 @obs
212 @sample
213 */
214 //-----
215 Method GetParcInterest(nI) Class STWReceiptTitle
216
217 Local nValor := 0
218
219 Default nI      := 0
220
221 If nI > 0
222     nValor := Self:aListTitles[4][nI][TIT_JURO]
223
224 EndIf
225
226 Return nValor
227 //-----
228 /*{Protheus.doc} GetParcOrig
229 Retorna o valor de Juros da parcela
230 @param  nI      Posicao da parcela no array aListTitles
231 @return  Valor original do titulo
232 */
233 Method GetParcOrig(nI) Class STWReceiptTitle
234 Local nReturn := 0
235 Default nI      := 0
236
237 If nI > 0
238     nReturn := Self:aListTitles[4][nI][TIT_SALD]
239 Else
240     nReturn := 0
241 EndIf
242
243 Return nReturn
244
245
246 //-----
247 /*{Protheus.doc} SetParcMulct
248 Altera o valor de Multa da parcela
249 @param  nI      Posicao da parcela no array aListTitles
250 @param  nValue  Novo valor
251 @return Nil
252 @obs
253 @sample
254 */
255 //-----
256 Method SetParcMulct(nI, nValue) Class STWReceiptTitle
257
258 Default nI      := 0
259 Default nValue   := 0
260

```

```

261 If nI > 0
262     Self:aListTitles[4][nI][TIT_MULT] := nValue
263 EndIf
264
265 Return Nil
266
267
268 //-----
269 /*{Protheus.doc} GetParcMulct
270 Retorna o valor de Multa da parcela
271 @param  nI      Posicao da parcela no array aListTitles
272 @return  Valor de Multa da parcela
273 @obs
274 @sample
275 */
276 //-----
277 Method GetParcMulct(nI) Class STWReceiptTitle
278 Local nValor := 0
279
280 Default nI      := 0
281
282 If nI > 0
283     nValor := Self:aListTitles[4][nI][TIT_MULT]
284 EndIf
285
286 Return nValor
287
288
289 //-----
290 /*{Protheus.doc} SetParcDiscount
291 Altera o valor de Desconto da parcela
292 @param  nI      Posicao da parcela no array aListTitles
293 @param  nValue  Novo valor
294 @return Nil
295 @obs
296 @sample
297 */
298 //-----
299 Method SetParcDiscount(nI, nValue) Class STWReceiptTitle
300
301 Default nI      := 0
302 Default nValue   := 0
303
304 If nI > 0
305     Self:aListTitles[4][nI][TIT_DESC] := nValue
306 EndIf
307
308 Return Nil
309
310
311 //-----
312 /*{Protheus.doc} GetParcDiscount

```

```

313 Retorna o valor de Desconto da parcela
314 @param  nI      Posicao da parcela no array aListTitles
315 @return  Valor de Desconto da parcela
316 @obs
317 @sample
318 /*/
319 //-----
320 Method GetParcDiscount(nI) Class STWReceiptTitle
321 Local nValor := 0
322
323 Default nI      := 0
324
325 If nI > 0
326     nValor := Self:aListTitles[4][nI][TIT_DESC]
327 EndIf
328
329 Return nValor
330
331 //-----
332 /*/{Protheus.doc} GetParcIncrease
333 Retorna o valor de Acrescimo da parcela
334 @param  nI      Posicao da parcela no array aListTitles
335 @return  Valor de acrescimo da parcela
336 @obs
337 @sample
338 /*/
339 //-----
340 Method GetParcIncrease(nI) Class STWReceiptTitle
341 Local nValor := 0
342
343 Default nI      := 0
344
345 If nI > 0
346     nValor := Self:aListTitles[4][nI][TIT_ACRS]
347 EndIf
348
349 Return nValor
350
351 //-----
352 /*/{Protheus.doc} GetParcTotal
353 Retorna o valor Total da parcela
354 @param  nI      Posicao da parcela no array aListTitles
355 @return  Valor Total da parcela
356 @obs
357 @sample
358 /*/
359 //-----
360 Method GetParcTotal(nI) Class STWReceiptTitle
361
362 Local nValor := 0      //Retorno
363
364 Default nI      := 0

```

```

365
366 If nI > 0
367     nValor := Self:aListTitles[4][nI][TIT_RECE]
368 EndIf
369
370 Return nValor
371
372
373 //-----
374 /*{Protheus.doc} SetParcTotal
375 Altera o valor Total da parcela
376 @param  nI      Posicao da parcela no array aListTitles
377 @param  nValue  Novo valor
378 @return Nil
379 @obs
380 @sample
381 */
382 //-----
383 Method SetParcTotal(nI, nValue) Class STWReceiptTitle
384
385 Default nI      := 0
386 Default nValue  := 0
387
388 If nI > 0
389     Self:aListTitles[4][nI][TIT_RECE] := nValue
390 EndIf
391
392 Return Nil
393
394
395 //-----
396 /*{Protheus.doc} ParcRecalculateTotal
397 Atualiza o valor a receber da parcela calculando multa, juros e desconto
398 @param  nI      Posicao da parcela no array aListTitles
399 @return Nil
400 @obs
401 @sample
402 */
403 //-----
404 Method ParcRecalculateTotal(nI) Class STWReceiptTitle
405
406 Default nI      := 0
407
408 If nI > 0
409     Self:SetParcTotal(nI, Self:aListTitles[4][nI][TIT_VAL0] + Self:aListTitles[4][nI][TIT_MULT] + Self:aListTitles[4][nI][TIT_JUR0] - Self:aListTitles[4][nI][TIT_DESC])
410 EndIf
411
412 Return Nil
413
414
415 /*{Protheus.doc} GetContingencyMode()
416 Verifica se esta trabalhando em modo NORMAL ou em MODO DE CONTINGENCIA

```

```

417 @return .T. Quando esta em MODO DE CONTINGENCIA / .F. Quando estiver em MODO NORMAL
418 @obs
419 @sample
420 /*/
421 //-----
422 Method GetContingencyMode() Class STWReceiptTitle
423 Return Self:lIsContingency
424
425
426 //-----
427 /*/{Protheus.doc} SetContingencyMode()
428 Altera entre modo NORMAL / e MODO DE CONTINGENCIA
429 @param lValue .T. para modo contingencia / .f. para modo normal
430 @return Nil
431 @obs
432 @sample
433 /*/
434 //-----
435 Method SetContingencyMode(lValue) Class STWReceiptTitle
436
437 Default lValue      := .F.
438
439 Self:lIsContingency := lValue
440
441 Return Nil
442
443
444 //-----
445 /*/{Protheus.doc} GetCustomer()
446 Retorna o cliente
447 @return Objeto cliente
448 @obs
449 @sample
450 /*/
451 //-----
452 Method GetCustomer() Class STWReceiptTitle
453 Return Self:oCustomer
454
455
456 //-----
457 /*/{Protheus.doc} SetCustomer()
458 Altera o cliente da operacao
459 @param oValue Novo cliente
460 @return Nil
461 @obs
462 @sample
463 /*/
464 //-----
465 Method SetCustomer(oValue) Class STWReceiptTitle
466
467 Default oValue      := Nil
468

```

```

469 Self:oCustomer := oValue
470
471 Return Nil
472
473
474 //-----
475 /*{Protheus.doc} GetPayment()
476 Retorna o objeto de pagamento
477 @return Objeto de pagamento
478 @obs
479 @sample
480 */
481 //-----
482 Method GetPayment() Class STWReceiptTitle
483 Return Self:oPayment
484
485
486 //-----
487 /*{Protheus.doc} SetPayment()
488 Altera o objeto de pagamento da operacao
489 @param oValue Novo objeto de pagamento
490 @return Nil
491 @obs
492 @sample
493 */
494 //-----
495 Method SetPayment(oValue) Class STWReceiptTitle
496
497 Default oValue      := Nil
498
499 Self:oPayment := oValue
500
501 Return Nil
502
503
504 //-----
505 /*{Protheus.doc} GetReverseMode()
506 Retorna se o MOD0 ESTORNO esta ativado
507 @param oValue Novo objeto de pagamento
508 @return .T. quando estiver em modo ESTORNO / .F. quando estiver em modo RECEBIMENTO
509 @obs
510 @sample
511 */
512 //-----
513 Method GetReverseMode() Class STWReceiptTitle
514 Return (Self:cOperation == OP_ESTOR)
515
516
517 //-----
518 /*{Protheus.doc} GetTotal
519 Alterna entre modo RECEBIMENTO / ESTORNO DE RECEBIMENTO
520 @param lValue .T. para ativar modo de ESTORNO / .F. para ativar o modo RECEBIMENTO

```

```

521 @return Nil
522 @obs
523 @sample
524 /*/
525 //-----
526 Method SetReverseMode(lValue) Class STWReceiptTitle
527
528 Default lValue      := .F.
529
530 If lValue
531     // Define que a operacao realizada sera Estorno
532     Self:cOperation := OP_ESTOR
533 Else
534     // Define que a operacao realizada sera Baixa de Titulos
535     Self:cOperation := OP_BAIXA
536 EndIf
537 Return Nil
538
539 //-----
540 /*/{Protheus.doc} SetTotNCC
541 Seta o valor de NCC utilizada.
542 @param
543 @return nRet      Valor de NCC utilizada
544 @obs
545 @sample
546 /*/
547 //-----
548 Method SetTotNCC(nValor) Class STWReceiptTitle
549 Self:nTotNcc := nValor
550 Return Nil
551
552 //-----
553 /*/{Protheus.doc} GetTotNCC
554 Retorna o valor de NCC utilizada.
555 @param
556 @return nRet      Valor de NCC utilizada
557 @obs
558 @sample
559 /*/
560 //-----
561 Method GetTotNCC() Class STWReceiptTitle
562 Local nRet := Self:nTotNcc
563 Return nRet
564
565
566 //-----
567 /*/{Protheus.doc} GetValueContingency
568 Obtem o valor de contingencia
569 @return Valor de contingencia
570 @obs
571 @sample
572 /*/

```

```

573 //-----
574 Method GetValueContingency() Class STWReceiptTitle
575 Return Self:nValueContingency
576
577
578 //-----
579 /*{Protheus.doc} SetValueContingency
580 Altera o valor de contingencia
581 @param nValue Novo valor de contingencia
582 @return Nil
583 @obs
584 @sample
585 */
586 //-----
587 Method SetValueContingency(nValue) Class STWReceiptTitle
588
589 Default nValue      := 0
590
591 Self:nValueContingency := nValue
592
593 Return Nil
594
595
596 //-----
597 /*{Protheus.doc} GetTotal
598 Retorna o total de todas as parcelas selecionadas
599 @param lExtrato Se é extrato
600 @return Total de todas as parcelas selecionadas
601 @obs
602 @sample
603 */
604 //-----
605 Method GetTotal(lExtrato) Class STWReceiptTitle
606
607 Local nRetorno      := 0    // Retorno do funcao
608 Local nI            := 0    // Contador
609
610 Default lExtrato    := .F.  // Se for extrato nao verifica se a parcela foi selecionada
611
612 If Self:lIsContingency
613     nRetorno := Self:nValueContingency
614 Else
615     For nI := 1 To Len(Self:aListTitles[4])
616         If lExtrato .OR. Self:aListTitles[4][nI][TIT_SELE]
617             nRetorno += Self:aListTitles[4][nI][TIT_RECE]
618         EndIf
619     Next nI
620 EndIf
621
622 Return nRetorno
623
624

```

```

625 //-----
626 /*{Protheus.doc} Print
627 Carrega o comprovante e imprime os documentos
628 @param oMdlGrd - Model principal
629 @return .T. quando impressao realizada com sucesso / .F. caso de falha
630 @obs
631 @sample
632 */
633 //-----
634 Method Print(oMdlGrd) Class STWReceiptTitle
635
636 Local lRetorno           := .F.           // Retorno da funcao
637 Local cTextoComprovante := ""           // Texto completo do comprovante a ser enviado para a impressora
638 Local cTotalizador      := ""           // Totalizador recebimento / estorno de estorno de recebimento de titulos
639 Local aRet              := {}           // Retorno evento
640 Local cForma            := ""           // Forma pgto
641 Local aValues           := Array(17)     // Array com valores da sangria em caso de estorno
642 Local lEmitNfce         := Iif(ExistFunc("LjEmitNfCe"), LjEmitNfCe(), .F.) // Sinaliza se utiliza NFC-e
643 Local lGuil             := SuperGetMV("MV_FTTEFGU",, .T.) // Ativa guilhotina
644 Local oTEF20            := IIF(ValType(STBGetTef()) == '0', STBGetTef(), STBGetChkRet())//Objeto do TEF ativo
645 Local nVlTotal         := 0             // Valor a ser impresso quando estorno
646 Local nRet              := 0
647 Local lSTRECTIT         := ExistBlock("STRECTIT") //ponto de entrada para customizar impressao dos titulos
648 Local nRecVias          := SuperGetMV("MV_RECVIAS",,2) //numero de vias para impressao do comprovante de Recebimento
649 Local cTxtCNF           := ""           //Texto que sera enviado para a impressora nao fiscal
650 Local nI                := 0           // Contador
651 Local lTemTEF           := .F.         // Verifica se foi impresso o comprovante TEF
652 Local aParamPE          := {} //Parametros do ponto de entrada
653 Local nX                := 0
654
655 Default oMdlGrd := Nil
656
657 If Self:GetTotal() > 0
658
659     CursorWait()
660
661     STFMessage( ProcName(0) , "ALERT", STR0023 ) //"Imprimindo comprovantes do recebimento"
662     STFShowMessage(ProcName(0))
663
664     cTextoComprovante := GetTextArray(Self:GetReceiptArray())
665
666     //Ponto de Entrada para customização do comprovante de recebimento de titulos
667     If lSTRECTIT
668         aParamPE := {cTextoComprovante,Self:aListTitles, Self:lIsContingency, Self:cPrefix, ;
669                     Self:cNumber, Self:cParcel, Self:nValueContingency, Self:cCnpjCpf, ;
670                     Self:cCode, Self:cBranch, Self:cName, Self:dDtRec, ;
671                     Self:cHrRec}
672         LjGrvLog("Antes da Chamada do Ponto de Entrada:STRECTIT",aParamPE)
673         cTextoComprovante := ExecBlock( "STRECTIT" ,.F.,.F.,aParamPE )
674         LjGrvLog("Apos a Chamada do Ponto de Entrada:STRECTIT", aParamPE)
675     EndIf
676

```

```

677 If Self:cOperation == OP_BAIXA
678
679     //deve-se enviar o Indice do totalizador pois as impressoras da SIGALOA esperam um valor numérico
680     cTotalizador := STWReceTotalizer("TOTALIZADORES","BAIXA")
681     cTotalizador := Iif(!Empty(cTotalizador),cTotalizador,"01")
682     cForma       := STWReceTotalizer("FORMA", "BAIXA")
683
684     // Emite os documentos na impressora:
685     // 1. Recebimento Nao Fiscal - Totaliza o recebimento no ECF
686     // 2. Se houver TEF, imprime o comprovante vinculado, referente ao Recebimento Nao Fiscal
687     // 3. Relatorio Gerencial das parcelas pagas
688
689     /*
690     Impressao do Comprovante TEF
691     */
692     If STIGetCard() .OR. STBGetCheck()
693         STWPrintComp( oTEF20, oMdlGrd )
694         lTemTEF := .T.
695         lRetorno := .T.
696     EndIf
697
698     If lEmitNFce
699
700         // numero de vias do Comprovante de Recebimento que serao impressas
701         For nI := 1 to nRecVias
702             cTxtCNF := cTextoComprovante
703             //Se for ultima via, nao precisa da quebra de linhas
704             If nI <> nRecVias
705                 cTxtCNF += Replicate( Chr(10),2 )
706             EndIf
707
708             nRet := STWPrintTextNotFiscal(cTxtCNF)
709
710             If lGuil
711                 cTxtCNF := TAG_GUIL_INI+TAG_GUIL_FIM
712                 nRet := STWPrintTextNotFiscal(cTxtCNF)
713             EndIf
714         Next
715
716         AAdd(aRet , nRet )
717         lRetorno := .T.
718
719     Else
720
721         If !lTemTEF
722             lRetorno := STWReceiveNotFiscal( cTotalizador, Self:GetTotal(), cForma ) == 0
723         EndIf
724
725         If lRetorno
726             For nX := 1 To nRecVias
727                 aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
728                                     "STManagReportPrint" ,;

```

```

729                                     {cTextoComprovante, 1})
730             Next nX
731         EndIf
732     EndIf
733
734     If Len(aRet) == 0 .OR. aRet[1] <> 0
735         STFMessage( ProcName(0) , "STOP", STR0001) //"Falha na impressão do comprovante de recebimento."
736         STFShowMessage(ProcName(0))
737         lRetorno := .F.
738     Else
739         lRetorno := .T.
740     EndIf
741
742     ElseIf Self:cOperation == OP_ESTOR
743
744         //deve-se enviar o Índice do totalizador pois as impressoras da SIGALOJA esperam um valor numérico
745         cTotalizador := STWReceTotalizer("TOTALIZADORES","ESTORNO")
746
747         //Valor total a ser impresso como sangria
748         nVlTotal := Self:GetTotal()
749
750         If nVlTotal > 0
751             // Independente se foi colocada chave no SIGALOJA
752             // ([Estorno Venda])
753             // totalizadores= XX
754             // Será emitido um cupom de Sangria
755             If !Empty(cForma)
756                 //Emite o documento na impressora
757                 If lEmitNfce
758                     lRetorno := .T.
759                 Else
760                     lRetorno := STWReceiveNotFiscal( cTotalizador, nVlTotal, cForma ) == 0
761                 EndIf
762             Else
763                 aValues[VALUE_MONEY] := nVlTotal
764                 lRetorno := STWSupplyBleeding( 1 , aValues , STBGetCurrency() , .T. )
765             EndIf
766         Else
767             lRetorno := .T.
768         EndIf
769
770         If lRetorno
771
772             If lEmitNfce
773                 nRet := STWPrintTextNotFiscal(cTextoComprovante)
774                 AAdd(aRet , nRet )
775                 lRetorno := .T.
776
777             If lGuil
778                 STWPrintTextNotFiscal(TAG_GUIL_INI+TAG_GUIL_FIM)
779             EndIf
780

```

```

781      // Quando estorno, nao exhibe relatorio gerencial
782      Else
783          aRet := STFFireEvent( ProcName(0)      ,; // Nome do processo
784                              "STManagReportPrint" ,;
785                              {cTextoComprovante, 1} )
786      EndIf
787
788      If Len(aRet) == 0 .OR. aRet[1] <> 0
789          STFMessage( ProcName(0) , "STOP", STR0001) //"Falha na impressão do comprovante de recebimento."
790          STFShowMessage(ProcName(0))
791          lRetorno := .F.
792      Else
793          lRetorno := .T.
794      EndIf
795  EndIf
796
797  If !lRetorno
798      STFMessage(ProcName(0), "STOP", STR0002) //"Falha na impressao do Documento Nao Fiscal."
799      STFShowMessage( ProcName(0))
800  EndIf
801 EndIf
802
803 CursorArrow()
804
805 Else
806     STFMessage(ProcName(0), "STOP", STR0003) //"Nenhum valor a receber foi informado."
807     STFShowMessage(ProcName(0))
808 EndIf
809
810 Return lRetorno
811
812
813 //-----
814 /*{Protheus.doc} LoadTitles
815 Conecta com a retaguarda e obtem a lista de títulos em aberto, preenchendo o array aListTitles
816 @return lRetorno - se obteve a lista de titulos na retaguarda
817 @obs
818 @sample
819 */
820 //-----
821 Method LoadTitles() Class STWReceiptTitle
822
823 Local aParametros := {} // Array com os parametros da funcao
824 Local nI           := 0 // Contador
825 Local lRetorno     := .F. // Retorno da funcao
826 Local lRet         := .F. // Retorno da funcao STRecLoad
827 Local aTitles      := {} // Lista de titulos a receber
828 Local aParam       := {} //Conteúdo que será enviado para o ponto de entrada
829 Local lConnect     := .F. //Conexão com a retaguarda
830
831 Aadd(aParametros, SuperGetMV("MV_LJRECEB"))
832

```

```

833 Aadd(aParametros, Self:cCode)      // cCodCli
834 Aadd(aParametros, Self:cBranch)    // cLojCli
835 Aadd(aParametros, Self:cCnpjCpf)   // cCPF
836
837 Aadd(aParametros, "")              // cCartao
838 Aadd(aParametros, "")              // cNumRA
839 Aadd(aParametros, Self:cPrefix)     // cPrefixo
840 Aadd(aParametros, Self:cNumber)     // cNum
841 Aadd(aParametros, Self:cParcel)     // cParcela
842 Aadd(aParametros, Nil)              // nRecno
843 Aadd(aParametros, Nil)              // dDataRef
844
845 If Type("cEstacao") <> "U"
846     Aadd(aParametros, cEstacao)      // Estação (PDV)
847     Aadd(aParametros, xNumCaixa())    // Caixa
848 Else
849     Aadd(aParametros, "")            // Estação (PDV)
850     Aadd(aParametros, "")            // Caixa
851 EndIf
852
853 CursorWait()
854
855 LjMsgRun( "Aguarde...Verificando conexão com o Host",, {|| lConnect := STWReCoHost() } ) //"Aguarde...Verificando conexão com o Host"
856
857 If lConnect
858     Conout("Recebimento - Buscando titulo na retaguarda !! ")
859     lRet := STBRemoteExecute( "STRecLoad",{Self:cOperation, aParametros},Nil,.F.,@aTitles )
860     Conout("Recebimento - Retorno " + cValToChar(lRet))
861     conout("-----" )
862     conout("-----" )
863 EndIf
864
865 CursorArrow()
866
867 lRet := lConnect
868
869 // Ponto de entrada para organizar os titulos que serao recebidos
870 If ExistBlock("STRECV")
871     //Parametros para o ponto de entrada STRECV
872     Aadd(aParam, Self:cOperation)      // Tipo de operação
873     Aadd(aParam, Self:cPrefix)         // Prefixo
874     Aadd(aParam, Self:cNumber)         // Numero do titulo
875     Aadd(aParam, Self:cCnpjCpf)       // Cpf ou Cnpj
876
877     LjGrvLog("Recebimento","Antes da Chamada do Ponto de Entrada:STRECV - {aTitles,aParam}")
878     aTitles := ExecBlock( "STRECV" ,.F.,.F.,{aTitles,aParam} )
879     LjGrvLog("Recebimento","Apos a Chamada do Ponto de Self:cOperationEntrada:STRECV - {aTitles,aParam}")
880     //Caso tenha algum problema com a comunicação com a Retaguarda ou não tenha nenhum titulo no Protheus
881     If Len(aTitles) > 3 .And. Len(aTitles[4]) > 0
882         lRet := .T.
883     EndIf
884 EndIf

```

```

885
886 If !lRet
887     STFMessage(ProcName(0), "STOP", STR0008 )//Erro ao carregar títulos -> //"Não foi possível carregar os títulos no momento - Verificar conexão"
888     STFShowMessage(ProcName(0))
889 Else
890     If ValType(aTitles) == "A" .AND. Len(aTitles) > 0
891         Self:aListTitles := aTitles
892         If ValType(Self:aListTitles) == 'A' .AND. Len(Self:aListTitles) > 0
893             If Self:aListTitles[1] == 0
894                 lRetorno := .T.
895                 If Len(Self:aListTitles[4]) = 0
896                     STFMessage(ProcName(0), "STOP", STR0020) //"Nenhum título encontrado!"
897                     STFShowMessage( ProcName(0))
898                     lRetorno := .F.
899                 Else
900                     // Cria primeira posicao logica de selecao do titulo no array e move as demais para baixo (medida de compatibilidade do LOJXREC)
901                     For nI := 1 To Len(Self:aListTitles[4])
902                         ASize(Self:aListTitles[4][nI], Len(Self:aListTitles[4][nI]) + 1)
903                         AIns(Self:aListTitles[4][nI], 1)
904                         Self:aListTitles[4][nI][TIT_SELE] := .F.
905                         If Self:cOperation == "1"
906                             Self:aListTitles[4][nI][TIT_RECE] += Self:aListTitles[4][nI][TIT_ACRS]
907                         EndIf
908                     Next nI
909                 EndIf
910             ElseIf Self:aListTitles[1] == 1
911                 // Problema no processamento
912                 STFMessage(ProcName(0), "STOP", Self:aListTitles[2] + " - " + Self:aListTitles[3]) // Cliente nao encontrado - 0 codigo do cliente informado nao foi
913                 encontrado.
914                 STFShowMessage(ProcName(0))
915             Else
916                 // Problema nao identificado
917                 STFMessage(ProcName(0), "STOP", Self:aListTitles[2] + " - " + Self:aListTitles[3]) // Outra causa
918                 STFShowMessage(ProcName(0))
919             EndIf
920         EndIf
921     EndIf
922 EndIf
923 If !lRetorno
924     // Limpa array de titulos em caso de falha
925     Self:aListTitles := {}
926 EndIf
927
928 Return lRetorno
929
930
931 //-----
932 /*{Protheus.doc} DropTitles
933 Chama função de baixa de títulos na retaguarda
934 @return .T. quando todos os títulos forem baixados com sucesso / .F. quando houver falha em algum processo
935 @obs

```

```

936 @sample
937 /*/
938 //-----
939 Method DropTitles(oMdlGrd, lContingency) Class STWReceiptTitle
940
941 Local aParametros      := {} // Array com os parametros da funcao
942 Local aListDropTitles := {} // Array com a lista de titulos (aParametros) a baixar na retaguarda
943 Local nI               := 0 // Contador
944 Local lRetorno         := .T. // Retorno da funcao - .T. quando baixados com sucesso / .F. quando houver falha em alguma baixa (aborta as proximas)
945 Local lRet             := .F. // Retorno da funcao STRecLoad
946 Local lRetTitles       := .F. // Retorno da funcao STDropTitles
947 Local aFormas          := {} // Array auxiliar com as formas de pgto utiliziadas na venda
948 Local aPgto            := {} // Array com as informações de pgto utilizado na baixa do titulo no LOJXREC
949 Local aPgtoSint        := {} // Array com os pgto sintetizados utilizado na baixa do titulo no LOJXREC
950 Local aTefpgto         := {} // Array com as informações do TEF utilizado para baixa dos titulos no LOJXREC
951 Local aTitSel          := {} // Array com as formas a serem gerados titulos no SE1 apos a baixa do titulos (para serem compensados posteriormente - CH/CC/CD)
952 Local aParamSE1        := {} // Array com os parametros para a execução da função STGeraTitRec na retaguarda.
953 Local aTitulos         := {} // Array com os titulos a serem baixados
954 Local cCliente         := "" // Cliente da baixa
955 Local cLojaCli         := "" // Loja do cliente
956 Local cInfoTit         := "" //
957 Local nContCheque      := 0 // Contador que controla informação do cheque
958 Local aPgtDet          := {} // Informações de detalhes das formas de pgto CH/CC/CD
959 Local aSE5Baixas       := {} // Array com as baixas realizadas na retaguarda.
960 Local aSE5Relacao      := {} // Array com a relação forma de pagamento x baixa de titulos x valores
961 Local aCompNcc         := {} // Nccs utilizadas para compensação
962 Local aCompTit         := {} // Titulos utilizadas na compensação
963 Local lTemFormaSel     := .F. // Verifica se é necessário criar os registro no SE1
964 Local aCheques         := STWGetCkRet() // Retorna as informações dos cheques
965 Local aNCCs            := STDGetNCCs("1") // NCCs do cliente
966 Local aSLV             := {} // Array com as informações a serem incluídas na tabela SLV.
967 Local aMovTitulos      := {} // Array para geração de SE5 local (para ser exibido no fechamento de caixa)
968 Local aMovForma        := {} // Array para geração de SE5 local (para ser exibido no fechamento de caixa por forma)
969 Local lSTDRecSE5       := ExistFunc("STDRecSE5")
970 Local nX               := 0 // Contador 2
971 Local nIntervalo       := SuperGetMV("MV_LJINTER") //Intervalo das parcelas
972 Local nValCC           := 0 // Valor Cartão
973 Local nDataCC          := // Data Cartão
974 Local nDiferenca       := 0 // Diferenca da divisão em parcelas no cartao
975 Local aMdlGrd         := {}
976 Local lRetComuBx       := .F. // Retorno da comunicacao com retaguarda
977 Local xRetTitles       := Nil // Retorno da funcao STDropTitles
978 Local lExecBxRet       := .T. // Controle para permitir tentar novamente caso ocorra erro
979 Local aNumCheq         := {} // Array com numeros do cheque
980 Local nCH              := 0 // Contador
981 Local cCPFCLi         := "" //CPF do cliente em recebimento offline
982 Local nC               := 0 //Contador de Recebimento Offline
983 Local lCmpsIdTRN       := SL4->(ColumnPos("L4_TRNID")) > 0 .And. SL4->(ColumnPos("L4_TRNPCID")) > 0 .And. SL4->(ColumnPos("L4_TRNEXID")) > 0 //Verifica se existem os
campos que guardam os IDs das transações TEF
984 Local cMvTpRet         := SuperGetMV("MV_LJCTRET",,"RI|RG|RB|RS") //Formas de pagamento para certificado de retenção, que não devem gerar registros na tabela de
recebimentos (SE1).
985

```

```

986 Default oMdlGrd := Nil
987 Default lContingency := .F. //Recebimento em contingência
988
989
990 LjGrvLog( "Recebimento_Titulo", "Inicio da baixa")
991
992 If ValType(oMdlGrd) == "0"
993
994     CursorWait()
995
996     //Para efetuar a compensação das NCCs com os titulos
997     For nI := 1 To Len(aNCCs)
998         If aNCCs[nI][1]
999             aAdd(aCompNcc, aNCCs[nI][5]) //RECNO
1000         EndIf
1001     Next
1002
1003     //Este loop é feito para que as informações necessários referente a forma de pagamento
1004     //estejam em variaveis auxiliares, para que seja possivel efetuar o tratamento a seguir da baixa de titulo.
1005     //Onde será feita a organização de qual forma baixa qual titulo.
1006     For nI := 1 To oMdlGrd:Length()
1007
1008         oMdlGrd:GoLine(nI)
1009         cFormaBx := AllTrim(oMdlGrd:GetValue("L4_FORMA"))
1010
1011         aAdd(aMdlGrd, {nI,;
1012             oMdlGrd:GetValue("L4_DATATEF"),; //1-Posicao forma
1013             oMdlGrd:GetValue("L4_HORATEF"),; //2-Data
1014             oMdlGrd:GetValue("L4_DOCTEF"),; //3-Hora
1015             oMdlGrd:GetValue("L4_AUTORIZ"),; //4-DOC
1016             oMdlGrd:GetValue("L4_INSTITU"),; //5-Autorizacao
1017             oMdlGrd:GetValue("L4_NSUTEF"),; //6-Instituicao
1018             oMdlGrd:GetValue("L4_TIPCART"),; //7-NSU
1019             cFormaBx,; //8-Tipo do cartao
1020             oMdlGrd:GetValue("L4_VALOR"),; // 9 -forma de pagamento
1021             oMdlGrd:GetValue("L4_BANDEIR"),; // 10 - valor do pagamento
1022             oMdlGrd:GetValue("L4_REDEAUT"),; // 11 - bandeira do cartão
1023             Iif(lCmpsIdTRN, oMdlGrd:GetValue("L4_TRNID"), ""),; // 12 - rede autorizada (adquirência)
1024             Iif(lCmpsIdTRN, oMdlGrd:GetValue("L4_TRNPCID"), ""),; // 13 - TRNID da Transação de Pagamento Digital
1025             Iif(lCmpsIdTRN, oMdlGrd:GetValue("L4_TRNEXID"), "")} // 14 - TRNPCID da Transação de Pagamento Digital
1026             // 15 - TRNEXID da Transação de Pagamento Digital
1027
1028         /**
1029         /****
1030         /*****
1031         //**** Tratamento para Inclusao de titulo SE1 (Formas de pagamentos diferente de R$) ****
1032         //Caso a forma seja NCC não deve entrar no tratamento de baixa, pois a mesma será compensada a parte na baixa
1033         If !(cFormaBx == "CR" .And. Len(aCompNcc) > 0)
1034             If cFormaBx == "CH"
1035                 nContCheque++
1036
1037                 For nX := 1 To Len(aCheques[nContCheque])
1038                     If Len(aCheques) >= nContCheque

```

```

1038     lTemFormaSE1 := .T.
1039
1040     If ValType(aCheques[1][1][1]) == 'A' .AND. Len(aCheques[nContCheque][1][1]) >= 11
1041         aAdd(aPgtDet, {aCheques[nContCheque][nX][1,11]      ;; //01- Valor
1042                     aCheques[nContCheque][nX][1,10]        ;; //02- Data
1043                     ""                                     ;; //03-
1044                     aCheques[nContCheque][nX][1,1]         ;; //04- Banco
1045                     aCheques[nContCheque][nX][1,3]         ;; //05- Agencia
1046                     aCheques[nContCheque][nX][1,4]         ;; //06- Conta
1047                     aCheques[nContCheque][nX][1,2]         ;; //07- Numero
1048                     aCheques[nContCheque][nX][1,5]         ;; //08- Compensacao
1049                     aCheques[nContCheque][nX][1,8]         ;; //09- RG
1050                     aCheques[nContCheque][nX][1,7]         ;; //10- Telefone
1051                     ""                                     ;; //11-
1052                     aCheques[nContCheque][nX][1,9]         ;; //12- Chq Terceiro
1053                     ""                                     ;; //13-
1054                     aCheques[nContCheque][nX][1,6]         }) //14-Nome do terceiro
1055     Else
1056         aPgtDet := {oMdlGrd:GetValue("L4_VALOR")           ;; //01- Valor
1057                 oMdlGrd:GetValue("L4_DATA")               ;; //02- Data
1058                 ""                                         ;; //03-
1059                 aCheques[nContCheque][1,1]               ;; //04- Banco
1060                 aCheques[nContCheque][1,3]               ;; //05- Agencia
1061                 aCheques[nContCheque][1,4]               ;; //06- Conta
1062                 aCheques[nContCheque][1,2]               ;; //07- Numero
1063                 aCheques[nContCheque][1,5]               ;; //08- Compensacao
1064                 aCheques[nContCheque][1,8]               ;; //09- RG
1065                 aCheques[nContCheque][1,7]               ;; //10- Telefone
1066                 ""                                         ;; //11-
1067                 aCheques[nContCheque][1,9]               ;; //12- Chq Terceiro
1068                 ""                                         ;; //13-
1069                 aCheques[nContCheque][1,6]               } //14-Nome do terceiro
1070     EndIf
1071     Aadd(aNumCheq, aCheques[nContCheque][nX][1,2])
1072 Else
1073     aPgtDet := {}
1074 EndIf
1075
1076 If ValType(aCheques[1][1][1]) == 'A' .AND. Len(aCheques[nContCheque][1][1]) >= 11
1077     aAdd(aPgto,{aCheques[nContCheque][nX][1,10]          ;; //1 - Data
1078             aCheques[nContCheque][nX][1,11]              ;; //2 - Valor
1079             cFormaBx                                     ;; //3 - Forma
1080             aPgtDet                                       ;; //4 - Info
1081             Nil                                           ;; //5 - Nil
1082             1                                              ;; //6 - Moeda
1083             Nil                                           ;; //7 - Nil
1084             iIf(!Empty(oMdlGrd:GetValue("L4_FORMAID")),oMdlGrd:GetValue("L4_FORMAID"),AllTrim(Str(nI))),; //8 - FormID
1085             AllTrim(Str(nContCheque))                     ;; //9 - ChequeId
1086             oMdlGrd:GetValue("L4_VALOR")                 }) //10 - Valor da moeda
1087
1088     aPgtDet := {}
1089 EndIf

```

```

1090         Next
1091         ElseIf !(IsMoney(cFormaBx) .OR. (cFormaBx) $ cMvTpRet)
1092             lTemFormaSE1 := .T.
1093             nValCC      := STBRound(oMdlGrd:GetValue("L4_VALOR")/oMdlGrd:GetValue("L4_PARC"), 2)
1094             nDiferenca  := STBRound(oMdlGrd:GetValue("L4_VALOR") - ((STBRound(oMdlGrd:GetValue("L4_VALOR") / oMdlGrd:GetValue("L4_PARC"),2)) *
oMdlGrd:GetValue("L4_PARC")), 2)
1095
1096         For nX := 1 To oMdlGrd:GetValue("L4_PARC")
1097             If nX == oMdlGrd:GetValue("L4_PARC")
1098                 nValCC := nValCC + nDiferenca
1099             EndIf
1100
1101             nDataCC := oMdlGrd:GetValue("L4_DATA") + If(nX = 1, 0, nIntervalo * (nX - 1))
1102
1103             aPgtDet := {      nValCC                                ,; //01- Valor
1104                             nDataCC                                ,; //02- Data
1105                             ""                                     ,; //03-
1106                             oMdlGrd:GetValue("L4_NUMCART")         ,; //04- Numero do cartao
1107                             SubStr(oMdlGrd:GetValue("L4_ADMINIS"),1,TamSx3("AE_COD")[1]),; //05- Cod.Adm Financeira
1108                             ""                                     ,; //06-
1109                             ""                                     ,; //07-
1110                             iIf(!Empty(oMdlGrd:GetValue("L4_FORMAID")),oMdlGrd:GetValue("L4_FORMAID"),AllTrim(Str(nI))) ,; //08- Id do cartao
1111                             ""                                     ,; //09-
1112                             ""                                     ,; //10-
1113                             ""                                     ,; //11-
1114                             .F.                                   ,; //12-
1115                             ""                                     ,; //13-
1116                             ""                                     } //14-
1117
1118             aAdd(aPgto,{      nDataCC                                ,; //1 - Data
1119                             nValCC                                ,; //2 - Valor
1120                             cFormaBx                             ,; //3 - Forma
1121                             aPgtDet                              ,; //4 - Info
1122                             Nil                                   ,; //5 - Nil
1123                             1                                    ,; //6 - Moeda
1124                             Nil                                   ,; //7 - Nil
1125                             iIf(!Empty(oMdlGrd:GetValue("L4_FORMAID")),oMdlGrd:GetValue("L4_FORMAID"),AllTrim(Str(nI))),; //8 - FormID
1126                             ""                                   ,; //9 - ChequeId
1127                             oMdlGrd:GetValue("L4_VALOR")         }) //10 - Valor da moeda
1128         Next
1129
1130         aPgtDet := {}
1131
1132         aAdd(aTefpgto, {iIf(!Empty(oMdlGrd:GetValue("L4_FORMAID")),oMdlGrd:GetValue("L4_FORMAID"),AllTrim(Str(nI))),; //1-Id da forma
1133                     oMdlGrd:GetValue("L4_DOCTEF")                ,; //2-Numeração do Documento TEF
1134                     oMdlGrd:GetValue("L4_NSUTEF")                ,; //3-Numeração do NSU TEF
1135                     oMdlGrd:GetValue("L4_AUTORIZ")              }) //4-Numeração da Autorização
1136     Else
1137         aPgtDet := {}
1138     EndIf
1139
1140     If cFormaBx $ "R$" .OR. (Len(aCheques) > 0 .AND. ValType(aCheques[1][1][1]) <> 'A' .AND. Len(aCheques[nContCheque][1][1]) < 11)

```

```

1141         aAdd(aPgto,{oMdlGrd:GetValue("L4_DATA")           ,; //1 - Data
1142                   oMdlGrd:GetValue("L4_VALOR")           ,; //2 - Valor
1143                   cFormaBx                               ,; //3 - Forma
1144                   IIF(cFormaBx == "R$", {}, aPgtDet)       ,; //4 - Info
1145                   Nil                                     ,; //5 - Nil
1146                   1                                       ,; //6 - Moeda
1147                   Nil                                     ,; //7 - Nil
1148                   iIf(!Empty(oMdlGrd:GetValue("L4_FORMAID")),oMdlGrd:GetValue("L4_FORMAID"),AllTrim(Str(nI))),; //8 - FormID
1149                   Iif(cFormaBx == "CH",AllTrim(Str(nContCheque)), "") ,; //9 - ChequeId
1150                   oMdlGrd:GetValue("L4_VALOR")   }) //10 - Valor da moeda
1151     EndIf
1152
1153     aAdd(aPgtoSint,{ cFormaBx           ,; //1-Forma
1154                   oMdlGrd:GetValue("L4_PARC") ,; //2-Quantidade de parcelas
1155                   oMdlGrd:GetValue("L4_VALOR") ,; //3-Valor da forma
1156                   iIf(!Empty(oMdlGrd:GetValue("L4_FORMAID")),oMdlGrd:GetValue("L4_FORMAID"),AllTrim(Str(nI))),; //4-Id da forma
1157                   oMdlGrd:GetValue("L4_DATA") ,; //5-Data
1158                   {} ,; //6-Array (não utilizado na baixa)
1159                   "" ,; //7-String (não utilizado na baixa)
1160
1161     Else
1162       aAdd(aMovForma,{ cFormaBx           ,; //1-forma de pagamento utilizadas
1163                     oMdlGrd:GetValue("L4_VALOR") ,; //2-Valor de abate do título para a forma
1164                     "" ,; //3-Prefixo do título
1165                     "" ,; //4-Numero do título
1166                     "" ,; //5-Parcela do título
1167                     "" ,; //6-Filial de origem
1168                     "" ,; //7-Ordem da forma de pagamento
1169
1170     EndIf
1171
1172     Next
1173
1174     //Guardando os titulos no array 'aTitulos' para pode efetuar o tratamento da baixa dos mesmo.
1175     cCPFCLI := Self:cCnpjCpf
1176     StwRecArr(IIF(!lContingency, Self:aListTitles[4], {}), @aCompTit, @aMovTitulos, aCompNCC, ;
1177               @aListDropTitles, @aSE5Relacao, aMdlGrd, @aMovForma, ;
1178               cUserName, @aNumCheq, SuperGetMV("MV_LJRECEB"), @aTitSel, ;
1179               lContingency, @cCliente, @cLojaCli, Self)
1180
1181     Self:dDtRec := Date()
1182     Self:cHrRec := Time()
1183
1184     //***** EXECUTANDO OS COMANDOS DE BAIXA NA RETAGUARDA *****
1185     While lExecBxRet .AND. (Len(aListDropTitles) > 0 .OR. Len(aCompNcc) > 0)
1186       LjGrvLog( "Recebimento_Titulo", "Titulos a baixar(aListDropTitles): ", aListDropTitles )
1187       LjGrvLog( "Recebimento_Titulo", "Ncc a compensar(aCompNcc): ", aCompNcc )
1188
1189       If lTemFormaSE1 .Or. lContingency
1190         //Verifica necessidade da chamada de geração de titulos no SE1/MDM/MDN/SLV
1191         aParamSE1 := { "1", aPgto      , aPgtoSint , aTitSel      ,;
1192                       Nil, Nil        , Nil        , Nil        ,;
1193                       Nil, cCliente   , cLojaCli    , Nil        ,;

```

```

1193         Nil, Nil          , .F.          , xNumCaixa()    ,;
1194         Nil, Nil          , aTefpgto     , cCPFcli ,;
1195         cEstacao}
1196     LjGrvLog( "Recebimento_Titulo", "Dados para geracao SE1(aParamSE1): ", aParamSE1 )
1197 EndIf
1198
1199 //ID da requisição de Baixa (comum entre PDV e retaguarda) utilizada no controle da baixa assincrona
1200 cRequisicao := "REC" + AllTrim(FWCodFil()) + UPPER(AllTrim(cEstacao)) + UPPER(AllTrim(cUserName)) + FWTimeStamp(1)
1201 LjGrvLog( "Recebimento_Titulo", "Antes executar STDropTitl na Retaguarda."+;
1202         IIF(lContingency, "Recebimento em contingência. CPF, Valor, Título:" + Self:cCnpjCpf + "," + cValToChar(Self:nValueContingency) + ","
+Self:cPrefix+"/"+Self:cNumber+"/"+Self:cParcel , "") + "Requisição: "+ cRequisicao)
1203
1204 If ExistBlock("STRECALT")
1205     LjGrvLog("Recebimento","Antes da Chamada do Ponto de Entrada:STRECALT - aListDropTitles",aListDropTitles)
1206     aListDropTitles := ExecBlock( "STRECALT" ,.F.,.F.,{aListDropTitles} )
1207     LjGrvLog("Recebimento","Apos a Chamada do Ponto de Entrada:STRECALT - aListDropTitles", aListDropTitles)
1208 EndIf
1209
1210 If Len(aListDropTitles) > 0 .Or. Len(aCompNcc) > 0
1211     lRetComuBx := STBRemoteExecute( "STDropTitl",{ aListDropTitles , {aCompTit,aCompNcc} , cUserName , lTemFormaSE1 ,;
1212         aParamSE1 , aTitSel , aSE5Relacao , aMdlGrd ,;
1213         cRequisicao , .T. , lContingency , Dtos(Self:dDtRec)+StrTran(Self:cHrRec, ":")} ,{"OK",
.T.} ,lContingency , @xRetTitles )
1214 EndIf
1215
1216 LjGrvLog( "Recebimento_Titulo", "Apos executar STDropTitl na Retaguarda. Retorno comunicacao(lRetComuBx):",lRetComuBx)
1217 LjGrvLog( "Recebimento_Titulo", "Apos executar STDropTitl na Retaguarda. Retorno comando(xRetTitles):",xRetTitles)
1218
1219 If lRetComuBx
1220     If ( ( ValType(xRetTitles) == "C" .AND. xRetTitles == 'OK' ) .OR.; //Se realizou a baixa no modo atualizado, com controle de transacao
1221         ValType(xRetTitles) == "L") .OR. lContingency //Se realizou a baixa com LOJXREC Antigo
1222         lRetTitles := .T.
1223     Else
1224         lRetTitles := .F.
1225     EndIf
1226 EndIf
1227
1228 CursorArrow()
1229
1230 lExecBxRet := .F.
1231
1232 If (!lRetComuBx .OR. !lRetTitles) .AND. ValType(xRetTitles) == "C"
1233
1234     LjGrvLog( "Recebimento_Titulo", "Não foi possível concluir a baixa do título na Retaguarda. Tentar novamente?",xRetTitles)
1235
1236     xRetTitles := STWRetMsg(xRetTitles) //Converte mensagem para formato amigavel para usuario nao tecnico
1237
1238     If MsgYesNo( xRetTitles + Chr(13)+ Chr(13)+;
1239         STR0029+Chr(13)+; // "Por esse motivo, não foi possível concluir a baixa na Retaguarda"
1240         STR0031 ) // "Deseja tentar novamente?"
1241         lExecBxRet := .T.
1242     CursorWait()

```

```

1243         EndIf
1244
1245         LjGrvLog( "Recebimento_Titulo", "Não foi possível concluir a baixa do título na Retaguarda. Tentar novamente?. Retorno continua:",lExecBxRet)
1246     EndIf
1247
1248     EndDo
1249 EndIf
1250
1251 If Len(aListDropTitles) > 0
1252     If !lRetComuBx
1253         STFMessage(ProcName(0), "STOP", STR0009)    //"Erro ao executar a baixa de Títulos no servidor"
1254         STFShowMessage(ProcName(0))
1255         lRetorno := .F.
1256     ElseIf !lRetTitles
1257         STFMessage(ProcName(0), "STOP", STR0025)    //"Não foi possível realizar a baixa na Retaguarda"
1258         STFShowMessage(ProcName(0))
1259     EndIf
1260     lRetorno := lRetTitles
1261 EndIf
1262
1263 //Popula tabela SE5 para conferencia de caixa
1264 If lRetorno .AND. lSTDRecSE5
1265     LjGrvLog( "Recebimento_Titulo", "Antes de executar STDRecSE5 (SE5 Local para conferencia de Caixa)",lRetorno)
1266     STDRecSE5(aMovTitulos,aMovForma, , lContingency)
1267     LjGrvLog( "Recebimento_Titulo", "Apos executar STDRecSE5 (SE5 Local para conferencia de Caixa)",lRetorno)
1268 EndIf
1269
1270 LjGrvLog( "Recebimento_Titulo", "Fim da baixa. Retorno continua:",lRetorno)
1271
1272 Return lRetorno
1273
1274
1275 //-----
1276 /*{Protheus.doc} ReverseDropTitles
1277 Chama função de estorno de baixa de títulos na retaguarda
1278 @return .T. quando todos os títulos forem estornados com sucesso / .F. quando houver falha em algum processo
1279 @obs
1280 @sample
1281 */
1282 //-----
1283 Method ReverseDropTitles() Class STWReceiptTitle
1284
1285 Local aListRever    := {}        // Array com a lista de titulos (controle loop de estornos)
1286 Local nI            := 0        // Contador
1287 Local lRetorno      := .F.      // Retorno da funcao - .T. quando baixados com sucesso / .F. quando houver falha em alguma baixa (aborta as proximas)
1288 Local lRet          := .F.      // Retorno da funcao que e executada na retaguarda
1289 Local aEstorno      := {}        // Retorno da
1290 Local nY            := 0        // Contador
1291 Local nEstTotal      := 0        // Valor total dos estornos
1292 Local aVlrCompNCC    := {}      // Valor de estorno de compensação (NCC)
1293 Local nTotNcc        := 0        // Valor de NCC utilizado na baixa
1294 Local cNumCaixa      := xNumCaixa() //Caixa que efetuou o estorno.

```

```

1295 Local aMovForma      := {}
1296 Local lFuncGrv       := ExistFunc("STDRecSE5")
1297 Local nZ              := 0
1298 Local aMDMLote        := {}           //Array de Lotes
1299 Local cMsgErro        := ""
1300 Local aListTitles      := {}           //Lista de títulos
1301 Local aMovFormas       := {}           //Array com formas de pagamentos para estornar TEF/PD
1302
1303
1304 CursorWait()
1305 LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Inicio - Self:aListTitles:",Self:aListTitles)
1306 If (STFProFile(29)[1]) // Verifica se tem permissao para estornar o caixa
1307   aEval(Self:aListTitles[4], { |t| IIF(t[TIT_SELE], aAdd( aListTitles, aClone(t) ) , )})
1308   For nI := 1 To Len(Self:aListTitles[4])
1309     If Self:aListTitles[4][nI][TIT_SELE]
1310
1311       aListRever := {Self:aListTitles[4][nI]}
1312       LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Titulos Seleccionados(aListRever):",aListRever)
1313
1314       lRet       := .T.
1315       nEstTotal  := 0
1316       aEstorno   := {}
1317       aMovForma  := {}
1318
1319       If ExistBlock("STRECEST")
1320         LjGrvLog("Recebimento","Antes da Chamada do Ponto de Entrada:STRECEST - aListRever",aListRever)
1321         aListRever := ExecBlock( "STRECEST" ,.F.,.F.,{aListRever} )
1322         LjGrvLog("Recebimento","Apos a Chamada do Ponto de Entrada:STRECEST - aListRever", aListRever)
1323       EndIf
1324
1325       If Len(aListRever) > 0
1326
1327         LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Antes de executar STReverseD")
1328         lRet := STBRemoteExecute( "STReverseD",{aListRever,cNumCaixa, aMDMLote,aListTitles },Nil,.F., @aEstorno )
1329         LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Apos executar STReverseD. Retorno aEstorno:",aEstorno)
1330
1331         //Tratamento por causa de possivel fonte desatualizado
1332         If ValType(aEstorno) == "L" .Or. !lRet .Or. Len(aEstorno) == 0
1333           LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Saiu do processo, retorno invalido da retaguarda (ValType(aEstorno) == 'L' .Or. !lRet .Or.
1334           Len(aEstorno) == 0). Retorno lRet:",lRet)
1335           Exit
1336         Endif
1337
1338         If ( !(Len(aEstorno) >= 4) .Or. Empty(aEstorno[3]) ) .AND. (Len(aEstorno[1]) > 0 .Or. Len(aEstorno[2]) > 0)
1339           aVlrCompNCC := aEstorno[1] //Estorno referente as NCCs
1340           aEstorno     := aEstorno[2] //Estorno referente as formas de pagamento (diferente de NCC)
1341           nCont := 0
1342           If Len(aEstorno) >= 4
1343             aMDMLote := aClone(aEstorno[4])
1344           EndIf
1345
1346           //Array para gerar movimentação no SE5 local

```

```

1346         If Len(aMovForma) == 0
1347
1348             //Tratamento para NCC
1349             If Len(aVlrCompNCC) == 1
1350                 For nZ := 1 To Len(aVlrCompNCC[1])
1351
1352                     nTotNcc += aVlrCompNCC[1][nZ][8]
1353                     nEstTotal += aVlrCompNCC[1][nZ][8]
1354
1355                     aAdd(aMovForma,{"CR"                ,; //Forma
1356                               aVlrCompNCC[1][nZ][8]    ,; //Valor
1357                               aVlrCompNCC[1][nZ][1]    ,; //Prefixo
1358                               aVlrCompNCC[1][nZ][2]    ,; //Numero
1359                               aVlrCompNCC[1][nZ][3]    ,; //Parcela
1360                               Nil                      ,; //Filial origem
1361                               Nil})
1362                 Next
1363             EndIf
1364
1365             //Demais formas de pagamentos
1366             For nZ := Len(aEstorno) To 1 Step -1
1367
1368                 If AllTrim(aEstorno[nZ][25]) == "BA"
1369                     nEstTotal += aEstorno[nZ][8]
1370                     aAdd(aMovForma,{aEstorno[nZ][24],; //Forma
1371                                   aEstorno[nZ][8] ,; //Valor
1372                                   aEstorno[nZ][1] ,; //Prefixo
1373                                   aEstorno[nZ][2] ,; //Numero
1374                                   aEstorno[nZ][3] ,; //Parcela
1375                                   Nil              ,; //Filial origem
1376                                   Nil})
1377                 //Exit
1378             EndIf
1379
1380             Next
1381
1382             Else
1383                 aEval(aEstorno,{|e| nCont += iif(AllTrim(e[25])=="BA",1,0)})
1384             EndIf
1385
1386         Else
1387             lRet := .F.
1388             If Len(aEstorno) >= 4
1389                 cMsgErro := aEstorno[3]
1390             EndIf
1391         EndIf
1392
1393         //Altera o valor do titulo com o valor de estorno para que este valor de estorn seja impresso
1394         If nEstTotal > 0
1395             Self:SetParcTotal(nI, nEstTotal)
1396             Self:SetTotNCC(nTotNcc)
1397         EndIf

```

```

1398         EndIf
1399
1400         If lRet
1401             Aadd(aMovFormas, aMovForma)
1402         Endif
1403
1404         If lRet .And. lFuncGrv
1405             LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Antes de executar STDRecSE5(aListRever,aMovForma)",{aListRever,aMovForma})
1406             STDRecSE5(aListRever,aMovForma,.T.)
1407             LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Apos executar STDRecSE5")
1408         EndIf
1409
1410
1411     EndIf
1412 Next nI
1413
1414
1415 // Busca na SLV se o titulo foi para em TEF ou Pagamento Digital para posteriormente estornar também
1416 IF lRet .AND. Len(aMovFormas) > 0
1417     STWESTTEFT(aMovFormas)
1418 Endif
1419
1420 If !lRet
1421     LjGrvLog( "Recebimento_Titulo", ">>>NC<<< - Processo de ESTORNO - Erro ao executar o estorno de Títulos no servidor: Erro retornado: "+ cMsgErro)
1422     STFMessage(ProcName(0), "POPUP", STR0010 + cMsgErro)//"Não foi possível realizar o estorno do título"
1423     STFShowMessage(ProcName(0))
1424     lRetorno := .F.
1425 Else
1426     //Tratamento para caso o LOJXREC nao esteja atualizado, onde o mesmo retornava uma variavel logica
1427     LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Estorno efetuado com sucesso")
1428     lRetorno := lRet
1429     STFMessage(ProcName(0), "ALERT", STR0022) //#"Estorno efetuado com sucesso!"
1430     STFShowMessage(ProcName(0))
1431
1432 EndIf
1433 Else
1434     LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Caixa sem permissao para efetuar estorno")
1435     lRetorno := .F.
1436     STFMessage(ProcName(0), "STOP",STR0026) //#"CAIXA SEM PERMISSAO PARA EFETUAR ESTORNO"
1437     STFShowMessage(ProcName(0))
1438 EndIf
1439 LjGrvLog( "Recebimento_Titulo", "Processo de ESTORNO - Final")
1440 CursorArrow()
1441
1442 Return lRetorno
1443
1444
1445 //-----
1446 /*{Protheus.doc} GetReceiptArray
1447 Obtem o array de texto do comprovante impresso
1448 @param lExtrato Se é extrato
1449 @return Array com o texto do comprovante

```

```

1450 @obs
1451 @sample
1452 */
1453 //-----
1454 Method GetReceiptArray(lExtrato) Class STWReceiptTitle
1455
1456 Local aRetorno      := {}           // Array de retorno - Texto do comprovante
1457 Local cTextoParcela := ""           // Texto referente a parcela completo
1458 Local cTextoDocParcela := ""        // Texto referente ao documento da parcela
1459 Local cValorFormatado := 0           // Valor armazenado temporariamente ja formatado, referente a parcela ou valor total
1460 Local nI             := 0           // Contador
1461 Local nDimensao       := STFGetStation("LARGCOL") // Numero de colunas do comprovante de recebimento
1462 Local aTextoTitulo   := {}         // Array Texto Titulo
1463 Local nGetTot         := 0           // Variável para controle do total do título
1464 Local nNccTot         := 0           // Variável para controle total do valor de NCC
1465 Local nSumTot         := 0           // Variável para a soma dos totais do valor do título + NCC
1466 Local cMoeda          := SuperGetMv("MV_SIMB1", .F., "R$") // Primeira moeda valida do sistema
1467
1468 Default lExtrato := .F.
1469
1470
1471 If lExtrato
1472     aTextoTitulo := {STR0011, STR0012, ""} // "E X T R A T O", "D E C O N T A"
1473 ElseIf Self:cOperation == OP_BAIXA
1474     aTextoTitulo := {STR0013, STR0014, IIF(Self:lIsContingency, STR0039, "")} // "C O M P R O V A N T E", "D E R E C E B I M E N T O" // " O F F L I N E"
1475 ElseIf Self:cOperation == OP_ESTOR
1476     aTextoTitulo := {STR0015, STR0016, ""} // "E S T O R N O", "D E R E C E B I M E N T O"
1477 EndIf
1478
1479 // 0 comprovante deve possuir no mínimo 40 colunas
1480 If ValType(nDimensao) == "N" .AND. nDimensao >= 40
1481     // Cabecalho
1482     Aadd(aRetorno, PadC(aTextoTitulo[1], nDimensao))
1483     Aadd(aRetorno, PadC(aTextoTitulo[2], nDimensao))
1484     Aadd(aRetorno, PadC(aTextoTitulo[3], nDimensao))
1485     Aadd(aRetorno, PadR("", nDimensao))
1486     Aadd(aRetorno, PadR("", nDimensao))
1487
1488     cTextoDocParcela := STR0040 + Self:cCnpjCpf // "CPF : "
1489     Aadd(aRetorno, PadR(cTextoDocParcela, nDimensao))
1490     If !Empty(Self:cName)
1491         cValorFormatado := STR0041 + Self:cName // "NOME : "
1492         Aadd(aRetorno, PadR(cValorFormatado, nDimensao))
1493     EndIf
1494
1495     Aadd(aRetorno, PadR("", nDimensao))
1496
1497 // Se Recebimento Normal ou Extrato, imprime as parcelas (nao imprime por contingencia)
1498 If !Self:lIsContingency .OR. lExtrato
1499     //////////////////////////////////////
1500     //Verifico se existe NCC estornada para a respectiva impressão e soma correta dos valores//
1501     //////////////////////////////////////

```

```

1502     If Self:nTotNcc > 0
1503         Aadd(aRetorno, PadR("", nDimensao))
1504         Aadd(aRetorno, PadR(STR0046 + cMoeda + " " + LTrim(Trans(Self:nTotNcc,"@E 999,999,999.99")),nDimensao))// "Valor NCC: "
1505         Aadd(aRetorno, PadR("", nDimensao))
1506     EndIf
1507
1508     Aadd(aRetorno, PadR(PadR(STR0017, nDimensao - Len(STR0018)) + STR0018, nDimensao))// "Documento" ### "Valor"
1509
1510     // Parcelas
1511     For nI := 1 To Len(Self:aListTitles[4])
1512         // Na emissao do extrato imprime todas as parcelas, no recebimento, somente as selecionadas
1513         If Self:aListTitles[4][nI][TIT_SELE] .OR. lExtrato
1514             cValorFormatado := FormatValueCurrency(Self:aListTitles[4][nI][TIT_RECE])
1515             cTextoDocParcela := Self:aListTitles[4][nI][TIT_PREF] + "/" + Self:aListTitles[4][nI][TIT_NUME] + "/" + Self:aListTitles[4][nI][TIT_PARC]
1516             cTextoParcela := cTextoDocParcela + Space(nDimensao - Len(cTextoDocParcela) - Len(cValorFormatado)) + cValorFormatado
1517
1518             Aadd(aRetorno, PadR(cTextoParcela, nDimensao))
1519         EndIf
1520     Next nI
1521 ElseIf Self:lIsContingency
1522
1523     cTextoDocParcela := STR0042 + Dtoc(Self:dDtRec) + " " + Left(Self:cHrRec,5) // "Data/Hora: "
1524     cValorFormatado := STR0043 + xNumCaixa() // "Caixa: "
1525
1526
1527     cTextoParcela := cTextoDocParcela + Space(nDimensao - Len(cTextoDocParcela) - Len(cValorFormatado)) + cValorFormatado
1528     Aadd(aRetorno, PadR(cTextoParcela, nDimensao))
1529
1530     Aadd(aRetorno, PadR("", nDimensao))
1531
1532     Aadd(aRetorno, PadR(PadR(STR0044, nDimensao - Len(STR0018)) + STR0018, nDimensao))// "Documento Informado" ### "Valor"
1533
1534     cValorFormatado := FormatValueCurrency(Self:GetTotal(lExtrato))
1535
1536     If !Empty(Self:cPrefix) .OR. !Empty(Self:cNumber) .OR. !Empty(Self:cParcel)
1537         cTextoDocParcela := Self:cPrefix + "/" + Self:cNumber + "/" + Self:cParcel
1538
1539     Else
1540         cTextoDocParcela := STR0045 // "<< NÃO INFORMADO >>"
1541     EndIf
1542
1543     cTextoParcela := cTextoDocParcela + Space(nDimensao - Len(cTextoDocParcela) - Len(cValorFormatado)) + cValorFormatado
1544     Aadd(aRetorno, PadR(cTextoParcela, nDimensao))
1545 EndIf
1546
1547 ///////////////////////////////////////////////////////////////////
1548 //Analisando resultado final para impressão do rodapé//
1549 ///////////////////////////////////////////////////////////////////
1550 nGetTot := Self:GetTotal(lExtrato)
1551 nNccTot := Self:nTotNcc
1552
1553 nSumTot := Iif(nNccTot >= nGetTot .Or. Self:cOperation == OP_BAIXA ,nGetTot,nGetTot + nNccTot)

```

```

1554
1555     // Rodapé
1556     cValorFormatado := FormatValueCurrency(nSumTot)
1557     Aadd(aRetorno, PadR("", nDimensao))
1558     Aadd(aRetorno, PadR(STR0019 + Space(nDimensao - Len(STR0019) - Len(cValorFormatado)) + cValorFormatado, nDimensao))/"T O T A L"
1559 Else
1560     STFMessage(ProcName(0), "STOP", STR0004) //"0 número de colunas do comprovante deve ser ao menos 40. Configure no cadastro de estações o campo LG_LARGCOL"
1561     STFSHOWMESSAGE(ProcName(0))
1562 EndIf
1563
1564 Return aRetorno
1565
1566
1567 //-----
1568 /*{Protheus.doc} ExtractPrint()
1569 Emite impressao do Extrato da conta
1570 @return .T. Quando impresso corretamente / .F. Quando houver falha na impressao
1571 @obs
1572 @sample
1573 */
1574 //-----
1575 Method ExtractPrint() Class STWReceiptTitle
1576
1577 Local lRetorno      := .F.      // Retorno
1578 Local cTextoComprovante := ""    // Texto do comprovante
1579
1580 If Self:LoadTitles()
1581     If Self:CountParcels() > 0
1582         cTextoComprovante := GetTextArray(Self:GetReceiptArray(.T.))
1583
1584         If IFRelGer( 0, cTextoComprovante, 1) == 0
1585             lRetorno := .T.
1586         Else
1587             STFMessage(ProcName(0), "STOP", STR0007) //"Falha na impressão do extrato."
1588             STFSHOWMESSAGE(ProcName(0))
1589         EndIf
1590     EndIf
1591 EndIf
1592
1593 Return lRetorno
1594
1595 //-----
1596 /*{Protheus.doc} VldCustomer
1597 Pesquisa o CPF/CNPJ do cliente desejado
1598 @param cCodCli Cliente
1599 @param cLojCli Loja
1600 @param cCPF CPF do Cliente
1601 @return aRet - Array com dados do cliente
1602 @obs
1603 @sample
1604 */
1605 //-----

```

```

1606 Method VldCustomer(cCodCli, cLojCli, cCPF) Class STWReceiptTitle
1607
1608 Local aArea      := GetArea()          // Armazena alias corrente
1609 Local aRet       := {}                // Array retorno
1610
1611 Default cCodCli   := ""
1612 Default cLojCli   := ""
1613 Default cCPF      := ""
1614
1615 If !Empty(cCodCli) .AND. !Empty(cLojCli)
1616     DbSelectArea("SA1")
1617     DbSetOrder(1)//A1_FILIAL+A1_COD+A1_LOJA
1618     If DbSeek(xFilial("SA1") + cCodCli + cLojCli )
1619         cCPF := SA1->A1_CGC
1620     EndIf
1621 ElseIf !Empty(cCPF)
1622     DbSelectArea("SA1")
1623     DbSetOrder(3)//A1_FILIAL+A1_CGC
1624     If DbSeek(xFilial("SA1") + cCPF)
1625         cCodCli := SA1->A1_COD
1626         cLojCli := SA1->A1_LOJA
1627     EndIf
1628 EndIf
1629
1630 Add(aRet,cCodCli)
1631 Add(aRet,cLojCli)
1632 Add(aRet,cCPF)
1633
1634 RestArea(aArea)
1635
1636 Return aRet
1637
1638 //-----
1639 /*{Protheus.doc} GetTextArray
1640 Obtem texto com quebra de linha(chr 10) com base no vetor
1641 @param aComprovante Vetor onde cada registro corresponde a uma linha
1642 @return Texto do vetor
1643 @obs
1644 @sample
1645 */
1646 //-----
1647 Static Function GetTextArray(aComprovante)
1648
1649 Local nI      := 0    // Contador
1650 Local cRetorno := ""  // Retorno da funcao
1651
1652 Default aComprovante := {}
1653
1654 For nI := 1 To Len(aComprovante)
1655     cRetorno += aComprovante[nI] + Chr(10)
1656 Next nI
1657

```

```

1658 Return cRetorno
1659
1660
1661 //-----
1662 /*{Protoheus.doc} FormatValueCurrency
1663 Formata o valor numerico como monetário
1664 @param  nValue - Valor a formatar
1665 @return  String de valor formatado como monetário
1666 @obs
1667 @sample
1668 */
1669 //-----
1670 Static Function FormatValueCurrency(nValue)
1671 Local cRet := ""
1672
1673 Default nValue := 0 // Retorno da funcao
1674
1675 cRet := SuperGetMV("MV_SIMB1") + Trans(nValue, "@E 999,999,999.99")
1676
1677 Return cRet
1678
1679 //-----
1680 /*{Protoheus.doc} STWVldFldRec
1681 Função responsável por validar se habilita ou não os campos Multa,Juros e Descontos
1682 para edição para o titulo selecionado.
1683
1684 @param  aVldTitulo - Titulo selecionado pelo usuario
1685 @return  Logico .T. -> Habilita para edição / .F. ->Desabilita edição de campos
1686 @obs
1687 @sample
1688 */
1689 //-----
1690 Function STWVldFldRec(aVldTitulo, lAtuValMulta, lAtuValJuros, lAtuValDesconto, ;
1691                      lAtuValtotal, cTp0p          , aLista          , aSelecionado)
1692 Local lRet := .T.
1693 Local xRet := .T.
1694 Local nI   := 0
1695 Local oObjRec := STIRetObjTit()
1696
1697 DEFAULT lAtuValMulta := .T.
1698 DEFAULT lAtuValJuros := .T.
1699 DEFAULT lAtuValDesconto := .T.
1700 DEFAULT lAtuValtotal := .T.
1701 DEFAULT aLista := {} //Lista dos Titulos
1702 DEFAULT aSelecionado := {} //Lista dos Titulos SELECIONADOS
1703 DEFAULT cTp0p := ""
1704
1705
1706 LjGrvLog( "Recebimento_Titulo", "STWVldFldRec(Valida edicao de campos) - Inicio")
1707
1708 If Len(aLista) > 0 .And. Len(aSelecionado)>0
1709     cTp0p := iIf(cTp0p=="R","1","2") //1-Recebimento /2-Estorno de recebimento

```

```

1710 EndIf
1711
1712 If ExistBlock("STValidRec")
1713
1714     nTotAntes := 0
1715     For nI := 1 To Len(aSelecioneado)
1716         If aSelecioneado[nI]
1717             nTotAntes := oObjRec:GetParcTotal(nI)
1718         EndIf
1719     Next nI
1720     LjGrvLog( "Recebimento_Titulo", "Antes do STVALIDREC - oObjRec:GetTotal():",nTotAntes)
1721     LjGrvLog( "Recebimento_Titulo", "Antes do STVALIDREC - aLista:",aLista)
1722     xRet := ExecBlock("STValidRec",.F.,.F.,{aVldTitulo,cTpOp,aLista , aSelecioneado})
1723
1724     nTotDepois := 0
1725     For nI := 1 To Len(aSelecioneado)
1726         If aSelecioneado[nI]
1727             nTotDepois := oObjRec:GetParcTotal(nI)
1728         EndIf
1729     Next nI
1730
1731     LjGrvLog( "Recebimento_Titulo", "Apos STVALIDREC - oObjRec:GetTotal():",nTotDepois)
1732     LjGrvLog( "Recebimento_Titulo", "Apos STVALIDREC - aLista:",aLista)
1733     LjGrvLog( "Recebimento_Titulo", "Retorno do PE STVALIDREC",xRet)
1734     STIUpdTit()
1735     LjGrvLog( "Recebimento_Titulo", "Apos STIUPDTIT aLista: ",aLista)
1736
1737     If Valtype(xRet) == "A"
1738         lRet                := xRet[1]
1739         lAtuValMulta        := xRet[2]
1740         lAtuValJuros        := xRet[3]
1741         lAtuValDesconto     := xRet[4]
1742         lAtuValtotal        := xRet[5]
1743     Else
1744         lRet := xRet
1745     EndIf
1746 EndIf
1747
1748 If !lRet //caso nao valide as alteracoes no titulo e retorne falso , desabilita os campos
1749     lAtuValMulta := .F.      //Habilita a edição dos campos de multa - STIRecSel
1750     lAtuValJuros := .F.      //Habilita a edição dos campos de juros - STIRecSel
1751     lAtuValDesconto := .F.    //Habilita a edição dos campos de desconto - STIRecSel
1752     lAtuValtotal := .F.      //Habilita a edição dos campos de Total - STIRecSel
1753 EndIf
1754
1755 LjGrvLog( "Recebimento_Titulo", "STWVldFldRec(Valida edicao de campos) - Fim. lRet",lRet)
1756
1757 Return lRet
1758
1759 //-------------------------------------
1760 /*/{Protheus.doc} STWReceTotalizer
1761 Função responsavel retornar a informação de forma de pagamento ou algum totalizador

```

```

1762 do recebimento de titulo
1763
1764 @param cInfo - Nome da TAG no SIGALOJA.ini ou FORMA para forma de pagamento
1765         cTipo - Tipo de operação
1766
1767 @return Caracter - Informação da forma ou do totalizador a ser impresso.
1768 @obs
1769 @sample
1770 /*/
1771 //-----
1772 Function STWReceTotalizer(cInfo, cTipo)
1773 Local cRet := ""
1774 Local cNatRece := IIf( ExistFunc("LjMExePara"), LjMExeParam("MV_NATRECE"), &(SuperGetMV("MV_NATRECE")))
1775 Local lAutomato := If(Type("lAutomatoX")<>"L",.F.,lAutomatoX)
1776
1777 Default cInfo := "TOTALIZADORES"
1778 Default cTipo := "BAIXA"
1779
1780 Do Case
1781
1782     Case AllTrim(Upper(cInfo)) == "FORMA"
1783
1784         cRet := cNatRece
1785
1786     Otherwise
1787         If !lAutomato
1788             If AllTrim(Upper(cTipo)) == "ESTORNO"
1789                 cRet := GetPvProfString("Estorno Venda" , cInfo , "", GetClientDir()+"SIGALOJA.INI")
1790             Else
1791                 cRet := GetPvProfString("Recebimento Titulos" , cInfo , "", GetClientDir()+"SIGALOJA.INI")
1792             EndIf
1793         Endif
1794     EndCase
1795
1796 Return cRet
1797
1798 //-----
1799 /*/{Protheus.doc} PrepSLVArray
1800 Função responsável por retornar os registros a serem incluídos na tabela SLV na retaguarda.
1801 @param aSE5Baixas - Baixas realizadas
1802         aSE5Relacao - Array com as relações Forma de pagamento x titulos
1803         oMdlGrd - Gride de formas de pagamento
1804 @return aRet - Array com informações a serem geradas na tabela SLV.
1805 @obs
1806 @sample
1807 /*/
1808 //-----
1809 Static Function PrepSLVArray(aSE5Baixas,aSE5Relacao,oMdlGrd)
1810 Local aRet := {}
1811 Local nI := 0
1812 Local nY := 0
1813 Local nPosRelacao := 0

```

```

1814
1815 For nI := 1 To Len(aSE5Baixas)
1816
1817     For nY := 1 To Len(aSE5Baixas[nI])
1818
1819         If AllTrim(aSE5Baixas[nI][nY][24]) $ "CC/CD"
1820
1821             nPosRelacao := aScan(aSE5Relacao, {|x| AllTrim(x[1])+AllTrim(x[3])+AllTrim(x[4])+AllTrim(x[5]) ==;
1822                 AllTrim(aSE5Baixas[nI][nY][24])+AllTrim(aSE5Baixas[nI][nY][1])+AllTrim(aSE5Baixas[nI][nY][2])+AllTrim(aSE5Baixas[nI][nY][3]) .And. ;
1823                 x[2]==aSE5Baixas[nI][nY][8] })
1824
1825             If nPosRelacao > 0
1826
1827                 oMdlGrd:GoLine(aSE5Relacao[nPosRelacao][7])
1828
1829                 aAdd(aRet, {aSE5Baixas[nI][nY][1],;           //01-PREFIXO
1830                     aSE5Baixas[nI][nY][2],;           //02-NUMERO
1831                     aSE5Baixas[nI][nY][3],;           //03-PARCELA
1832                     aSE5Baixas[nI][nY][4],;           //04-TIPO
1833                     aSE5Baixas[nI][nY][9],;           //05-E5_SEQ
1834                     aSE5Relacao[nPosRelacao][6] ,;     //06-FILORIG
1835                     aSE5Baixas[nI][nY][5],;           //07-CLIENTE
1836                     aSE5Baixas[nI][nY][6],;           //08-LOJA
1837                     aSE5Baixas[nI][nY][24],;           //09-Forma Pgto
1838                     aSE5Baixas[nI][nY][8],;           //10-Valor
1839                     oMdlGrd:GetValue("L4_DATATEF"),;    //11-Data
1840                     oMdlGrd:GetValue("L4_HORATEF"),;    //12-Hora
1841                     oMdlGrd:GetValue("L4_DOCTEF"),;     //13-DOC
1842                     oMdlGrd:GetValue("L4_AUTORIZ"),;    //14-Autorizacao
1843                     oMdlGrd:GetValue("L4_INSTITU"),;    //15-Instituicao
1844                     oMdlGrd:GetValue("L4_NSUTEF"),;     //16-NSU
1845                     iIf(!Empty(oMdlGrd:GetValue("L4_TIPCART")), oMdlGrd:GetValue("L4_TIPCART"), aSE5Baixas[nI][nY][24]),; //17-Tipo do cartao
1846                     "",;                                //18-SEQOPER
1847                     0;                                  //19-RECNO
1848                     } )
1849
1850             EndIf
1851
1852         EndIf
1853
1854     Next
1855
1856 Next
1857
1858 Return aClone(aRet)
1859
1860
1861 //-----
1862 /*{Protheus.doc} STWValidRecep
1863 Função responsável por executar o Ponto de Entrada para validação dos titulos selecionados
1864 para verificação se baixa ou não os titulos.
1865 @param      aLista      - Lista com todos os titulos (selecionados e não selecionados)

```

```

1866         aSelecionado - Lista logica com a posição do titulos selecionados.
1867 @return      lRet - .T. -> Permite a baixa dos titulos / .F. -> Não permite a baixa dos titulos
1868 @obs
1869 @sample
1870 /*/
1871 //-----
1872 Function STWValidRecep(cTpOp,aLista,aSelecionado)
1873 Local lRet      := .T.           //Retorno
1874 Local nI        := 0             //Contador
1875 Local aTitulos  := {}            //Relação de Títulos
1876
1877 For nI := 1 To Len(aSelecionado)
1878     If aSelecionado[nI]
1879
1880         aAdd(aTitulos, {aLista[nI,TIT_PREF],; //02 - PREFIXO
1881                         aLista[nI,TIT_NUME],; //03 - TITULO
1882                         aLista[nI,TIT_PARC],; //04 - PARCELA
1883                         aLista[nI,TIT_VREA],; //05 - VENCIMENTO
1884                         aLista[nI,TIT_VALO],; //06 - VALOR
1885                         aLista[nI,TIT_FILI],; //15 - FILIAL
1886                         aLista[nI,TIT_TIPO],; //11 - TIPO DO TITULO
1887                         aLista[nI,TIT_MULT],; //07 - MULTA
1888                         aLista[nI,TIT_JURO],; //08 - JUROS
1889                         aLista[nI,TIT_DESC],; //09 - DESCONTO
1890                         aLista[nI,TIT_RECE]}) //10 - RECEBIMENTO
1891
1892     If aLista[nI,TIT_RECE] <= 0
1893         STFMessage(ProcName(0), "STOP", STR0028) //"Título sem nenhum valor para ser recebido"
1894         STFShowMessage(ProcName(0))
1895         lRet := .F.
1896         Exit
1897     EndIf
1898 EndIf
1899 Next
1900
1901 If lRet .AND. Len(aTitulos) > 0 .AND. ExistBlock("STVALBXREC")
1902     cTpOp := iIf(cTpOp=="R","1","2") //1-Recebimento /2-Estorno de recebimento
1903     lRet := ExecBlock("STVALBXREC",.F.,.F.,{cTpOp,aTitulos})
1904 EndIf
1905
1906 Return lRet
1907
1908 //-----
1909 /*/{Protheus.doc} STWldVlFinal()
1910 Chamada da função responsavel por validar se o valor informado no
1911 no campo Valor final esta correto
1912
1913 @return
1914 /*/
1915 //-----
1916 Function STWldVlFinal(nGetVlrTot, nGetTotOrig, nVlrAdicio )
1917 Local lRet := .T.

```

```

1918 Local nVlrTitulo      := nGetTotOrig + nVlrAdicio
1919
1920 // Valor informado para pagamento deve ser no minimo o valor de ( (Acrescimo + Juros + Multa) - Desconto )
1921 If nGetVlrTot > nVlrTitulo
1922     STFMessage(ProcName(0), "STOP", STR0027) //"Valor digitado maior do que o valor pendente no titulo."
1923     STFShowMessage(ProcName(0))
1924     lRet := .F.
1925 ElseIf nGetVlrTot < nVlrAdicio
1926     STFMessage(ProcName(0), "STOP", STR0047 ) //"Valor digitado menor do que o valor pendente no titulo."
1927     STFShowMessage(ProcName(0))
1928     lRet := .F.
1929 EndIf
1930
1931 Return lRet
1932
1933 //-----
1934 /*{Protheus.doc} STWSCliLoj()
1935 Função responsavel por setar o codigo do cliente e da loja quando
1936 o cliente for selecionado no Menu F6
1937
1938 @return
1939 */
1940 //-----
1941 Function STWSCliLoj(cCodCli, cCodLoja)
1942
1943 Local lRet := .T.
1944
1945 Default cCodCli := ""
1946 Default cCodLoja := ""
1947
1948 If !Empty(cCodCli) .And. !Empty(cCodLoja)
1949     cCodCliTit := cCodCli
1950     cCodLojaTit := cCodLoja
1951 Else
1952     lRet := .F.
1953 EndIf
1954
1955 Return lRet
1956
1957 //-----
1958 /*{Protheus.doc} STWGcliLoj()
1959 Função responsavel por recuperar o codigo do cliente e da loja do
1960 cliente selecionado no Menu F6
1961
1962 @return
1963 */
1964 //-----
1965 Function STWGcliLoj()
1966
1967 Local aRet := {}
1968
1969 aAdd(aRet,cCodCliTit)

```

```

1970 aAdd(aRet,cCodLojaTit)
1971
1972 Return aRet
1973
1974 //-----
1975 /*{Protheus.doc} STWRetMsg()
1976 Quando erro conhecido/tratado/esperado, converte texto que recebeu da retaguarda para um formato amigavel para usuario nao tecnico(caixa)
1977 @return
1978 */
1979 Static Function STWRetMsg( cMsgErro )
1980 Local cRet      := ""
1981 Local nPosIni    := 0
1982 Local nPosFim    := 0
1983 Local cTipoFalha := ""
1984 Local cDetFalha  := ""
1985
1986 Default cMsgErro := ""
1987
1988 //Verifica se erro de registro alocado por outro usuario
1989 //nPosIni := At("Lock failed On Alias=SE1",cMsgErro)
1990 nPosIni := At("Lock failed On Alias",cMsgErro)
1991 If nPosIni > 0
1992     cTipoFalha := "Lock failed On Alias"
1993 EndIf
1994
1995 Do Case
1996
1997     Case cTipoFalha == "Lock failed On Alias"           //Falha por motivo registro alocado na retaguarda
1998
1999         cTipoFalha := SubStr(cMsgErro,nPosIni,24)       //Captura lock com nome da tabela : "Lock failed On Alias=SE1"
2000
2001         nPosIni := At("Nome do Usuario : ",cMsgErro)    //captura nome do usuario que alocou o registro
2002         If nPosIni > 0
2003             nPosFim := At(" ",SubStr(cMsgErro,nPosIni+18)) //procura primeira espaco vazio apos inicio da descricao do nome do usuario
2004             cDetFalha += "[" + AllTrim(SubStr(cMsgErro,nPosIni+18,nPosFim)) + "]"
2005         EndIf
2006
2007     Do Case
2008
2009         Case cTipoFalha == "Lock failed On Alias=SE1"
2010             cRet := STR0032 + cDetFalha                  //"0 Título selecionado(SE1) estava em uso por outro usuário:"
2011
2012         Case cTipoFalha == "Lock failed On Alias=SE8"
2013             cRet := STR0033 + cDetFalha                  //"0 Registro de Saldo Bancário(SE8) estava em uso por outro usuário:"
2014
2015         Case cTipoFalha == "Lock failed On Alias=SA6"
2016             cRet := STR0034 + cDetFalha                  //"0 Registro de Bancos/Caixa(SA6) estava em uso por outro usuário:"
2017
2018         Case cTipoFalha == "Lock failed On Alias=SA1"
2019             cRet := STR0035 + cDetFalha                  //"0 Cadastro de Cliente do título selecionado estava em uso por outro usuário:"
2020
2021     Otherwise

```

```

2022             cRet := STR0036 + SubStr(cTipoFalha,22,3)+STR0037 + cDetFalha    //"Registro da tabela:" ##  " estava em uso por outro usuário:"
2023
2024         EndCase
2025
2026     OtherWise
2027
2028         //Tratamento genérico
2029         cRet := STR0038 + Chr(13) + cMsgErro           //"Ocorreu um evento inesperado na Retaguarda: "
2030
2031 EndCase
2032
2033 Return cRet
2034 //-----
2035 /*{Protheus.doc} StwRecArr()
2036 Realiza a geração dos dados do título e movimento bancário do recebimento
2037 @param aListTitles - Títulos retornados pela rotina de consulta de títulos
2038 @param aCompTit - Títulos utilizadas na compensação
2039 @param aMovTitulos - Array para geração de SE5 local (para ser exibido no fechamento de caixa)
2040 @param aCompNCC - Nccs utilizadas para compensação
2041 @param aListDropTitles - Array com a lista de títulos (aParametros) a baixar na retaguarda
2042 @param aSE5Relacao - Array com a relação forma de pagamento x baixa de titulos x valores
2043 @param aMdlGrd - Model de Pagamentos
2044 @param aMovForma - Array para geração de SE5 local (para ser exibido no fechamento de caixa por forma)
2045 @param cUserName - Usuário que realizou o recebimento
2046 @param aNumCheq - Informações dos cheques
2047 @param cMV_LJRECEB - Parâmetro MV_LJRECEB
2048 @param aTitSel - Array com as formas a serem gerados titulos no SE1 apos a baixa do titulos (para serem compensados posteriormente - CH/CC/CD)
2049 @param lContingency - Recebimento em contingencia (offline)?
2050 @param cCliente - Cliente associado ao título
2051 @param cLojaCli - Loja do Cliente associado ao título
2052 @param oWFReceipt - Objeto de Recebimento
2053 @param aFormas - Formas de pagamento do recebimento
2054 @param nValorRec - Valor recebido offline
2055 @return nSaldoCr - Saldo do título para gerar uma NCC
2056 */
2057 //-----
2058 //Tratamento do Array de Recebimento de títulosd
2059 Function StwRecArr(aListTitles , aCompTit , aMovTitulos , aCompNCC , ;
2060                 aListDropTitles , aSE5Relacao , aMdlGrd , aMovForma , ;
2061                 cUserName , aNumCheq , cMV_LJRECEB , aTitSel , ;
2062                 lContingency , cCliente , cLojaCli , oWFReceipt, ;
2063                 aFormas , nValorRec )
2064
2065 Local nI := 0 //Contador de títulos
2066 Local aTitulos := {} //titulos
2067 Local nC := 0 //Contador de Títulos
2068 Local nPosTit := 0 //Posição do título
2069 Local lPrimBaixa := .T. //Primeira baixa
2070 Local nPrimMulta := 0 //Primeira Multa
2071 Local nPrimJuros := 0 //Primeiro Juros
2072 Local nPrimDescon := 0 //Primeiro Desconto
2073 Local cFormaBx := "" //Forma de Baixa

```

```

2074 Local nSaldoTit      := 0    //Saldo do Título
2075 Local cNumCheq       := 0    //Numero do Cheque
2076 Local nCH            := 0    //Contador de Cheque
2077 Local nValorAbate    := 0    //Valor de Abatimento
2078 Local nTotPago       := 0    //total Pago
2079 Local nSaldoCr       := 0    //Saldo Credor
2080 Local cInfoTit       := ""    //Chave do Título
2081 Local nDifRec        := 0    //Diferença a receber
2082 Local nValTroco      := 0
2083 Local aParametros    := {}
2084
2085 Default aListTitles   := {}
2086 Default aCompTit      := {}
2087 Default aMovTitulos   := {}
2088 Default aCompNCC      := {}
2089 Default aListDropTitles := {}
2090 Default aSE5Relacao   := {}
2091 Default aMdlGrd       := {}
2092 Default aMovForma     := {}
2093 Default cUserName     := {}
2094 Default aNumCheq      := {}
2095 Default cMV_LJRECEB   := ""
2096 Default aTitSel       := {}
2097 Default lContingency  := .F.
2098 Default cCliente      := ""
2099 Default cLojaCli      := ""
2100 Default oWFReceipt    := NIL
2101 Default aFormas        := {}
2102 Default nValorRec      := 0
2103
2104 nDifRec := nValorRec
2105
2106 If lContingency .AND. Len(aListTitles) = 0 .AND. ValType(oWFReceipt) == "0"
2107
2108     aListTitles := {}
2109
2110     aAdd( aListTitles, Array(25) )
2111     aFill(aListTitles[1], 0)
2112     aListTitles[1][TIT_SELE] := .T.                // Posicao logica.
2113     aListTitles[1][TIT_PREF] := oWFReceipt:cPrefix // Prefixo do titulo.
2114     aListTitles[1][TIT_NUME] := oWFReceipt:cNumber // Titulo.
2115     aListTitles[1][TIT_PARC] := oWFReceipt:cParcel // Parcela.
2116     aListTitles[1][TIT_VREA] := Ctod("")           // Vencimento real.
2117     aListTitles[1][TIT_VAL0] := oWFReceipt:nValueContingency // Valor.
2118     aListTitles[1][TIT_RECE] := oWFReceipt:nValueContingency // Recebimento.
2119     aListTitles[1][TIT_TIPO] := ""                 // Tipo.
2120     aListTitles[1][TIT_CONT] := ""                 // Contrato.
2121     aListTitles[1][TIT_CLIE] := oWFReceipt:cCode   // Cliente.
2122     aListTitles[1][TIT_LOJA] := oWFReceipt:cBranch // Loja.
2123     aListTitles[1][TIT_FILI] := ""                 // Filial.
2124     aListTitles[1][TIT_CACR] := .F.                 // Posicao logica.
2125     aListTitles[1][TIT_VENC] := Ctod("")           // Vencimento original.

```

```

2126     aListTitles[1][TIT_LOCK] := "" // Cod de uso do Registro
2127     aListTitles[1][TIT_DTBX] := Ctod("") // Dt. da baixa
2128     aListTitles[1][TIT_SALD] := oWFReceipt:nValueContingency // Saldo do titulo
2129
2130     nValorRec := oWFReceipt:nValueContingency
2131 EndIf
2132
2133 For nI := 1 To Len( aListTitles )
2134     If aListTitles[nI][TIT_SELE] //Titulo Selecionados?
2135         LjGrvLog( "Recebimento_Titulo", "Titulo SELECONADO(aListTitles[nI])",aListTitles[nI])
2136         nSaldoTit := aListTitles[nI][TIT_RECE] - aListTitles[nI][TIT_MULT] - aListTitles[nI][TIT_JURO] + aListTitles[nI][TIT_DESC]
2137         Add(aTitulos, {nI,, //1-Posição original do titulo
2138             (nSaldoTit) ,; //2-Saldo
2139             aListTitles[nI][TIT_MULT],; //3-Multa
2140             aListTitles[nI][TIT_JURO],; //4-Juros
2141             aListTitles[nI][TIT_DESC]} //5-Desconto
2142
2143         //Para compensar com nccs caso necessário
2144         aAdd(aCompTit, {Nil,, //1-Nil
2145             aListTitles[nI][TIT_MULT],; //2-Multa
2146             aListTitles[nI][TIT_JURO],; //3-Juros
2147             aListTitles[nI][TIT_DESC],; //4-Desconto
2148             aListTitles[nI][TIT_RECNO]} //5-Recno
2149
2150         aAdd(aMovTitulos, aListTitles[nI] )
2151
2152     EndIf
2153 Next
2154
2155 //IMPORTANTE:
2156 //Este tratamento abaixo consiste na no abatimento dos titulos de acordo com as formas de pagamento,
2157 //ou seja, enquanto o valor da forma de pagamento utilizada for maior que o valor do titulo, será usada a forma.
2158 //Assim que a forma acabar seu saldo, será utilizada outra forma para baixar o titulo. Ate o momento em que os dois
2159 //se acabem.
2160 //Obs.:Este tratamento nao contempla baixas parcial, pois ate o momento o totvs pdv nao contempla essa funcionalidade.
2161
2162 nC := 0
2163 For nC := 1 to Len(aMdlGrd)
2164     aAdd(aFormas,{ aMdlGrd[nC, 09] ,; // 9 Forma de pagamento
2165         aMdlGrd[nC, 10] ,; //10 Valor da forma de pagamento
2166         aMdlGrd[nC, 01]} )
2167
2168     nTotPago += aMdlGrd[nC, 10]
2169 Next nC
2170
2171 If nValorRec > 0
2172     nValTroco := nTotPago - nValorRec
2173 EndIf
2174
2175 nCH := 1
2176 While (Len(aTitulos) > 0 .And. Len(aFormas) > 0)
2177

```

```

2178  /** Tratamento para baixa **
2179  nPosTit := aTitulos[1][1] //Atualiza posição do titulo original
2180
2181  // Os valores de Multa, Juros e Descontos serao aplicados somente na primeira baixa de cada titulo
2182  // Assim, se nao for a primeira baixa estes campos sao zerados no array
2183  If lPrimBaixa
2184      nPrimMulta := aListTitles[nPosTit][TIT_MULT]
2185      nPrimJuros := aListTitles[nPosTit][TIT_JURO]
2186      nPrimDescon := aListTitles[nPosTit][TIT_DESC]
2187      aTitulos[1][2] += (nPrimMulta + nPrimJuros - nPrimDescon)
2188  Else
2189      nPrimMulta := 0
2190      nPrimJuros := 0
2191      nPrimDescon := 0
2192  EndIf
2193
2194  //Atualiza a forma de pagamento
2195  cFormaBx := AllTrim(aFormas[1][1])
2196
2197  //Atualiza valor da forma de pagamento
2198  nVlForma := aFormas[1][2]
2199
2200  //Atualiza saldo do titulo
2201  nSaldoTit := aTitulos[1][2]
2202
2203  // Tratamento para pegar o numero do Cheque
2204  If cFormaBx == "CH"
2205      cNumCheq := aNumCheq[nCH]
2206
2207      If nVlForma <= nSaldoTit
2208          nCH++
2209      EndIf
2210  EndIf
2211
2212  //Caso o valor da forma seja maior significa que o restante de seu valor
2213  //sera usado no proximo titulo
2214  If nVlForma > nSaldoTit
2215      nValorAbate := nSaldoTit
2216  Else
2217      //Caso o valor do titulo seja maior que o valor da forma de pgto
2218      //significa que sera uma baixa parcial do titulo com a forma
2219      nValorAbate := nVlForma
2220  EndIf
2221
2222  //Abate os valores de baixa dos respectivos arrays de controles
2223  aFormas[1][2] -= nValorAbate
2224  aTitulos[1][2] -= nValorAbate
2225  nDifRec -= nValorAbate
2226
2227  //Subtrai o valor pago
2228  nTotPago -= nValorAbate
2229

```

```

2230 //Quando a forma for NCC NÃO pode ser incluída no array de baixas a serem realizadas, pois a mesma será
2231 //compensada primeiramente à baixa. Porém faz-se necessário calcular a simulação da sua baixa para que não ocorra
2232 //a relação de títulos compensados e o mesmo estar na baixa da forma. (Cenário de NCC + Formas de pgto + baixa de vários títulos).
2233 If !(cFormaBx == "CR" .And. Len(aCompNcc) > 0)
2234
2235 //Parametros para função de baixa na retaguarda
2236 aParametros := {}
2237 Aadd(aParametros, cMV_LJRECEB) //1
2238 Aadd(aParametros, aListTitles[nPosTit][TIT_PREF]) //2
2239 Aadd(aParametros, aListTitles[nPosTit][TIT_NUME]) //3
2240 Aadd(aParametros, aListTitles[nPosTit][TIT_PARC]) //4
2241 Aadd(aParametros, aListTitles[nPosTit][TIT_VREA]) //5
2242 Aadd(aParametros, aListTitles[nPosTit][TIT_VALO]) //6
2243 Aadd(aParametros, nPrimMulta) //7
2244 Aadd(aParametros, nPrimJuros) //8
2245 Aadd(aParametros, nPrimDescon) //9
2246 Aadd(aParametros, nValorAbate) //10
2247 Aadd(aParametros, aListTitles[nPosTit][TIT_TIPO]) //11
2248 Aadd(aParametros, aListTitles[nPosTit][TIT_CONT]) //12
2249 Aadd(aParametros, aListTitles[nPosTit][TIT_FILI]) //13
2250 Aadd(aParametros, cFormaBx) //14
2251 Aadd(aParametros, lPrimBaixa) //15
2252 Aadd(aParametros, nPrimMulta) //16
2253 Aadd(aParametros, nPrimJuros) //17
2254 Aadd(aParametros, nPrimDescon) //18
2255 Aadd(aParametros, 0) //19
2256 Aadd(aParametros, {}) //20
2257 Aadd(aParametros, .F.) //21
2258 Aadd(aParametros, cUserName) //22
2259 Aadd(aParametros, aListTitles[nPosTit][TIT_ABAT]) //23
2260 Aadd(aParametros, {}) //24
2261 Aadd(aParametros, {}) //25
2262 Aadd(aParametros, cNumCheq) //26
2263 Aadd(aParametros, {}) //27
2264
2265 //Este array faz com que seja criada corretamente as tabelas SLV na retaguarda
2266 //De forma que junto com o controle de baixa de título acima
2267 //seja respeitado os valores das baixas na geração da SLV
2268 aAdd(aSE5Relacao, { cFormaBx ; //1-forma de pagamento utilizadas
2269 nValorAbate ; //2-Valor de abate do título para a forma
2270 aListTitles[nPosTit][TIT_PREF] ; //3-Prefixo do título
2271 aListTitles[nPosTit][TIT_NUME] ; //4-Numero do título
2272 aListTitles[nPosTit][TIT_PARC] ; //5-Parcela do título
2273 aListTitles[nPosTit][TIT_FILI] ; //6-Filial de origem
2274 aFormas[1,3] }) //7-Ordem da forma de pagamento
2275
2276 aAdd(aMovForma, aSE5Relacao[Len(aSE5Relacao)])
2277
2278 /**
2279 /***
2280 /****
2281 /***** Tratamento para Inclusão de título SE1 (Formas de pagamentos diferente de R$) ****

```

```

2282     cCliente := aListTitles[nPosTit][TIT_CLIE]
2283     cLojaCli := aListTitles[nPosTit][TIT_LOJA]
2284
2285     cInfoTit := aListTitles[nPosTit][TIT_PREF]+aListTitles[nPosTit][TIT_NUME]+aListTitles[nPosTit][TIT_PARC]
2286
2287     //Se ja foi incluido no array nao precisa incluir novamente
2288     If aScan(aTitSel, {|x| cInfoTit == x[2]+x[3]+x[4] }) == 0
2289         aAdd(aTitSel,{ .T. ,;
2290             aListTitles[nPosTit][TIT_PREF],;
2291             aListTitles[nPosTit][TIT_NUME],;
2292             aListTitles[nPosTit][TIT_PARC],;
2293             aListTitles[nPosTit][TIT_VREA],;
2294             aListTitles[nPosTit][TIT_VALO],;
2295             aListTitles[nPosTit][TIT_MULT],;
2296             aListTitles[nPosTit][TIT_JURO],;
2297             aListTitles[nPosTit][TIT_DESC],;
2298             aListTitles[nPosTit][TIT_RECE],;
2299             aListTitles[nPosTit][TIT_TIPO],;
2300             aListTitles[nPosTit][TIT_CONT],;
2301             aListTitles[nPosTit][TIT_CLIE],;
2302             aListTitles[nPosTit][TIT_LOJA],;
2303             aListTitles[nPosTit][TIT_FILI],;
2304             aListTitles[nPosTit][TIT_REC],;
2305             aListTitles[nPosTit][TIT_ACRS],;
2306             aListTitles[nPosTit][TIT_CACR],;
2307             aListTitles[nPosTit][TIT_ABAT],;
2308             aListTitles[nPosTit][TIT_VENC],;
2309             aListTitles[nPosTit][TIT_LOCK],;
2310             aListTitles[nPosTit][TIT_MOED],;
2311             aListTitles[nPosTit][TIT_INTE],;
2312             aListTitles[nPosTit][TIT_DTBX],;
2313             aListTitles[nPosTit][TIT_SALD]})
2314     EndIf
2315
2316 EndIf
2317
2318 //Deletando o titulo do array de controle caso seu saldo
2319 //seja zero ou negativo, pois o mesmo não sera mais usado
2320 If aTitulos[1][2] <= 0
2321     aDel(aTitulos,1)
2322     aSize(aTitulos, len(aTitulos)-1)
2323     lPrimBaixa := .T.
2324 Else
2325     lPrimBaixa := .F.
2326 EndIf
2327
2328 //Deletando a forma de pgto do array de controle caso seu saldo
2329 //seja zero ou negativo, pois o mesmo não sera mais usado
2330 If aFormas[1][2] <= 0
2331     aDel(aFormas,1)
2332     aSize(aFormas, len(aFormas)-1)
2333 ElseIf Len(aFormas) == 1 .And. Len(aTitulos) <= 0 .And. nValTroco == 0

```

```

2334     If lContingency
2335         nValTroco := 0
2336     Else
2337         nValTroco := aFormas[1][2]
2338     EndIf
2339 EndIf
2340
2341 If Len(aParametros) > 0
2342     Aadd(aParametros, nValTroco)           //28 - TROCO
2343     Aadd(aParametros, 1)                   //29
2344     Aadd(aListDropTitles, aParametros)
2345 EndIf
2346
2347 End
2348
2349 //Calcula o Valor da NCC a ser gerada - Valor recebido - Valor do Título (em caso de recebimento off-line)
2350 If nValorRec = 0
2351     nSaldoCr := nTotPago
2352 Else
2353     nSaldoCr := nDifRec
2354 EndIf
2355
2356 Return nSaldoCr
2357
2358 //-----
2359 /*{Protheus.doc} STWReCoHost
2360 Verifica Conexão com o Host
2361 @return lConnect - Conectado com o Host
2362 @obs
2363 @sample lConnect := STWReCoHost()
2364 */
2365 //-----
2366 Static Function STWReCoHost()
2367     Local lConnect := .F.           //Conectou?
2368
2369 //Testa a comunicacao atraves da STFTTestRemoteExecute
2370 lConnect := STFTTestRemoteExecute()
2371
2372 Return lConnect
2373
2374
2375 //-----
2376 /*{Protheus.doc} GetParcValor
2377 Retorna o valor origem da parcela
2378 @param nI      Posicao da parcela no array aListTitles
2379 @return Valor origem da parcela
2380 @obs
2381 @sample
2382 */
2383 //-----
2384 Method GetParcValor(nI) Class STWReceiptTitle
2385

```

```

2386 Local nValor := 0 //Retorno
2387
2388 Default nI := 0
2389
2390 If nI > 0
2391     nValor := Self:aListTitles[4][nI][TIT_VAL0]
2392 EndIf
2393
2394 Return nValor
2395
2396 //-----
2397 /*{Protheus.doc} STWESTTEFT
2398 description Estorna o TEF
2399 @param aFPagtos, array com dados das formas de pagamentos
2400 */
2401 //-----
2402
2403 Static Function STWESTTEFT(aFPagtos)
2404
2405 Local oTEF20 := Nil
2406 Local aTEF := {}
2407 Local lCmpsIdTRN := SLV->(ColumnPos("LV_TRNID")) > 0 .And. SLV->(ColumnPos("LV_TRNPCID")) > 0 .And. SLV->(ColumnPos("LV_TRNEXID")) > 0 //Verifica se existem os campos que
    guardam os IDs das transações TEF
2408 Local aRegSLV := {} //Array com registro do SLV
2409 Local nSLVPrefix := TamSx3("LV_PREFIX0")[1]
2410 Local nSLVNumero := TamSx3("LV_NUMERO")[1]
2411 Local nSLVParc := TamSx3("LV_PARCELA")[1]
2412 Local cKey := ""
2413 Local nI := 0
2414 Local nX := 0
2415 Local nPosSLV := 0
2416 Local lRetTRN := .F.
2417
2418 If lCmpsIdTRN
2419
2420     For nI := 1 To Len(aFPagtos[1])
2421
2422         If aFPagtos[1][nI][1] $ "CC/CD/PX/PD"
2423
2424             cKey := Padr(AllTrim(aFPagtos[1][nI][3]),nSLVPrefix)
2425             cKey += Padr(AllTrim(aFPagtos[1][nI][4]),nSLVNumero)
2426             cKey += Padr(AllTrim(aFPagtos[1][nI][5]), nSLVParc)
2427
2428             aRegSLV:={}
2429
2430             If STBRemoteExecute( "STGETSLV",{xFilial("SLV")+cKey},Nil,.F.,@aRegSLV )
2431
2432                 If len(aRegSLV) > 0
2433
2434                     aTEF := AClone(aRegSLV)
2435
2436

```

```
2437             Exit
2438
2439         Endif
2440
2441     Endif
2442
2443 Endif
2444
2445 Next
2446
2447 For nX := 1 To Len(aTEF)
2448
2449     If !Empty(aTEF[nX][6])//Data da Transação
2450
2451         STBFunAdm(,aTEF[nX], @lRetTRN)
2452
2453         If !lRetTRN
2454             If MsgYesNo(STR0049+ Chr(10)+Chr(12)+STR0050) //"Ocorreu erro no Cancelamento TEF." //"Deseja tentar Novamente?"
2455                 nX := nX - 1
2456             Else
2457                 MsgInfo("",STR0051) //"Cancelamento TEF não foi realizado. Favor cancelar a transação TEF através do Menu TEF Gerencias"
2458             Endif
2459         Endif
2460     Endif
2461 Endif
2462 Next
2463
2464 Endif
2465
2466 Return
```

Componentes/RechargeMobile/STBGetSetRecMob.prw

39 linhas após remoção de metadados de autoria/data

```
1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static aRecharge := {} //Guarga o retorno da recarga de celular
5
6
7 //-----
8 /*{Protheus.doc} STBSetRec
9 Seta o retorno da recarga de celular
10
11 @param      aRec - Dados da recarga
12 @return     Nil
13 @obs
14 @sample
15 */
16 //-----
17 Function STBSetRec( aRec )
18
19 Default aRec := {}
20
21 ParamType 0 Var      aRec      As Array      Default      {}
22
23 aRecharge := aRec
24
25 Return Nil
26
27
28 //-----
29 /*{Protheus.doc} STBGetRec
30 Retorna os dados da recarga de celular
31
32 @param
33 @return aRecharge - Array com dados da recarga
34 @obs
35 @sample
36 */
37 //-----
38 Function STBGetRec()
39 Return aRecharge
```

Componentes/RechargeMobile/STBPrepDatRechMob.prw

251 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBPREPDATRECHMOB.CH"
4 #DEFINE __FORMATEF "CC;CD"
5
6 //-----
7 /*{Protheus.doc} STDRechMob
8 Prepara as informações referente a recarga para serem gravadas na retaguarda
9
10 @param      cCodeItem Codigo do item de recarga
11 @return      Nil
12 @obs
13 @sample
14 */
15 //-----
16 Function STBRechMob(oRetTran)
17
18 Local oDlgVA      := Nil           //Objeto da tela
19 Local oDlgChq     := Nil           //Objeto da tela
20 Local lBotaoOK    := .F.           //Variavel de retorno
21 Local lCancela    := .F.           //Variavel de cancelamento
22 Local nVezes     := 0              //Parametro da Tela
23 Local nCont      := 0              //Parametro da Tela
24 Local aFontes    := {}             //Configuração de fontes para tela
25 Local aDados     := {}             //Dados da transacao
26 Local aDadosChq  := {}             //Dados do TEF
27 Local cNatureza   := ""
28 Local lLjMexeParam := FindFunction("LjMexeParam")
29 Local cDocTEF     := ""
30 Local cNSUTEF     := ""           //NSU TEF
31 Local lIsCartao   := .F.           //Se o pagamento é em cartão
32 Local oTEF20      := STBGetTef()   //Objeto TEF
33 Local aAdmSel     := {}            //Array da administradora financeira selecionada
34 Local nCodRet     := 0              //Codigo do retorno do RemoteExecute
35 Local uRet        := Nil           //Mensagem de retorno do RemoteExecute
36 Local lLgSerNFisc := SLG->(ColumnPos("LG_SERNFIS")) > 0
37 Local cPrefTit    := ""           //Prefixo do título
38 Local aNotas      := {}            //Array com numeração de NFce
39 Local nCupom      := 0              //Numero do cupom Fiscal
40 Local cMsgLog     := ""            //Log de Mensagem
41 Local lFisPrint   := STFGetCfg("lUseECF") //usa impressora fiscal
42 Local aIsFinanProp := {}            //Armazena se é financeira própria, e se não for armazena o código do cliente
43 Local dDtVcRCel   := CTOD(' / / ') //Data de vencimento Recarga de Celular
44
45 Default oRetTran := Nil
46
47 ParamType 0 Var oRetTran As Object Default Nil
48
49
50 //Testo se é nfce para pegar o numero do cupom pelo ljxdnota
51 cPrefTit := STFGetStat("LG_SERIE")
52 nCupom   := oRetTran:nCupom

```

```

53 If !lFisPrint
54   If lLgSerNFisc .AND. !Empty(cPrefTit := STFGetStation("SERNFIS"))
55     LjxDNota( cPrefTit , 3, .F., 1, @aNotas,,,,,,,,,"DOCNF" ) // DOC/SERIE
56
57     If Len(aNotas) > 0 .AND. Len(aNotas[1]) > 1
58       nCupom := Val(aNotas[1][2])
59     Else
60       LjGrvLog( "STBRecCorBank2","Falha ao obter numeração de Série Não Fiscal, será mantido o número da última venda " + Str(nCupom) )
61     EndIf
62
63
64   Else
65     cMsgLog := STR0030 //"Para realizar a Recarga Não-Fiscal é necessário criar e informar o campo LG_SERNFIS, criado no compatibilizador U_UPDL0111"
66     STFMessage("STBRechMob", "ALERT", cMsgLog)
67     STFShowMessage("STBRechMob")
68     LjGrvLog( "STBRechMob",cMsgLog)
69   EndIf
70 EndIf
71
72 aDados :=      {   {STR0001      , cPrefTit}                                ,; //01 //"Serie"
73                   {STR0002      , nCupom}                                ,; //02 //"No Cupom"
74                   {STR0003      , LjParcela(1, SuperGetMV("MV_1DUP"))}    ,; //03 //"Parcela"
75                   {STR0004      , Nil}                                    ,; //04 //"Nat. Opera."
76                   {STR0005      , SuperGetMV("MV_CLIPAD",, "")}          ,; //05 //"Client Padr"
77                   {STR0006      , SuperGetMV("MV_LOJAPAD",, "")}         ,; //06 //"Loja"
78                   {STR0007      , dDataBase}                             ,; //07 //"Dt. Emissao"
79                   {STR0008      , dDataBase}                             ,; //08 //"Dt. Venc."
80                   {STR0009      , "RECARGA DE CELULAR " +;
81                     Iif(GetAPOInfo("LOJA1934.PRW")[4] >= Ctod("01/12/2017"),oRetTran:oRetorno:cCelular + " - ", "- ") +; //Número do Telefone e Estação no
82                     STFGetStation("CODIG0")},;                               //09 //"Historico"
83                   {STR0010      , 1}                                    ,; //10 //"Moeda"
84                   {STR0011      , "LOJXTEF"}                            ,; //11 //"Rotina"
85                   {STR0012      , oRetTran:oRetorno:nValor}             ,; //12 //"Val. Rec."
86                   {STR0013      , xNumCaixa()}                           ,; //13 //"Caixa"
87                   {STR0014      , Nil}                                    ,; //14 //"Tip Form Pg"
88                   {"Doc TEF"    , ""}                                    ,; //15 "Doc TEF"
89                   {"NSU TEF"    , ""}                                    ,; //16 "NSU TEF"
90                   {"AdminFin"   , ""}                                    ,; //17 "Adm financeira"
91                   {"NumMov"     , AllTrim(STDNumMov())}                 ,; //18 "Numero do movimento"
92
93 aDadosChq := { { 0 ,;
94                 dDatabase ,;
95                 1 ,;
96                 Space(TamSx3("EF_BANCO")[1]) ,;
97                 Space(TamSx3("EF_AGENCIA")[1]) ,;
98                 Space(TamSx3("EF_CONTA")[1]) ,;
99                 Space(TamSx3("EF_NUM")[1]) ,;
100                Space(TamSx3("EF_COMP")[1]) ,;
101                Space(TamSx3("L4_RG")[1]) ,;
102                Space(TamSx3("L4_TELEFON")[1]) ,;
103                .F. ,;

```

```

104      .F.                                     ;;
105      SuperGetMV("MV_SIMB1")                 ;;
106      Space(TamSx3("EF_EMITENT")[1]))      ;;
107      Array(14)                              ;;
108      { .F., .F., .F.                       ;;
109      .T., .T., .T.                         ;;
110      .T., .F., .F.                       ;;
111      .F., .F., .F.                       ;;
112      .F., .F. }                           ;;
113      { STR0015                             ;; //"Valor do Titulo"
114        STR0016                             ;; //"Data do Vencimento"
115        STR0017                             ;; //"Parcela"
116        STR0018                             ;; //"Banco"
117        STR0019                             ;; //"Agência"
118        STR0020                             ;; //"Conta"
119        STR0021                             ;; //"Num.Cheque"
120        STR0022                             ;; //"Compensação"
121        STR0023                             ;; //"RG"
122        STR0024                             ;; //"Telefone"
123        STR0025                             ;; //"Utiliza nas próximas parcelas"
124        STR0026                             ;; //"Cheque de Terceiro"
125        STR0027                             ;; //"Moeda"
126        STR0028                             ;; //"Emitente"
127      }
128      If Alltrim(oRetTran:oRetorno:cForma) == "1"
129        aDados[14][2] := SuperGetMV("MV_SIMB1")
130      ElseIf Alltrim(oRetTran:oRetorno:cForma) == "2"
131        aDados[14][2] := "CH"
132      ElseIf Alltrim(oRetTran:oRetorno:cForma) == "3"
133        aDados[14][2] := "CD"
134        cDocTEF      := oRetTran:oRetorno:cNsuAutor
135        cNSUTEF      := oRetTran:oRetorno:cNsu
136      ElseIf Alltrim(oRetTran:oRetorno:cForma) == "4"
137        aDados[14][2] := "CC"
138        cDocTEF      := oRetTran:oRetorno:cNsuAutor
139        cNSUTEF      := oRetTran:oRetorno:cNsu
140      EndIf
141
142      If aDados[14][2] $ __FORMATEF
143        lIsCartao := .T.
144        aDados[15][2] := LjRmvChEs(cDocTEF)
145        aDados[16][2] := LjRmvChEs(cNSUTEF)
146
147        If Len(oRetTran:oRetorno:aAdmin) > 0
148          aDados[17][2] := oRetTran:oRetorno:aAdmin[1,1]
149        Else
150          //Chama tela para seleção da Administradora
151          aAdmSel := STICrdSlAdm( oTEF20:oConfig:aAdmin, oRetTran:oRetorno:nValor )
152          If Len(aAdmSel) == 1
153            oRetTran:oRetorno:aAdmin := aClone(aAdmSel)
154            aDados[17][2] := PadR(aAdmSel[1][1], TamSX3("AE_COD")[1])
155            LjGrvLog( Nil, "Adm.Fin. escolhida: ", aAdmSel[1][3])

```

```

156         If FindFunction("STDIsFinPr")
157             //Retornar se é administradora propria
158             aIsFinanProp := STDIsFinPr(aDados[17][2])
159             //Se não for financeira propria, grava o titulo para a administradora.
160             If !aIsFinanProp[1][1]
161                 aDados[05][2] := aIsFinanProp[1][2]
162             EndIf
163         EndIf
164     Else
165         LjGrvLog(Nil, "Nenhuma Adm.Fin. escolhida" )
166     EndIf
167 EndIf
168
169 If aDados[14][2] == "CC"
170     cNatureza := "MV_NATCART"
171 Else
172     cNatureza := "MV_NATTEF"
173 EndIf
174 ElseIf aDados[14][2] == "CH"
175     cNatureza := "MV_NATCHEQ"
176
177     aFontes := STFDefFont()
178
179     LjTelaCheq( @oDlgChq      , STR0029      , @oDlgVA      , @aDadosChq      , ;           //"DADOS DO CHEQUE"
180                @aFontes      , @lBotaoOK      , @lCancela      , @nVezes      , ;
181                @nCont      )
182
183 ElseIf aDados[14][2] == SuperGetMV("MV_SIMB1")
184     cNatureza := "MV_NATDINH"
185 EndIf
186
187 If !LjMExeParam
188     cNatureza := LjMExeParam(cNatureza)
189 Else
190     cNatureza := SuperGetMV(cNatureza)
191     If Left(cNatureza, 1) == "&"
192         cNatureza := &(SubStr(cNatureza, 2))
193     EndIf
194 EndIf
195
196 aDados[4][2] := cNatureza
197
198 dDtVcRCel := Iif(aDados[14][2] == SuperGetMV("MV_SIMB1"),aDados[8][2],DataValida(aDados[8][2]))
199
200 //Efetua geração do titulo de recarga na retaguarda
201 STBRemoteExecute( "INCTITRCEL",;
202                  { aDados[1][2]          , AllTrim(Str(aDados[2][2])), aDados[3][2] , aDados[14][2] ,;
203                    aDados[4][2]          , aDados[5][2]          , aDados[6][2] , aDados[7][2] ,;
204                    dDtVcRCel            , aDados[9][2]          , aDados[10][2] , aDados[11][2] ,;
205                    aDados[12][2]         , aDados[13][2]         , aDadosChq[1,4] , aDadosChq[1,5] ,;
206                    aDadosChq[1,6]        , aDadosChq[1,7]        , aDadosChq[1,8] , aDadosChq[1,9] ,;
207                    aDadosChq[1,10]       , aDadosChq[1,12]       , aDados[15][2] , aDados[16][2] ,;

```

```

208             aDados[17][2]           , lIsCartao           , aDados[18][2];
209         },;
210         Nil ,.T., @uRet,/*cType*/,/*cKeyOri*/,@nCodRet;
211     )
212
213 //Realiza a Gravação da SE5 para recarga de celular
214 If ExistFunc("STDGrvSE5")
215     STDGrvSE5(aDados[13][2],aDados[7][2],aDados[9][2],aDados[14][2],aDados[10][2],aDados[12][2],aDados[4][2],aDados[11][2])
216 EndIf
217 If nCodRet == -101 .OR. nCodRet == -108
218     LjGrvLog( "Recargar_Celular","Servidor PDV nao Preparado. Funcionalidade nao existe ou host responsavel não associado - INCTITRCEL ")
219     LjGrvLog( "Recargar_Celular","Cadastre a funcionalidade e vincule ao Host da Retaguarda - INCTITRCEL ")
220     STDIncRecM(aDados, aDadosChq, aDados[14][2])
221 EndIf
222
223 Return
224
225
226 //-----
227 /*/{Protheus.doc} STBProdRec
228 Valida se o produto é um item de recarga ou não
229
230 @param      cCodeItem  Código do item de recarga
231 @return      lRet - .T. ou .F. produto é um item de recarga ou não
232 @obs
233 @sample
234 */
235 //-----
236 Function STBProdRec(cCodeItem)
237
238 Local cMvLjPRec      := SuperGetMV("MV_LJPREC", Nil, "")      //Retorna o codigo do produto para recarga
239 Local lRet           := .F.                                   //Retorno da função
240
241 ParamType 0 Var cCodeItem As Character Default ""
242
243 Default cCodeItem := ""
244
245 If !Empty( cMvLjPRec ) .AND. Alltrim(cCodeItem) $ Alltrim(cMvLjPRec)
246     lRet := .T.
247 Else
248     lRet := .F.
249 EndIf
250
251 Return lRet

```

Componentes/RechargeMobile/STDIncludeRecMob.prw

304 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STDINCLUDERECMOB.CH"
4
5
6 //-----
7 /*{Protheus.doc} STDIncDatR
8 Grava as informações da recarga não fiscal nas tabela SE1 e SEF
9 @param aDadosRM   Dados da recarga
10 @param aDadosChq   Dados do cheque
11 @param cTipo       Tipo de recarga
12 @return           Retorna se gravou as Informacoes da recarga
13 @obs
14 @sample
15 */
16 //-----
17 Function STDIncRecM(aDadosRM, aDadosChq, cTipo)
18
19 Local aDados           := {} //Campos do SE1 que necessitam ser enviados
20 Local lContinue        := .F. //Retorno do componente de comunicacao com a retaguarda
21 Local uRet             := Nil //Retorno das funcoes
22 Local cMsg             := ""
23
24 Default aDadosRM       := {}
25 Default aDadosChq      := {}
26 Default cTipo          := ""
27
28 aDados := {
29   { "E1_PREFIXO" ,aDadosRM[1][2] ,Nil} ;; //1
30   { "E1_NUM"      ,AllTrim(Str(aDadosRM[2][2])) ,Nil} ;; //2
31   { "E1_PARCELA"  ,aDadosRM[3][2] ,Nil} ;; //3
32   { "E1_TIPO"     ,aDadosRM[14][2] ,Nil} ;; //4
33   { "E1_NATUREZ"  ,aDadosRM[4][2] ,Nil} ;; //5
34   { "E1_CLIENTE"  ,aDadosRM[5][2] ,Nil} ;; //6
35   { "E1_LOJA"     ,aDadosRM[6][2] ,Nil} ;; //7
36   { "E1_EMISSAO"  ,aDadosRM[7][2] ,Nil} ;; //8
37   { "E1_VENCTO"   ,aDadosRM[8][2] ,Nil} ;; //9
38   { "E1_VENCREA"  ,DataValida(aDadosRM[8][2]) ,Nil} ;; //10
39   { "E1_HIST"     ,aDadosRM[9][2] ,Nil} ;; //11
40   { "E1_MOEDA"    ,aDadosRM[10][2] ,Nil} ;; //12
41   { "E1_ORIGEM"   ,aDadosRM[11][2] ,Nil} ;; //13
42   { "E1_FLUXO"    , "S" ,Nil} ;; //14
43   { "E1_VALOR"    ,aDadosRM[12][2] ,Nil} ;; //15
44   { "E1_PORTADO"  ,aDadosRM[13][2] ,Nil} ;; //16
45   { "E1_BCOCHQ"   ,aDadosChq[1,4] ,Nil} ;; //17
46   { "E1_AGECHQ"   ,aDadosChq[1,5] ,Nil} ;; //18
47   { "E1_CTACHQ"   ,aDadosChq[1,6] ,Nil} ;; //19
48   { "E1_DOCTEF"   , "" ,Nil} }//20
49
50 If Len( aDadosRM ) > 15
51   aDados[20, 02] := aDadosRM[15][2]
52   aAdd(aDados, {"E1_NSUTEF", aDadosRM[16][2], NIL})
53 EndIf

```

```

53
54 lContinue := STBRemoteExecute( "Fina040"      ;;
55                               { aDados }      ;;
56                               Nil              ;;
57                               .T.              ;;
58                               @uRet           )
59
60 If ValType( uRet ) = "A" .AND. Len(uRet) > 1
61
62   If ValType(uRet[2]) = "C"
63     cMsg := uRet[2]
64   EndIf
65
66   If ValType(uRet[1]) = "L"
67
68     lContinue := uRet[1]
69   EndIf
70   If !lContinue
71     If Empty(cMsg)
72       cMsg := STR0001 //"Problemas ao realizar a geração do Título de Contas a Receber"
73     EndIf
74   EndIf
75 EndIf
76
77 If lContinue .AND. cTipo == "CH"
78   uRet := NIL
79   lContinue := STBRemoteExecute( "LJRecGrvCH",;
80                               { aDadosChq[1,4] , aDadosChq[1,5] , aDadosChq[1,6] , aDadosChq[1,7] ,;
81                               aDadosRM[12][2] , aDadosRM[7][2] , aDadosChq[1,8] , aDadosChq[1,9] ,;
82                               aDadosChq[1,10] , aDadosChq[1,12] , aDadosRM[1][2] , AllTrim(Str(aDadosRM[2][2])),;
83                               aDadosRM[3][2] , aDadosRM[14][2] } ,;
84                               Nil ,;
85                               .T. ,;
86                               @uRet )
87
88   If ValType(uRet) = "L"
89     lContinue := uRet
90   EndIf
91
92   If !lContinue
93     If Empty(cMsg)
94       cMsg := STR0002 //"Problemas ao realizar a geração dos dados do Cheque"
95     EndIf
96   EndIf
97
98 If lContinue .And. cTipo == SuperGetMV( "MV_SIMB1" )
99   uRet := NIL
100   aDados := {
101     { "E1_PREFIXO" , PadR( aDadosRM[1][2], TamSx3("E1_PREFIXO")[1]) , Nil},; // 01
102     { "E1_NUM" , PadR(AllTrim(Str(aDadosRM[2][2])), TamSx3("E1_NUM")[1]) , Nil},; // 02
103     { "E1_PARCELA" , PadR(aDadosRM[3][2], TamSx3("E1_PARCELA")[1]) , Nil},; // 03
104     { "E1_TIPO" , PadR( aDadosRM[14][2], TamSx3("E1_TIPO")[1]) , Nil},; // 04
105     { "E1_MOEDA" , aDadosRM[10][2] , Nil},; // 05

```

```

105      { "E1_TXMOEDA"      ,0                      , Nil},; // 06
106      { "E1_ORIGEM"      ,aDadosRM[11][2]          , Nil},; // 07
107      { "AUTVALREC"      ,aDadosRM[12][2]          , Nil},; // 08
108      { "AUTMOTBX"       , "NOR"                  , Nil},; // 09
109      { "AUTDTBAIXA"     ,dDataBase                , Nil},; // 10
110      { "AUTDTCREDITO"   ,dDataBase                , Nil},; // 11
111      { "AUTHIST"        ,aDadosRM[9][2]          , Nil}} // 12
112
113      lContinue := STBRemoteExecute( "FINA070"      ,;
114                                   { aDados, 3 }    ,;
115                                   Nil              ,;
116                                   .T.              ,;
117                                   @uRet           )
118
119      If ValType( uRet ) = "A" .AND. Len(uRet) > 1
120          If ValType(uRet[2]) = "C"
121              cMsg := uRet[2]
122          EndIf
123
124          If ValType(uRet[1]) = "L"
125              lContinue := uRet[1]
126          EndIf
127
128          If !lContinue
129              If Empty(cMsg)
130                  cMsg := STR0003 //"Problemas ao realizar a baixa do título da Recarga de Celular"
131              EndIf
132          EndIf
133      EndIf
134
135  EndIf
136
137  If !lContinue .AND. !Empty(cMsg)
138      STFMessage("STDIncRecM", "ALERT", cMsg)
139      STFShowMessage("STDIncRecM")
140
141  EndIf
142
143  Return lContinue
144
145
146  //-----
147  /*{Protheus.doc} STDIncSE1
148  Grava as informações da recarga não fiscal nas tabela SE1
149
150  @param aDadosRM  Dados da recarga
151  @return  Retorna se gravou as Informacoes da recarga
152  @obs
153  @sample
154  */
155  //-----
156  Function STDIncSE1(aDados)

```

```

157
158 Local aRet := Array(2)
159
160 Default aDados := {}
161
162 Private lMsErroAuto := .F.
163
164 If Len(aDados) > 0
165     MExecAuto( { |x,y| Fina040( x, y ) }, aDados, 3 ) //Inclusao
166     If lMsErroAuto
167         CoNout(MostraErro())
168         aRet[1] := .F.
169         aRet[2] := MostraErro()
170     Else
171         aRet[1] := .T.
172         aRet[2] := ""
173     EndIf
174
175
176 Else
177     aRet[1] := .F.
178     aRet[2] := "Array aDados chegou vazio!"
179 EndIf
180
181 Return aRet
182
183
184 //-----
185 /*{Protheus.doc} STDBxSE1
186 Grava a Baixa do Título gerado na SE1
187
188 @param aDadosRM  Dados da recarga
189 @return  Retorna se gravou a baixa das Informacoes da recarga
190 @obs
191 @sample
192 */
193 //-----
194 Function STDBxSE1(aDados, nOper)
195
196 Local aRet := Array(2)
197
198 Default aDados := {}
199 Default nOper := 3
200
201 Private lMsErroAuto := .F.
202
203 If Len(aDados) > 0
204     //Ajustando os campos chave do Título para não dar Título não localizado
205
206     //Foi colocada a conversão de Data pois o FwCallFuncionality do STBRemote Execute estava retornando TimeStamp na Data, dando erro na baixa
207     AEval(aDados,{|d| IIF( d[1] $ "AUTDTBAIXA|AUTDTCREDITO", d[2] := StoD(Dtos(d[2])),)})
208

```

```

209 MSExecAuto( { |x,y| Fina070( x, y ) }, aDados, n0per ) //Inclusao
210 If lMsErroAuto
211     CoNout(MostraErro())
212     aRet[1] := .F.
213     aRet[2] := MostraErro()
214 Else
215     aRet[1] := .T.
216     aRet[2] := ""
217 EndIf
218 Else
219     aRet[1] := .F.
220     aRet[2] := "Array aDados chegou vazio!"
221 EndIf
222
223 Return aRet
224 //-----
225 /*{Protheus.doc} STDGrvSE5
226 Grava a SE5 da recarda de celular não-fiscal
227
228 @param cPortado      Portador
229 @param dEmiss         Emissão da recarga
230 @param cHist          Historioco da SE5
231 @param cTipo          Tipo da SE5
232 @param nMoeda         Moeda da Venda
233 @param nValTit        Valor da recarga
234 @param cNatureza      Natureza da SE5
235 @param cOrigem        Origem da Gravação
236
237 @return
238 @obs
239 @sample
240 */
241 //-----
242 Function STDGrvSE5(cPortado,dEmiss,cHist,cTipo,nMoeda,nValTit,cNatureza,cOrigem)
243
244 Local aArea      := GetArea()
245 Local aAreaSA6   := SA6->(GetArea())
246
247 SA6->(DbSetOrder(1))
248 SA6->(DbSeek(xFilial("SA6") + cPortado + "."))
249 If SE5->(Reclock("SE5",.T.))
250     REPLACE SE5->E5_FILIAL WITH xFilial("SE5")
251     REPLACE SE5->E5_DATA WITH dEmiss
252     REPLACE SE5->E5_BANCO WITH cPortado
253     REPLACE SE5->E5_AGENCIA WITH SA6->A6_AGENCIA
254     REPLACE SE5->E5_CONTA WITH SA6->A6_NUMCON
255     REPLACE SE5->E5_HISTOR WITH cHist
256     REPLACE SE5->E5_TIPO WITH cTipo
257     REPLACE SE5->E5_TIPODOC WITH "VL"
258     REPLACE SE5->E5_MOEDA WITH StrZero(nMoeda,TAMX3("E5_MOEDA"))[1])
259     REPLACE SE5->E5_VALOR WITH nValTit
260     REPLACE SE5->E5_DTDIGIT WITH dDataBase

```

```

261 REPLACE SE5->E5_DTDISP0 WITH SE5->E5_DATA
262 REPLACE SE5->E5_NATUREZ WITH cNatureza
263 REPLACE SE5->E5_SITUA WITH "TX" //Não é necessário subir para a retaguarda pois ja foi gerada anteriormente
264 REPLACE SE5->E5_ORIGEM WITH cOrigem
265 REPLACE SE5->E5_NUMMOV WITH AllTrim(STDNumMov())
266 SE5->(MsUnLock())
267 EndIf
268
269 RestArea(aAreaSA6)
270 RestArea(aArea)
271
272 Return
273
274 //-----
275 /*{Protheus.doc} STDIsFinPr
276 Retorna se é financeira própria, e se não for armazena o código da financeira(Cliente)
277
278 @param cAdminis      Administradora
279
280 @return      aRet Array com duas posições, sendo a primeira um logico que informa se é financeira propia e a segunda que retorna o codigo da financeira(Cliente) caso não seja.
281 @obs
282 @sample
283 */
284 //-----
285 Function STDIsFinPr(cAdminis)
286
287 Local aArea      := GetArea()
288 Local aAreaSAE   := SAE->(GetArea())
289 Local aRet       := {}           //Armazena se é financeira própria, e se não for armazena o código do cliente
290
291 Default cAdminis := ""
292
293 SAE->(DbSetOrder(1))//AE_FILIAL+AE_COD
294
295 If SAE->(DbSeek(xFilial("SAE") + cAdminis)) .AND. SAE->AE_FINPRO == "N"
296     AADD(aRet,{.F.,SAE->AE_CODCLI})
297 Else
298     AADD(aRet,{.T.,""})
299 EndIf
300
301 RestArea(aAreaSAE)
302 RestArea(aArea)
303
304 Return aRet

```

Componentes/RechargeMobile/STWRechargeMobile.prw

153 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWRECHARGEMOBILE.CH"
4
5
6 //-----
7 /*{Protheus.doc} STWRecMob
8 Workflow para recarga de celular
9
10 Quando o código do item for informado, o sistema assume como uma recarga fiscal
11 caso contrário é uma recarga não fiscal.
12
13 @param      cCodeItem Código do item de recarga
14 @param      oNewTEF Objeto TEF
15 @return      aRetTEF - Array com informacoes do TEF
16 @obs
17 @sample
18 */
19 //-----
20 Function STWRecMob(cCodeItem, oNewTEF)
21
22 Local oDados      := Nil                      //Instancia da classe de transacao de recarga de celular
23 Local oRetTran    := Nil                      //Retorno da transacao da recarga
24 Local aRet        := {}                      //Retorno da função
25 Local cNumCup     := Space(STBLenReceipt())  //Numero do cupom
26 Local aRetTEF     := {}                      //Retorno do TEF
27 Local nDecimais   := 0                      //Numero de casas decimais
28 Local aDados      := {}                      //Dados da transacao
29 Local lEmitNfce := FindFunction("LjEmitNFCe") .AND. LjEmitNFCe() // Sinaliza se utiliza NFC-e
30 Local lFisPrint  := STFGetCfg("lUseECF")      //usa impressora fiscal
31 Local lLgSerNFisc := .F. //Campo de série não fiscal
32 Local cMsgLog     := ""                      //Mensagem de Log da Recarga
33
34 Default cCodeItem := ""
35 Default oNewTEF   := STBGetTEF()
36
37 // Evento pegar Numero do cupom
38 If Empty(cCodeItem)
39
40     If Type("nMoedaCor") <> "U"
41         nDecimais := MsDecimais(nMoedaCor)
42     Else
43         nDecimais := MsDecimais(1)
44     EndIf
45
46 EndIf
47
48
49 //Buca o Numero do Cupom Fiscal
50 aDados := { cNumCup , Nil}
51
52 If lFisPrint

```

```

53     aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
54                          "STGetReceipt" ,; // Nome do evento
55                          aDados)
56 EndIf
57
58 If !FisPrint .AND. !EmitNfce .And. (Len(aRet) == 0 .OR. aRet[1] <> 0 .OR. Len(aDados) == 0)
59     MsgStop( STR0001, "STGetReceipt" ) //"Falha na obtenção do cupom"
60 Else
61
62     cNumCup := aDados[1]
63
64     If !FisPrint //Se for NFe
65         //NFCe pega o numero Da última venda
66         cNumCup := STDCSLastSale("L1_DOC")
67     EndIf
68
69     If Empty(cCodeItem )
70
71         cNumCup := Soma1(cNumCup)
72     EndIf
73
74     oDados := LJCDadosTransacaoRC():New(0, Val(cNumCup), Date(), Time())
75
76     If STBProdRec(cCodeItem)
77
78         oRetTran := oNewTef:RecargaCel():RecFis(oDados)
79
80         If oRetTran:oRetorno:lTransOk
81
82             oNewTef:Cupom():Inserir("V", oRetTran:oRetorno:oViaCaixa,oRetTran:oRetorno:oViaCliente, "R",;
83             oNewTef:RecargaCel():GetTotalizador(), oNewTef:RecargaCel():GetFormaPgto(oRetTran:oRetorno:cForma, oNewTef:Formas()) ,
84             oRetTran:oRetorno:nValor, 1,;
85             0)
86
87             STBSetPric(oRetTran:oRetorno:nValor)
88
89             aAdd(aRetTEF,{'Transacao', .T.})
90             aAdd(aRetTEF,{'Hora', oRetTran:oRetorno:cHora})
91             aAdd(aRetTEF,{'Data', oRetTran:oRetorno:dData})
92             aAdd(aRetTEF,{'Valor', oRetTran:oRetorno:nValor})
93             aAdd(aRetTEF,{'Cupom', oRetTran:nCupom})
94             aAdd(aRetTEF,{'ViaCaixa', oRetTran:oRetorno:cViaCaixa})
95             aAdd(aRetTEF,{'ViaCliente', oRetTran:oRetorno:cViaCliente})
96
97             //oNewTef:Confirmar() - Confirmacao da transacao e na venda
98             STBSetRec(aRetTEF)
99         Else
100             STBSetPric(0)
101             aAdd(aRetTEF,{'Transacao', .F.})
102         EndIf
103

```

```

104 ElseIf Empty(cCodeItem)
105
106     lLgSerNFisc := SLG->(ColumnPos("LG_SERNFIS")) > 0
107
108     If ( lLgSerNFisc .AND. !Empty(STFGetStat("SERNFIS")) ) .OR. lFisPrint
109
110         oRetTran := oNewTef:RecargaCel():RecNFis(oDados)
111
112         If oRetTran:oRetorno:lTransOk
113
114             oNewTef:Cupom():Inserir(
115                 "v"
116                 oRetTran:oRetorno:oViaCaixa      ,;
117                 oRetTran:oRetorno:oViaCliente    ,;
118                 "RNF"                             ,;
119                 oNewTef:RecargaCel():GetTotalizador() ,;
120                 oNewTef:RecargaCel():GetFormaPgto(oRetTran:oRetorno:cForma) ,;
121                 oRetTran:oRetorno:nValor          ,;
122                 1                                  ,;
123                 0                                  )
124
125             If oNewTEF:Cupom():Imprimir()
126                 oNewTef:Confirmar()
127                 //Gravação da transação
128                 STBRechMob(oRetTran)
129             Else
130                 oNewTef:Desfazer()
131             EndIf
132         EndIf
133     Else
134         cMsgLog := STR0002 //"Para realizar a Recarga Não-Fiscal é necessário criar e informar o campo LG_SERNFIS, criado no compatibilizador U_UPDL0111"
135         STFMessage("STWRechMob", "STOP", cMsgLog)
136         STFShowMessage("STWRechMob")
137         LjGrvLog( "STWRechMob",cMsgLog)
138     EndIf
139 EndIf
140
141 If ValType(oDados) == "0"
142     FreeObj(oDados)
143     oNewTef:oConfig:oRecCel:oTrans:aColecao := {}
144 EndIf
145
146 Return aRetTEF

```

Componentes/RecoverySale/STBRecoverySale.prw

371 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBRECOVERYSALE.CH"
4
5 Static lRecovered := .F.
6
7 //-----
8 /*{Protheus.doc} STBRScanRecovery
9 verifica se pode Recuperar Vendas
10
11 @param
12 @return lRet          Retorna se pode recuperar Venda
13 @obs
14 @sample
15 */
16 //-----
17 Function STBRScanRecovery()
18
19 Local lRet := .T.          // Retorna se pode recuperar vendas
20
21 If SuperGetMv("MV_CTRLFOL",,.F.) .AND. AllTrim(cPaisLoc) $ "CHI|COL"
22     lRet := .F.
23 EndIf
24
25 Return lRet
26
27 //-----
28 /*{Protheus.doc} STBRSSStationChange
29 Verifica troca de ECF
30
31 @param
32 @return lRet          Retorna se ocorreu a troca de ECF
33 @obs
34 @sample
35 */
36 //-----
37 Function STBRSSStationChange( cPDV )
38
39 Local lRet          := .F.          // Retorna se trocou ECF
40 Local cPDVPrint := ""          // PDV do periférico
41 Local aPDV        := {}          // Retorno evento get PDV
42 Local aPrinter    := {}
43
44 Default cPDV := ""
45
46 ParamType 0 Var      cPDV      As Character      Default      ""
47
48 If STFUseFiscalPrinter()
49     /*/
50     Get PDV Impressora
51     /*/
52     aPrinter := STFFireEvent( ProcName(0) , "STGetPDV" , aPDV )

```

```

53
54     If ValType(aPrinter) <> "A" .OR. Len(aPrinter) == 0 .OR. aPrinter[1] <> 0 .OR. Len(aPDV) < 1
55
56         STFMessage(ProcName(),"STOP", "Falha na obtenção do cupom")
57         STFShowMessage(ProcName())
58         cPDVPrint := ""
59
60     Else
61         cPDVPrint := AllTrim(aPDV[1])
62     EndIf
63
64 Else
65     cPDVPrint := AllTrim(STFGetStat("PDV"))
66 EndIf
67
68 If cPDVPrint <> AllTrim(cPDV)
69     lRet := .T.
70 EndIf
71
72 Return lRet
73
74
75 //-----
76 /*{Protheus.doc} STBRSHasPrint
77 Verifica se a venda possui itens de reserva e/ou de impressão
78
79 @param   aSL2                Itens da venda
80 @return  aRet[1]             Retorna se possui itens de reserva
81 @return  aRet[2]             Retorna se possui itens de impressão
82 @obs
83 @sample
84 */
85 //-----
86 Function STBRSHasPrint( aSL2 )
87
88 Local lReserve      := .F.        // Possui itens de reserva?
89 Local lPrint        := .F.        // Possui Itens de Impressão?
90 Local nI             := 0         // Contador
91 Local nPosReserve    := 0         // Posicao do campo
92 Local aRet           := {}        // Retorno da funcao
93
94 Default aSL2 := {}
95
96 ParamType 0 Var aSL2 As Array Default {}
97
98 nPosReserve := AScan( aSL2[1] , { |x| x[1] == "L2_RESERVA" } )
99
100 For nI := 1 To Len(aSL2)
101     If Empty(aSL2[nI][nPosReserve][2])
102         lPrint := .T.
103     Else
104         lReserve := .T.

```

```

105     EndIf
106 Next nI
107
108 AADD( aRet , lReserve )
109 AADD( aRet , lPrint      )
110
111 Return aRet
112
113
114 //-----
115 /*{Protheus.doc} STBRNumPrinter
116 Busca o Ultimo Cupom da Impressora
117
118 @param
119 @return  cNumDoc          Retorna o Ultimo Cupom da impressora
120 @obs     Se der erro na impressora retorna "ERROR", se nao usa impressora retorna vazio
121 @sample
122 */
123 //-----
124 Function STBRNumPrinter()
125
126 Local cNumDoc      := ""          // Retorna o Ultimo Cupom da impressora, caso use
127 Local nRet         := 0           // Retorno da impressora
128 Local aPrinter     := {}         // Retorno da impressora
129 Local aDados       := {Space(STBLenReceipt()),Nil}
130
131 aPrinter := STFFireEvent(ProcName(0), "STGetReceipt", aDados)
132
133 If Len(aPrinter) == 0 .OR. aPrinter[1] <> 0 .OR. Len(aDados) == 0
134     cNumDoc := "ERROR"
135     STFMessage(ProcName(),"STOP","Erro com a Impressora Fiscal. Operação não efetuada.")
136     STFShowMessage(ProcName())
137 Else
138     cNumDoc := PADR(aDados[1],TamSX3("L1_DOC")[1])
139 EndIf
140
141 Return cNumDoc
142
143
144 //-----
145 /*{Protheus.doc} STBRSCloseCupCancel
146 Recuperação de venda com o Cupom fechado
147
148 @param  aSale            Venda com Cabeçalho, Itens e Pagamentos (SL1,SL2,SL4)
149 @return lRet
150 @obs
151 @sample
152 */
153 //-----
154 Function STBRSCloseCupCancel( aSale, lNFCe )
155
156 Local lRet           := .T.        // Retorna se pode continuar a recuperar a venda

```

```

157 Local aSL1      := {}          // SL1
158 Local aSL2      := {}          // SL2
159 Local aSL4      := {}          // SL4
160 Local cNumCup   := ""          // Armazena o numero do Cupom da impressora
161 Local aRet       := {}          // Armazena retorno da impressora
162 Local nI        := 0           // Contador
163 Local aProfile   := {}          // Armazena o retorno da permissao
164 Local lError     := .F.         // Indica que houve erro
165 Local aIsOpenReceipt:= { "5" , "" } // Armazena retorno se o cupom está aberto
166
167 Default aSale := {}
168 Default lNFCe := STBGetNFCe()
169
170 ParamType 0 Var   aSale  As Array   Default   {}
171
172 aSL1 := aSale[1]
173 aSL2 := aSale[2]
174 aSL4 := aSale[3]
175
176 aRet := STFFireEvent( ProcName(0),"STPrinterStatus",aIsOpenReceipt) // Verifica Cupom Fechado
177
178 If !lNFCe .AND. Len(aRet) > 0 .AND. ValType(aRet[1]) == "N" .AND. aRet[1] <> 7
179
180     STFMessage("STRecoverySale","STOP","0 ECF não está com o cupom aberto. Este cupom será cancelado.")
181     STFShowMessage("STRecoverySale") // Mostrar a mensagem antes q cancele(ou peça permissao se preciso for)
182
183     LjGrvLog( "L1_NUM: "+aSL1[AScan( aSL1 , { |x| x[1] == "L1_NUM" } )][2], "0 ECF não está com o cupom aberto. Este cupom será cancelado." ) //Gera LOG
184
185     cNumCup := STBRSNumPrinter()
186
187     If (cNumCup <> "ERROR") .AND. !(Empty(cNumCup))
188
189         If AllTrim(cNumCup) == AllTrim(aSL1[AScan( aSL1 , { |x| x[1] == "L1_DOC" } )][2])
190             aProfile := STFPROFILE(8) // Permissao cancelamento
191             aRet := STFFireEvent( ProcName(0) , "STCancelReceipt" , {aProfile[2]} )
192             If Len(aRet) > 0 .AND. aRet[1] == 0
193                 STDCSDeleteSale( aSL1[AScan( aSL1 , { |x| x[1] == "L1_NUM" } )][2] )
194             Else
195                 lError := .T.
196             EndIf
197         Else
198             lError := .T. // Se a numeracao eh diferente, nao consegue cancelar, (deve ter aberto comprovante TEF)
199             lRet := .F.    // Retorna que nao continua a recuperar a venda
200         EndIf
201     Else
202         lError := .T.
203         lRet := .F.    // Retorna que nao continua a recuperar a venda
204     EndIf
205
206 If lError
207
208

```

```

209     STFMessage("STRecoverySale","STOP","Erro com a Impressora Fiscal. Será necessário efetuar Nota de Devolução.")
210     STFShowMessage("STRecoverySale")
211
212     LjGrvLog( "L1_NUM: "+aSL1[AScan( aSL1 , { |x| x[1] == "L1_NUM" } )][2], "Erro com a Impressora Fiscal. Será necessário efetuar Nota de Devolução." ) //Gera LOG
213
214     /*/
215     Caso ocorra erro no cancelamento , continua a gravacao do cupom, pois o mesmo nao
216     pode ser cancelado no ECF. Neste caso sera necessario realizar uma NOTA DE DEVOLUCAO.
217     /*/
218
219     aSL1[AScan( aSL1 , { |x| x[1] == "L1_SITUA" } )][2] := "00" // "00" - Venda Efetuada Com Sucesso
220     aSL1[AScan( aSL1 , { |x| x[1] == "L1_STATUS" } )][2] := "T" // Indica que houve erro na Transacao TEF
221     aSL1[AScan( aSL1 , { |x| x[1] == "L1_VENDTEF" } )][2] := "N"
222     aSL1[AScan( aSL1 , { |x| x[1] == "L1_DATATEF" } )][2] := ""
223     aSL1[AScan( aSL1 , { |x| x[1] == "L1_HORATEF" } )][2] := ""
224     aSL1[AScan( aSL1 , { |x| x[1] == "L1_DOCTEF" } )][2] := ""
225     aSL1[AScan( aSL1 , { |x| x[1] == "L1_AUTORIZ" } )][2] := ""
226     aSL1[AScan( aSL1 , { |x| x[1] == "L1_INSTITU" } )][2] := ""
227     aSL1[AScan( aSL1 , { |x| x[1] == "L1_DOCCANC" } )][2] := ""
228     aSL1[AScan( aSL1 , { |x| x[1] == "L1_DATCANC" } )][2] := ""
229     aSL1[AScan( aSL1 , { |x| x[1] == "L1_HORCANC" } )][2] := ""
230     aSL1[AScan( aSL1 , { |x| x[1] == "L1_NSUTEF" } )][2] := ""
231     aSL1[AScan( aSL1 , { |x| x[1] == "L1_TIPCART" } )][2] := ""
232     If SL1->(FieldPos("L1_TIMEATE")) > 0
233         aSL1[AScan( aSL1 , { |x| x[1] == "L1_TIMEATE" } )][2] := 0
234     EndIf
235     /*/
236     Realiza a persistencia dos dados
237     /*/
238     STFSaveTab( "SL1" , aSL1 , .F. )
239
240
241     EndIf
242
243 EndIf
244
245 Return lRet
246
247
248 //-----
249 /*{Protheus.doc} STBRSCancelSale
250 Realiza o cancelamento da venda caso, durante a recuperacao da venda, seja identificado que a venda tem itens de reserva ou;
251 é um orçamento importado.
252
253 @param aSale Venda com Cabeçalho, Itens e Pagamentos (SL1,SL2,SL4)
254 @return lRet Realizou cancelamento
255 @obs
256 @sample
257 /*/
258 //-----
259 Function STBRSCancelSale( aSale, lHasPrint )
260

```

```

261 Local lRet          := .T.          // Fluxo da funcao
262 Local aRet          := {}           // Armazena retorno da impressora
263 Local lCancCup      := .T.          // Indica se cancelou cupom fiscal
264 Local aSL1          := {}           // Cabeçalho da venda
265 Local aProfile      := {}           // Armazena Retorno de permissao
266 Local cGCCli        := ""           //
267 Local cNome          := ""           //
268 Local cNumOrc        := ""           //
269 Local cNumOrig       := ""           //
270
271 Default aSale        := {}
272
273 ParamType 0 Var      aSale           As Array      Default {}
274
275 If STFGetCfg("lUseECF")
276   aRet := STFFireEvent( ProcName(0)      , ; // Nome do processo
277                         "STPrinterStatus" , ; // Nome do evento
278                         { "9", "" }      ) // Verifica STATUS
279
280   If Len(aRet) > 0 .AND. ValType(aRet[1]) == "N" .AND. aRet[1] <> 0
281     HELP(' ',1,'FRT011') // "Erro com a Impressora Fiscal. Operação não efetuada.", "Atenção"
282     lRet := .F.
283   EndIf
284
285   If lRet
286     aSL1 := {"L1_SITUA", "07"} // "07" - Solicitado o Cancelamento do Cupom
287
288     // Realiza a persistência
289     STFSaveTab( "SL1" , aSL1 , .F. )
290
291     If lHasPrint
292       aProfile := STFFPROFILE(8) // Permissao cancelamento
293       aRet := STFFireEvent( ProcName(0) , "STCancelReceipt" , {aProfile[2]} )
294       If Len(aRet) == 0 .OR. aRet[1] <> 0
295         lCancCup := .F.
296       EndIf
297     EndIf
298
299     If lCancCup
300       STDCSDeleteSale( aSale[1][AScan( aSale[1] , { |x| x[1] == "L1_NUM" } )][2] )
301       STFMessage(ProcName(),"ALERT", "CUPOM FISCAL CANCELADO") //"CUPOM FISCAL CANCELADO"
302       STFShowMessage(ProcName())
303
304       If STBIsPaf()
305         cGCCli := aSale[1][AScan( aSale[1] , { |x| x[1] == "L1_CGCCLI" } )][2]
306         STFFireEvent(ProcName(0), "STGrvMdz", {"R4",aSale[1][AScan( aSale[1] , { |x| x[1] == "L1_NUM" } )][2], cNome, cGCCli}) //"Suprimento"
307       EndIf
308     EndIf
309   EndIf
310 EndIf
311
312

```

```

313 Else
314
315     cNumOrc := aSale[1][AScan( aSale[1] , { |x| x[1] == "L1_NUM" } )][2]
316     cNumOrig := aSale[1][AScan( aSale[1] , { |x| x[1] == "L1_NUMORIG" } )][2]
317     aSL1 := {"L1_SITUA", "07"} // "07" - Solicitado o Cancelamento do Cupom
318
319     // Realiza a persistência
320     STFSaveTab( "SL1" , aSL1 , .F. )
321
322     STDCSDeleteSale(cNumOrc ,Nil ,cNumOrig)
323
324     STFMessage(ProcName(),"ALERT", "CUPOM CANCELADO") //"CUPOM CANCELADO"
325     STFShowMessage(ProcName())
326
327 EndIf
328
329 Return
330
331 //-----
332 /*{Protheus.doc} STBRSZReduction
333 Realiza recuperacao da venda
334
335 @param   aSale           Venda com Cabeçalho, Itens e Pagamentos (SL1,SL2,SL4)
336 @return  lRet            Retorna se Realizou a Recuperação da venda
337 @obs
338 @sample
339 */
340 //-----
341 Function STBRSZReduction( )
342
343 Return
344
345 //-----
346 /*{Protheus.doc} STBIsRecovered
347 Informa se a venda esta sendo recuperada
348
349 @return  lRecovered      Informa se a venda esta sendo recuperada
350 @obs
351 @sample
352 */
353 //-----
354 Function STBIsRecovered()
355 Return lRecovered
356
357 //-----
358 /*{Protheus.doc} STBR7Recovered
359 Reseta variavel static que informa se a venda esta sendo recuperada
360
361 @obs
362 @sample
363 */
364 //-----

```

```
365 Function STB7Recovered(lSet)
366
367 DEFAULT lSet := .F.
368
369 lRecovered := lSet
370
371 Return
```

Componentes/RecoverySale/STDRecoverySale.prw

386 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "FWMVCDEF.CH"
3 #INCLUDE "STDRecoverySale.ch"
4
5 Static cSiglaSat      := IIF( ExistFunc("LjSiglaSat"),LjSiglaSat(), "SAT" )    //Retorna sigla do equipamento que esta sendo utilizado
6
7 Function STDRecoverySale()
8
9 Return
10
11 //-----
12 /*{Protheus.doc} STDRSGetSale
13 Busca venda a ser Recuperada
14
15 @param
16 @return aSale[1]          Retorna Cabeçalho da Venda (SL1)
17 @return aSale[2]          Retorna Itens da Venda (SL2)
18 @return aSale[3]          Retorna Formas de Pagamento da venda (SL4)
19 @obs
20 @sample
21 */
22 //-----
23 Function STDRSGetSale()
24
25 Local aArea          := GetArea()          // Guarda alias corrente
26 Local aSale          := {}                // Retorno da funcao
27 Local aSL1           := {}                // SL1
28 Local aSL2           := {}                // SL2
29 Local aItens         := {}                // Itens
30 Local aSL4           := {}                // SL4
31 Local aPay           := {}                // Pagamentos
32 Local nI             := 0                 // Contador
33 Local nItem          := 0                 // Variavel para Contorle de itens (Numerico)
34 Local cItem          := ""                // Variavel para Contorle de itens (Caracter)
35 Local lPDVOnline:= ExistFunc("STFGetOnPdv") .AND. STFGetOnPdv() // Variável para definir se o totvs PDV está no modo online
36
37 // Detectar Registros Nao Finalizados
38 DbSelectArea("SL1")
39
40 IF lPDVOnline
41     SL1->(DbSetOrder(8))// L1_FILIAL+L1_PDV+L1_DOC
42     DbSeek(xFilial("SL1")+PadR(STFGetStation("PDV"),TamSX3("LG_PDV")[1])+Replicate("Z",TamSX3('L1_DOC')[1]),.T.)
43     SL1->(DbSkip(-1))
44 Else
45     SL1->(DbSetOrder(1))// L1_FILIAL+L1_NUM
46     SL1->(Dbgoto(SL1->(LASTREC())))
47     While !SL1->(BOF()) .AND. !(xFilial("SL1") == SL1->L1_FILIAL)
48         SL1->(dbSkip(-1))
49     EndDo
50 Endif
51
52 If !SL1->(BOF()) .AND. !SL1->(EOF()) .AND. SL1->L1_SITUA >= "01" .AND. SL1->L1_SITUA <= "99" .AND. SL1->L1_SITUA <> '07'

```

```

53 // aSL1 := DbStruct()
54 For nI := 1 To fCount()
55     AADD( aSL1 , { SL1->(FieldName(nI)) , &(SL1->(FieldName(nI))) } )
56 Next nI
57
58 DbSelectArea("SL2")
59 DbSetOrder(1)//L2_FILIAL+L2_NUM+L2_ITEM+L2_PRODUTO
60 If DbSeek(xFilial("SL2")+SL1->L1_NUM)
61     While !EOF() .AND. (xFilial("SL2") == SL1->L1_FILIAL) .AND. (SL2->L2_NUM == SL1->L1_NUM)
62
63         // -- Reorganizo Item da tabela SL2
64
65         nItem ++
66         cItem := AllTrim(Str(nItem))
67
68         If Len(cItem) == 1
69             cItem := '0' + cItem
70         EndIf
71
72         If nItem > 99
73             cItem := STBPegaIT(nItem)
74         EndIf
75
76         If !( cItem == SL2->L2_ITEM )
77             RecLock("SL2",.F.)
78             SL2->L2_ITEM := cItem
79             SL2->(MsUnLock())
80         EndIf
81
82         For nI := 1 To fCount()
83             AADD( aItens , { SL2->(FieldName(nI)) , &(SL2->(FieldName(nI))) } )
84         Next nI
85
86         AADD( aSL2 , aItens )
87         aItens := {}
88         SL2->(DbSkip())
89     EndDo
90
91 EndIf
92
93 DbSelectArea("SL4")
94 DbSetOrder(1)//L4_FILIAL+L4_NUM+L4_ORIGEM
95 If DbSeek(xFilial("SL4")+SL1->L1_NUM)
96     While !EOF() .AND. (xFilial("SL4") == SL1->L1_FILIAL) .AND. (SL4->L4_NUM == SL1->L1_NUM)
97         For nI := 1 To fCount()
98             AADD( aPay , { SL4->(FieldName(nI)) , &(SL4->(FieldName(nI))) } )
99         Next nI
100         AADD( aSL4 , aPay )
101         aPay := {}
102         SL4->(DbSkip())
103     EndDo
104 EndIf

```

```

105
106 EndIf
107
108 If !Empty(aSL1)
109     AADD(aSale , aSL1)
110     AADD(aSale , aSL2)
111     AADD(aSale , aSL4)
112 ElseIf !Empty(SLI->LI_DOC) .And. SLI->LI_SITUA == "T3" .And. RecLock("SL1",.F.)
113     MsgAlert(STR0005 + AllTrim(SLI->LI_DOC) + STR0007) // "0 Cupom : " # ", Já foi finalizado."
114     SLI->LI_SITUA := "00"
115     SLI->(MsUnLock())
116 EndIf
117
118 RestArea(aArea)
119
120 Return aSale
121
122 //-----
123 /*{Protheus.doc} STDRSSameCash
124 Verifica se o caixa logado eh o mesmo caixa aberto
125
126 @param
127 @return lRet          Retorna se o caixa logado eh o mesmo caixa aberto
128 @obs
129 @sample
130 */
131 //-----
132 Function STDRSSameCash()
133
134 Local lRet          := .T.          // Retorno funcao
135 Local aArea         := GetArea()    // Armazena area corretne
136 Local aAreaSLI      := SLI->(GetArea()) // Armazena area SLI
137 Local cEstacao      := AllTrim(STFGetStat("CODIGO")) // Estacao
138
139 DbSelectArea("SLI")
140 DbSetOrder(1) // LI_FILIAL + LI_ESTACAO + LI_TIPO
141 If SLI->(DbSeek(xFilial("SLI") + PadR(cEstacao,TamSX3("LI_ESTACAO")[1]) + "OPE"))
142     If !(Empty(SLI->LI_MSG))
143         If (AllTrim(SLI->LI_USUARIO) <> AllTrim(cUserName))
144             lRet := .F.
145             STFMessage("STRecoverySale","STOP","Existe um cupom do caixa: " + (AllTrim(SLI->LI_USUARIO)) + " " + ; //"Existe um cupom do caixa: "
146             "para ser recuperado. Acesse o sistema com esse caixa para concluir a operacao") //"para ser recuperado. Acesse o sistema com esse caixa para concluir a operacao"
147             STFShowMessage("STRecoverySale")
148         EndIf
149     EndIf
150 EndIf
151
152 RestArea(aAreaSLI)
153 RestArea(aArea)
154
155 Return lRet
156

```

```

157 //-----
158 /*{Protheus.doc} STDSatRecovery
159 @description  Função responsável por realizar toda a recuperação
160               da venda SAT no TOTVSPDV
161
162 @return       lRet -> Sempre verdadeiro (já preparado para futuras
163               implementações)
164 */
165 //-----
166 Function STDSatRecovery(lFinCnc)
167 Local lRet      := .T.
168 Local aAreaSL1  := {}
169 Local aAreaSL2  := {}
170 Local aAreaSL4  := {}
171 Local aCpAreaSL1:= {}
172 Local aCpAreaSL2:= {}
173 Local aRetSitC  := {}
174 Local aValidCancel:= {}
175 Local aAux      := Array(2)
176 Local cNFisCanc := ""
177 Local cMsg      := ""
178
179 Default lFinCnc := .F.
180
181 If ExistFunc("SATValidCanc")
182   If !lFinCnc .And. SATValidCanc()
183
184     LjGrvLog( "SAT", "Duplicando a venda para cancelamento no SAT e processamento do livro fiscal referente a venda cancelada.")
185     LjGrvLog( "SAT", "Venda original: " + SL1->L1_NUM)
186
187     //Guarda posição do orçamento original
188     aAreaSL1  := SL1->(GetArea())
189     aAreaSL2  := SL2->(GetArea())
190     aAreaSL4  := SL4->(GetArea())
191
192     STDDblOrc() //copia orçamento
193     LjGrvLog( "SAT", "Venda copia numero: " + SL1->L1_NUM)
194
195     //Guarda posição do orçamento novo (cópia)
196     aCpAreaSL1 := SL1->(GetArea())
197     aCpAreaSL2 := SL2->(GetArea())
198
199     //Retorna posição do orçamento original
200     RestArea(aAreaSL4)
201     RestArea(aAreaSL2)
202     RestArea(aAreaSL1)
203
204     LjGrvLog( "SAT", "Cancelando a venda original: " + SL1->L1_NUM)
205     LJSatUltimo(.T.)
206
207     //Retorna posição do orçamento novo (cópia)
208     RestArea(aCpAreaSL2)

```

```

209      RestArea(aCpAreaSL1)
210      LjGrvLog( "SAT", "posicionando na nova venda (copia) para finalização correta: " + SL1->L1_NUM)
211      ElseIf lFinCnc .And. ExistFunc("LjSaCtrCnc")
212
213          //Cn=Transacao de cancelamento ficou pendente em algum ponto
214          aRetSitC:= LjSaTraCtr(LjSaCtrCnc(.F.,.F.,.T.,.F.,""))
215          LjGrvLog( "SAT", "Finalizando cancelamento que foi interrompido")
216
217          //Guarda posição do orçamento original
218          aAreaSL1 := SL1->(GetArea())
219          aAreaSL2 := SL2->(GetArea())
220          aAreaSL4 := SL4->(GetArea())
221
222          SL1->( DbSetOrder(2) ) //L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
223          If SL1->( DbSeek(aRetSitC[2] + STFGetStation("SERIE") + aRetSitC[4] + aRetSitC[3]) )
224              If LjSatFinCnc(.F.)
225                  //" Venda referente ao orçamento #" + " cancelada no Equipamento de SAT está pendente de cancelamento no Protheus " + " Será realizado o cancelamento dessa
venda!"
226                  MsgAlert(STR0001 + AllTrim(SL1->L1_NUM) + StrTran(STR0002, "SAT", cSiglaSat) + CHR(13) + STR0003)
227
228                  //Gera cancelamento SAT
229                  lRet := LJSatxCanc(.F.,@cNFisCanc)
230
231                  If lRet
232                      aValidCancel := STBCSCanCancel(StrZero(Val(aRetSitC[4]),6))
233                      If aValidCancel[1]
234                          STISetCancel( aValidCancel )
235                          aAux[1] := ""
236                          aAux[2] := ""
237                          lRet := STWCsFinalized(aValidCancel[3] , aValidCancel[4] , aValidCancel[5], @aAux[1], @aAux[2], cNFisCanc )
238
239                          If lRet
240                              LjSatAjTab(.F., .F.,.T.,cNFisCanc,"",NIL)
241                          EndIf
242                      EndIf
243                      LjSatFinCnc(.T.) //apaga a sessão
244                  Else
245                      cMsg := cSiglaSat + " - Venda já cancelada ou excedeu o período de 30 minutos." //"SAT - Venda já cancelada ou excedeu o período de 30 minutos."
246                      LjSaCtrCnc(.F.,.T.,.F.,.F.,"") //Apaga o arquivo de sinal de cancelamento
247                  EndIf
248                  Else
249                      cMsg := cSiglaSat + " - Recuperação de cancelamento não encontrado na sessão "
250                  EndIf
251                  Else
252                      lRet := .F.
253                      cMsg := cSiglaSat + " - Venda não encontrada." //"SAT - Venda não encontrada."
254                  EndIf
255
256          //Retorna posição do orçamento original
257          RestArea(aAreaSL4)
258          RestArea(aAreaSL2)
259          RestArea(aAreaSL1)

```

```

260
261     If lRet
262         LjGrvLog( "SAT", "Cancelamento Finalizado")
263
264         If !Empty(cMsg)
265             LjGrvLog("SAT",cMsg)
266         EndIf
267         LjSaCtrCnc(.F.,.T.,.F.,.F.,"") //Apaga o arquivo de sinal de cancelamento
268     EndIf
269 Else
270     //Exclui a SL4 da venda, pois será gerado ao ser finalizada.
271     STDRecovSL4()
272 EndIf
273 Else
274     LjGrvLog( "SAT", "AVISO: Atualizar fonte LOJSAT.PRW para a correta recuperação da venda.")
275     LJSatUltimo(.T.)
276 EndIf
277
278 Return lRet
279
280
281 //-----
282 /*{Protheus.doc} STDDblOrc
283 @description      Duplica Orcamento
284 @obs              Não foi duplicado o SL4 pois no momento da finalização
285                  ja são gerados os registros conforme selecionado na venda
286 */
287 //-----
288 Function STDDblOrc()
289
290 Local aStruSL1 := {} //Estrutura inteira do dicionario da SL1
291 Local aDadosSL1 := {} //Conteudo dos campos da SL1
292 Local aStruSL2 := {} //Estrutura inteira do dicionario da SL2
293 Local aDadosSL2 := {} //Conteudo dos campos da SL2
294 Local nX := 0 //contador
295 Local cNewNum := "" //numero do orçamento
296 Local nPos := 0 //posicao para recuperar valor de uma campo da busca do aScan
297 Local nLinha := 0 //numero de registros adicionados para cada estrutura da SL2 , SL4
298 Local nSaveSx8 := GetSx8Len() // Numeracao do SX8
299 Local nRegNumOrig := SL1->(RECNO()) //numero do orçamento original a ser copiado
300 Local cMay := ""
301 Local nTent := 0
302
303 SL1->(DBSetOrder(1))//L1_FILIAL + L1_NUM
304
305 cNewNum := CriaVar("L1_NUM") //Nova numeração
306
307 // Caso o SXE e o SXF estejam corrompidos cNumOrc estava se repetindo.
308 cMay := Alltrim(xFilial("SL1"))+cNewNum
309 FreeUsedCode() //libera codigos de correlativos reservados pela MayIUseCode()
310
311 // Se dois orcamentos iniciam ao mesmo tempo a MayIUseCode impede que ambos utilizem o mesmo numero.

```

```

312 While SL1->(DbSeek(xFilial("SL1")+cNewNum)) .OR. !MayIUseCode(cMay)
313   If ++nTent > 20
314     Final(STR0004) //"Impossível gerar numero sequencial de orcamento correto."
315   Endif
316   While (GetSX8Len() > nSaveSX8)
317     ConfirmSX8()
318   End
319   cNewNum := CriaVar("L1_NUM")
320   FreeUsedCode()
321   cMay := Alltrim(xFilial("SL1")+cNewNum
322 End
323 While (GetSX8Len() > nSaveSX8)
324   ConfirmSX8()
325 End
326
327 //Reposiciona na venda original
328 SL1->(DBG0T0(nRegNumOrig))
329
330 aStruSL1:= SL1->(dbStruct())
331 aStruSL2:= SL2->(dbStruct())
332
333 //****SL1****
334 For nX := 1 To Len(aStruSL1)
335   aAdd(aDadosSL1, { aStruSL1[nX][1], &("SL1->" + aStruSL1[nX][1]) } )
336 Next nX
337 nPos := aScan(aDadosSL1, {|x| x[1] == "L1_NUM"} )
338 aDadosSL1[nPos][2] := cNewNum
339
340 //****SL2****
341 SL2->(DbSetOrder(1)) // L2_FILIAL + L2_NUM + L2_ITEM + L2_PRODUTO
342 SL2->(DbSeek(xFilial("SL2") + SL1->L1_NUM))
343 While !SL2->(EOF()) .And. xFilial("SL2") + SL1->L1_NUM == SL2->L2_FILIAL + SL2->L2_NUM
344   aAdd(aDadosSL2, { })
345   nLinha := Len(aDadosSL2)
346   For nX := 1 To Len(aStruSL2)
347     aAdd(aDadosSL2[nLinha], { aStruSL2[nX][1], &("SL2->" + aStruSL2[nX][1]) } )
348   Next nX
349
350   nPos := aScan(aDadosSL2[nLinha], {|x| x[1] == "L2_NUM"} )
351   aDadosSL2[nLinha][nPos][2] := cNewNum
352
353   SL2->(DbSkip())
354 EndDo
355
356 //Cria o orçamento novo
357 STFSaveTab("SL1",aDadosSL1,.T.)
358
359 For nX := 1 To Len(aDadosSL2)
360   STFSaveTab("SL2",aDadosSL2[nX],.T.)
361 Next nX
362
363 SL1->(DBSetOrder(1)) //L1_FILIAL + L1_NUM

```

```
364 SL1->(DBSeek(xFilial("SL1")+cNewNum))
365 SL2->(DBSetOrder(2)) //L2_FILIAL + L2_NUM
366 SL2->(DBSeek(xFilial("SL2")+cNewNum))
367
368 Return Nil
369
370 //-----
371 /*{Protheus.doc} STDRecovSL4
372 @description      Limpa o SL4 gravado para venda recuperada
373 */
374 //-----
375 Static Function STDRecovSL4()
376 Local cSL4Fil := xFilial("SL4")
377
378 SL4->(DbSetOrder(1)) // L4_FILIAL + L4_NUM
379 SL4->(DbSeek( cSL4Fil + SL1->L1_NUM))
380 While !SL4->(EOF()) .And. SL4->(L4_FILIAL + L4_NUM) == cSL4Fil + SL1->L1_NUM
381     If SL4->(Reclock("SL4", .F.))
382         SL4->(DBDelete())
383     EndIf
384     SL4->(DBSkip())
385 End
386 Return Nil
```

Componentes/RecoverySale/STWRecoverySale.prw

345 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STWRECOVERYSALE.CH"
4
5 Static lVendaRecup := .F.
6
7 //-----
8 /*{Protheus.doc} STWRecoverySale
9 Recuperação de Venda
10
11 @param lEmptySale, lógico
12 @return lRecovery          Retorna se a venda foi recuperada
13 @obs
14 @sample
15 */
16 //-----
17 Function STWRecoverySale( lEmptySale )
18 Local lRecovery      := .T.          // Retorna se recuperou a venda
19 Local lImported      := .F.
20 Local lReserve       := .F.
21 Local aHasPrint      := {}           // Armazena 1 - possui itens de reserva , 2 - possui itens de impressao
22 Local aSale          := {}           // Armazena venda a ser recuperada
23 Local aSL1           := {}           // Armazena cabecalho do aSale
24 Local aSL2           := {}           // Armazena Itens do aSale
25 Local aSL4           := {}           // Armazena Pagamentos do aSale
26 Local lNFCe          := LjEmitNFCe() // Sinaliza se utiliza NFC-e
27 Local lUseSat        := STFCfg("lUseSAT",.F.) //verifica se utiliza SAT
28 Local lInutNFCE      := .F.
29 Local lCancelSale    := .F.
30
31 DEFAULT lEmptySale := .F.
32
33 /*/
34     Busca venda a ser recuperada
35 /*/
36
37 LjGrvLog( Nil, "Inicio - Processo de Recuperação de Venda." ) //Gera LOG
38 LjGrvLog( Nil, "Verifica se existe venda a ser recuperada." )
39
40 aSale := STDRSGetSale()
41
42 Do Case
43     Case Len(aSale) <= 0
44         lRecovery := .F.
45         lEmptySale := .T.
46     Case Len(aSale[2]) <= 0 // Nenhum item adicionado
47         lRecovery := .F.
48     Otherwise
49         aSL1 := aSale[1]
50         aSL2 := aSale[2]
51         aSL4 := aSale[3]
52         LjGrvLog( Nil, "Existe venda à ser recuperada. L1_NUM = " + aSale[1][AScan( aSale[1], { |x| x[1] == "L1_NUM" } )][2] ) //Gera LOG

```

```

53 EndCase
54
55 /*/
56 Verifica se pode recuperar Venda
57 /*/
58 If lRecovery
59     lRecovery := STBRSCanRecovery()
60 EndIf
61
62 If !lRecovery .AND. !lEmptySale .AND. Len(aSale[1]) > 0
63     LjGrvLog( "L1_NUM: " + aSale[1][AScan( aSale[1], { |x| x[1] == "L1_NUM" } )][2], "Realiza a deleção do SL1, SL2 e SL4 e SE5" ) //Gera LOG
64     STDCSDeleteSale( aSale[1][AScan( aSale[1], { |x| x[1] == "L1_NUM" } )][2] )
65 EndIf
66
67 /*/
68 Valida se eh o mesmo operador, senão fechar o POS
69 /*/
70 If lRecovery
71
72     If !( STDRSSameCash() )
73         lRecovery := .F.
74     EndIf
75 EndIf
76
77 /*/
78 Detectar Troca de ECF
79 /*/
80 If lRecovery
81     cPDV := aSL1[AScan( aSL1, { |x| x[1] == "L1_PDV" } )][2]
82     If STBRSSStationChange( cPDV )
83
84         STDCSDeleteSale( aSL1[AScan( aSL1, { |x| x[1] == "L1_NUM" } )][2] )
85         lRecovery := .F.
86         STFMessage(ProcName(),"STOP","Os itens foram impressos em outro ECF. Este cupom será cancelado.")
87         STFShowMessage(ProcName())
88         LjGrvLog( "L1_NUM: " + aSL1[AScan( aSL1, { |x| x[1] == "L1_NUM" } )][2], "Os itens foram impressos em outro ECF. Este cupom será cancelado." ) //Gera LOG
89     EndIf
90 EndIf
91
92 /*/
93 Detectar Cupom Aberto
94 /*/
95 If lRecovery
96     aHasPrint := STBRSHasPrint( aSL2 )
97
98     If aHasPrint[2] // Tem Impressao
99         // Se o cupom estiver fechado, faz tratamento e nao recupera a venda
100         lRecovery := STBRSCloseCupCancel( aSale )
101     EndIf
102 EndIf
103
104 /*/

```

```

105     Caso seja Importação de Orçamento
106  /*/
107  If lRecovery
108     lImported := !Empty(aSale[1][AScan( aSale[1] , { |x| x[1] == "L1_NUMORIG" } )][2])
109     lReserve  := aHasPrint[1]
110
111     lCancelSale := IsInCallStack("STFPOSAUTO")
112
113     If lReserve .OR. lImported .OR. lCancelSale // Se houver reserva ou o orçamento seja importado, o cupom deve ser cancelado. A venda, portanto, não sera recuperada
114         lRecovery := .F.
115         If lReserve
116             STFMessage(ProcName(),"STOP","O Sistema irá finalizar o Cupom, pois existe item de reserva.") //"O Sistema irá finalizar o Cupom, pois existe item de reserva."
117             LjGrvLog( "L1_NUM: "+aSale[1][AScan( aSale[1], { |x| x[1] == "L1_NUM" } )][2], "O Sistema irá finalizar o Cupom, pois existe item de reserva." ) //Gera LOG
118         ElseIf lImported
119             STFMessage(ProcName(),"STOP","O Sistema irá finalizar o Cupom. O orçamento deve ser importado novamente.") //"O Sistema irá finalizar o Cupom, O orçamento deve
120             ser importado novamente"
121             LjGrvLog( "L1_NUM: "+aSale[1][AScan( aSale[1], { |x| x[1] == "L1_NUM" } )][2], "O Sistema irá finalizar o Cupom. O orçamento deve ser importado novamente." )
122             //Gera LOG
123         EndIf
124         STFShowMessage(ProcName())
125
126         LjGrvLog( "L1_NUM: "+aSale[1][AScan( aSale[1], { |x| x[1] == "L1_NUM" } )][2], "Realiza o cancelamento da venda." ) //Gera LOG
127         STBRSCancelSale( aSale, aHasPrint[2] )
128     EndIf
129 EndIf
130 /*/
131 Realizar recuperação
132 /*/
133 If lRecovery
134     LjGrvLog( "L1_NUM: "+aSale[1][AScan( aSale[1], { |x| x[1] == "L1_NUM" } )][2], "Recuperando da venda..." ) //Gera LOG
135     STB7Recovered(.T.)
136     STWRSVerRe( .F. , .T. )
137     If !lNFCE
138         STWCancelSale( .T. , /*lIsProgressSale*/ , /*cSuperior*/ , /*cDoc*/ , ;
139             /*cNumSale*/ , /*lCancVenc*/ , /*cNFisCanc */ , aSale[1][AScan( aSale[1], { |x| x[1] == "L1_NUM" } )][2] )
140     ElseIf !lUseSat
141         lInutNFCE := STBChkInut()
142     EndIf
143     lRecovery := STWRSRecovery( aSale, lInutNFCE ) //Depois recupera a venda, carregando-a como uma nova venda
144     STWRSVerRe( .T. )
145 EndIf
146
147 LjGrvLog( Nil, "Fim - Processo de Recuperação de Venda." ) //Gera LOG
148
149 Return lRecovery
150
151 //-----
152 /*{Protheus.doc} STWRSRecovery
153 Realiza recuperação da venda
154

```

```

155 @param  aSale                Venda com Cabeçalho, Itens e Pagamentos (SL1,SL2,SL4)
156 @return  lRet                Retorna se Realizou a Recuperação da venda
157 @obs
158 @sample
159 /*/
160 //-----
161 Function STWRSRecovery( aSale, lInutNFCe )
162 Local lRet                := .T.                // Retorna se Recuperou a venda
163 Local aSL1                := {}                // Cabeçalho da venda
164 Local aSL2                := {}                // Itens da venda
165 Local aSL4                := {}                // Formas de pagamento
166 Local aPrinter            := {}                // Armazena Retorno da Impressora
167 Local aDados              := {}
168 Local nI                  := 0                // Contador
169 Local nPosL1Situa         := 0                // Posicao do SL1_SITUA no array
170 Local cTotItens           := ""                // Total de itens da Impressora
171 Local cTotal              := ""                // Armazena o valor Total da impressora
172 Local cCliCode            := ""                // Armazena codigo cliente
173 Local cCliStore           := ""                // Armazena Loja Cliente
174 Local cCliType            := ""                // Armazena Tipo do Cliente (A1_TIPO)
175 Local lNFCe               := STFGetStat("NFCe") // nfce
176 Local lUseSat              := LjUseSat()        // Usa Sat
177 Local cVendedor           := ""                // Vendedor da venda recuperada
178 Local nPosIDITREL         := 0                // Posição do campo L2_IDITREL
179 Local lPendSale           := .F.
180 Local cCliCPF              := ""                // L1_CGCCLI para SIGACRD
181 Local cCliCart             := ""                // L1_CGCCART para SIGACRD
182 Local nPosL1Num           := 0                // Posição do L1_Num na aSL1
183
184
185 Default aSale              := {}
186 Default lInutNFCe          := .F.
187
188 aSL1 := aSale[1]
189 aSL2 := aSale[2]
190 aSL4 := aSale[3]
191
192 nPosL1Situa := AScan( aSL1 , { |x| x[1] == "L1_SITUA" } )
193
194 If !lNFCe .AND. !lUseSat // Quando é PAF-ECF utiliza o L1_NUM novo já carregado no Basket
195     nPosL1Num := AScan( aSL1 , { |x| x[1] == "L1_NUM" } )
196 Endif
197
198 If lRet
199
200     /*/
201     Atualiza o SL1 na Cesta
202     /*/
203
204     For nI := 1 To Len(aSL1)
205         If nI <> nPosL1Num
206             STDSPBasket( "SL1" , aSL1[nI,1],aSL1[nI,2] )

```

```

207     Endif
208 Next nI
209
210 cCliCode      := aSL1[AScan( aSL1 , { |x| x[1] == "L1_CLIENTE" } )][2]      // Armazena codigo cliente
211 cCliStore     := aSL1[AScan( aSL1 , { |x| x[1] == "L1_LOJA" } )][2]         // Armazena Loja Cliente
212 cCliType      := aSL1[AScan( aSL1 , { |x| x[1] == "L1_TIPOCLI" } )][2]       // Armazena Tipo do Cliente (A1_TIP0)
213 cCliCPF       := aSL1[AScan( aSL1 , { |x| x[1] == "L1_CGCCLI" } )][2]       // CPF/CNPJ
214 cCliCart      := aSL1[AScan( aSL1 , { |x| x[1] == "L1_CGCCART" } )][2]      // Cartão SIGACRD
215
216 /*/
217     Seto o model do cliente
218 /*/
219 STWCustomerSelection(cCliCode+cCliStore)
220
221 If ExistFunc("STBSetCrdIdent")
222     STBSetCrdIdent( cCliCart, cCliCPF, cCliCode, cCliStore )           // Seto SIGACRD
223 EndIf
224
225 /*/
226     Seto o model do vendedor
227 /*/
228 cVendedor := aSL1[AScan( aSL1 , { |x| x[1] == "L1_VEND" } )][2]      // Armazena codigo vendedor
229 STWSalesmanSelection(cVendedor)
230
231 If lInutNFCE
232     If !Empty(STDGPBasket("SL1","L1_DOC")) .AND. STDGPBasket("SL1","L1_DOC") <> "NFCE" .AND.;
233         !Empty(STDGPBasket("SL1","L1_SERIE")) .AND. ;
234         (Empty(STDGPBasket("SL1","L1_KEYNFCE")) .OR. ;
235         (!Empty(STDGPBasket("SL1","L1_KEYNFCE")) .AND. STDGPBasket("SL1","L1_SITUA") == "65"))
236         lPendSale := STWInuNFCE(STDGPBasket("SL1","L1_DOC") , STDGPBasket("SL1","L1_SERIE"), STDGPBasket("SL1","L1_PDV"),;
237                               STDGPBasket("SL1","L1_OPERADO"), STDGPBasket("SL1","L1_ESTACAO"), "" )
238         If lPendSale
239             lPendSale := STWDelPay( STDGPBasket("SL1","L1_NUM") )
240         EndIf
241     Else
242         //Deleta os dados do pagamento
243         lPendSale := STWDelPay( STDGPBasket("SL1","L1_NUM") )
244     EndIf
245 EndIf
246 EndIf
247
248 /*/
249     Registra os Itens
250 /*/
251 nPosIDITREL := AScan( aSL2[1] , { |x| x[1] == "L2_IDITREL" } )
252 For nI := 1 To Len(aSL2)
253
254     //Seto vendedor da venda original - Multiplos vendedroes
255     If SuperGetMV("MV_LJPDVEN", ,.F.)
256         STDSPBasket("SL1","L1_VEND" , aSL2[nI][AScan( aSL2[nI] , { |x| x[1] == "L2_VEND" } )][2] )
257     EndIf
258

```

```

259 //Seta quantidade
260 STBSetQuant( aSL2[nI][AScan( aSL2[nI] , { |x| x[1] == "L2_QUANT" } )][2] )
261
262 // Registra Item
263 STWItemReg( nI , aSL2[nI][AScan( aSL2[1] , { |x| x[1] == "L2_PRODUTO" } )][2] , cCliCode , cCliStore , ;
264 STBGetCurrency(), aSL2[nI][AScan( aSL2[1] , { |x| x[1] == "L2_VALDESC" } )][2] , "V" , NIL , ;
265 NIL , cCliType,,,;
266 ,,,T,,,;
267 ,,,,;
268 ,,,,;
269 Iif(nPosIDITREL > 0,aSL2[nI][nPosIDITREL][2],""),aSL2[nI][AScan ( aSL2[1] , { |x| x[1] == "L2_VALEPRE" } )][2])
270
271 If aSL2[nI][AScan( aSL2[nI] , { |x| x[1] == "L2_SITUA" } )][2] == "05"
272 STWCancelProcess( Val(aSL2[nI][AScan( aSL2[nI] , { |x| x[1] == "L2_ITEM" } )][2]) )
273 EndIf
274 Next nI
275
276
277 If !lNFCe .And. !lUseSat
278 /*/
279 Verifica a impressora tem funções de total de itens e de subtotal da venda e faz a comparacao dos valores
280 /*/
281 // Verifica Status da Impressora - Evento STPrinterStatus
282 aPrinter := STFFireEvent(ProcName(0),"STPrinterStatus",{ "12" , "" })
283 If Len(aPrinter) > 0 .AND. ValType(aPrinter[1]) == "N" .AND. aPrinter[1] == 0 // Possui
284 aDados := {""}
285
286 /*/
287 Verifica o numero total de itens impressos - Evento STNumItemPrint
288 /*/
289
290 aPrinter := STFFireEvent(ProcName(0),"STNumItemPrint",aDados)
291 If Len(aPrinter) > 0 .AND. ValType(aPrinter[1]) == "N" .AND. aPrinter[1] == 0 .AND. Len(aDados) > 0 // Possui
292 cTotItens := aDados[1]
293 If Val(cTotItens) == Len(aSL2)
294 aDados := {"",""}
295
296 /*/
297 Verifica o subtotal da venda - Evento STSubTotalReceipt
298 /*/
299
300 aPrinter := STFFireEvent(ProcName(0),"STSubTotalReceipt",aDados)
301 If Len(aPrinter) > 0 .AND. ValType(aPrinter[1]) == "N" .AND. aPrinter[1] == 0 .AND. Len(aDados) > 0
302 cTotal := aDados[1]
303 If Val(cTotal)/100 <> aSL1[AScan( aSL1 , { |x| x[1] == "L1_VALMERC" } )][2] // Consistencia dos totais
304 STFMessage(ProcName(),"STOP","Existe diferenças entre o Cupom Fiscal e o Sistema. Por favor, cancele o Cupom Fiscal.")
305 STFShowMessage(ProcName())
306 lRet := .F.
307 EndIf
308 EndIf
309 Else
310 STFMessage(ProcName(),"STOP","Existe diferenças entre o Cupom Fiscal e o Sistema. Por favor, cancele o Cupom Fiscal.")

```

```

311             STFS showMessage(ProcName())
312             lRet := .F.
313         EndIf
314     EndIf
315 EndIf
316 EndIf
317 EndIf
318
319 SL1->(RollBackSx8())
320 STB7Recovered(lRet)
321
322 Return lRet
323
324 //-----
325 /*{Protheus.doc} STWRSVerRe
326 Retorna o conteúdo da variável estática lVendaRecup
327
328 @param lReset, lógico, reseta variavel ?
329 @return lVendaRecup, lógico, valida se esta numa recuperada
330 */
331 //-----
332 Function STWRSVerRe(lReset,lVal)
333
334 Default lReset := .F.
335 Default lVal := NIL
336
337 If lReset
338     lVendaRecup := .F.
339 EndIf
340
341 If ValType(lVal) == "L"
342     lVendaRecup := lVal
343 EndIf
344
345 Return lVendaRecup

```

Componentes/SalesOperations/STBSalesOperations.prw

489 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STBSALESOPERATIONS.CH"
4 #INCLUDE "AUTODEF.CH"
5
6 Static cSiglaSat      := LjSiglaSat()           //Retorna sigla do equipamento que esta sendo utilizado
7 Static lLjPDAccesso  := ExistFunc("LjPDAccesso")
8
9 //-----
10 /*{Protheus.doc} STBLstOperCashier
11 Lista operacoes de caixa disponiveis
12
13 As operacoes de caixa incluem, mas nao se limitam a:
14 Tarefas relativas ao inicio e encerramento de venda,
15 atendimento de venda e operacoes nao ligadas diretamente à venda.
16
17 @return  aRet - Lista de Funcoes
18
19 @obs
20 @sample
21 */
22 //-----
23 Function STBLstOperCashier()
24 Local aRet      := {}                          // Array de retorno das rotinas disponiveis
25 Local lMobile   := .F.                        // Smart Client Mobile
26
27 lMobile := STFGetCfg("lMobile", .F.)
28
29 If (STFProFile(12,,,,.T.)[1]) //"Acesso para acessar a tecla de funcoes"
30
31     //Em versoes Mobile os menus serão especificos
32     If ValType(lMobile) == "L" .AND. lMobile
33         aRet := STBMenuMobile()
34     Else
35         //Menu para versao Desktop(Padrao)
36         aRet := STBMenuDesktop()
37     EndIf
38
39 EndIf
40
41 Return aRet
42
43
44 //-----
45 /*{Protheus.doc} STBMenuDesktop
46 Retorna as funcoes de menu disponiveis na versao Desktop do PDV
47
48 @return  Ret - Lista de Funcoes
49
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Function STBMenuDesktop()
55
56 Local aRet          := {}                // Array de retorno das rotinas disponiveis
57 Local lUseECF       := STFGetCfg("lUseECF") // Usa ECF
58 Local lPafEcfcf     := STFGetCfg("lPafEcfcf") // PAF-ECF
59 Local lUsaTef       := STFGetCfg("lUsaTef") // Usa TEF
60 Local nOption       := 0                // Numero do atalho
61 Local lOnlyMenuFiscal := STFGetCfg("lOnlyMenuFiscal") // Mostrar apenas menu fiscal PAF-ECF
62 Local lSTMenu       := ExistBlock("STMenu") // Ponto de entrada para add novos itens no menu
63 Local lSTMenEdt     := ExistBlock("STMenEdt") // Ponto de entrada para edicao do Menu
64 Local aPEMenu       := {}                // Array que recebe os itens do ponto de entrada
65 Local nI            := 0                // Variavel de loop
66 Local lEmitNfce     := LjEmitNfce()      // Sinaliza se utiliza NFC-e
67 Local lUseSat       := STFGetCfg("lUseSAT",.F.) //utiliza SAT
68 Local cFuncReimp    := "STIPanReimpSale()" // Função padrao para reimpressao do SAT
69 Local cFuncCanc     := ""                // Funcao de cancelamento de venda
70 Local cMVLOJANF     := AllTrim( GetMV("MV_LOJANF") )
71 Local lMVFISNOTA    := GetMV("MV_FISNOTA") .and. !Empty(cMVLOJANF) .and. cMVLOJANF <> "UNI"
72 Local lIsPafNfce    := STFGetCfg("lPAFNfce")
73
74 If (STFProFile(12,,,,.T.)) [1] // "Acesso para acessar a tecla de funcoes"
75
76     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0034 , "STWCash()" , "", "01"} ) // "Encerramento de caixa"
77
78     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0036 , "STISupplyBleeding('1')", "", "02"} ) // "Sangria de caixa"
79
80     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0037 , "STISupplyBleeding('2')", "", "03"} ) // "Suprimento de caixa"
81
82     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0039 , "iIf(STBVldPD('CADCLI'), (IIF(ExistFunc('STIConsentTerm'),STIConsentTerm(),STIIncCustomer())), Nil )", "", "04"} )
// "Cadastro de Clientes"
83
84     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0045 , "STIValPre()" , "", "05"} ) // "Vale Presente"
85
86     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0047 , "STWInfoCNPJ(.T.)" , "", "06"} ) // "Informar CPF"
87
88     If lSTMenu
89         aPEMenu := ExecBlock("STMenu",.F.,.F.)
90         If Len(aPEMenu) > 0
91             For nI := 1 To Len(aPEMenu)
92                 AAdd( aRet , { AllTrim(STR(++nOption)) , aPEMenu[nI][1], aPEMenu[nI][2], "", ""})
93             Next nI
94         EndIf
95     EndIf
96
97     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0038 , "STISalesmanSelection()" , "", "07"} ) // "Alterar Vendedor"
98
99     cFuncCanc := "(" + IIF( !lEmitNfce, "STICancelSale()", "STIGerCancel()") + ",STIFocoReg()" + ")"
100 //Funcao de cancelamento de venda
101
102
103     If !lUseSat

```

```

104      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0006 , cFuncCanc, "", "08"} ) //"Cancelar Venda"
105
106      If lEmitNfce .And. !lMVFISNOTA
107      NFC-e"
108      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0077 , "Iif(ExistFunc('STIREIMPNFCE'),STIREIMPNFCE(), LjNFCeReImp())" , "", "30"} ) // "Reimprimir
109      else
110      If lEmitNfce
111      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0077 + "/NF-e" , "LjNFCeReImp()" , "", "30"} ) // "Reimprimir NFC-e"
112      ElseIf lMVFISNOTA
113      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0078 , "LjNFCeReImp()" , "", "30"} ) // "Reimprimir NF-e"
114      EndIf
115      EndIf
116      Else
117      AAdd( aRet , { AllTrim(STR(++nOption)) , StrTran(STR0075,"SAT",cSiglaSat) , cFuncCanc,"", "08"} ) // "Cancelar SAT"
118
119      If lMVFISNOTA
120      AAdd( aRet , { AllTrim(STR(++nOption)) , StrTran(STR0076+ "/NF-e","SAT",cSiglaSat) , cFuncReimp , "", "30"} ) // "Reimprimir SAT" + "NF-e"
121      Else
122      AAdd( aRet , { AllTrim(STR(++nOption)) , StrTran(STR0076,"SAT",cSiglaSat) , cFuncReimp , "", "30"} ) // "Reimprimir SAT"
123      Endif
124      EndIf
125      If ExistFunc("STFRetrNfe") .And. ExistFunc("STFPdv0n") .And. STFPdv0n(cEstacao) .And. !LjGetCPDV()[1]
126      AAdd( aRet , { AllTrim(STR(++nOption)) , "Retransmissão NF-e" , "STFRetrNfe()" , "", "31"} ) // "Retransmissão NF-e"
127      EndIf
128
129      If !SuperGetMV( "MV_VLTROCA",,.F. )
130      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0046 , "STIValeTroca()" , "", "09"} ) //"Vale Troca"
131      EndIf
132
133      //Se For homologacao do Paf nao mostra tela de recebimentos
134      If !(STBHomolPaf())
135      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0009 , "iIf(STBVldPD('RECTIT'), STIPanelReceb('R'), Nil)" , "", "10"} ) // "Recebimento de Titulo"
136      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0043 , "iIf(STBVldPD('ESTTIT'), STIPanelReceb('E'), Nil)" , "", "11"} ) // "Estorno de Titulo"
137      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0060 , "STICancelReceb()" , "", "12"} ) //"Cancelar Recebimento"
138      EndIf
139
140      If STFGetCfg("lMultCoin") // Usa Multimoeda
141
142      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0010 , "Alert("+STR0044+")" , "", "13"} ) // "Totais da Venda - Diversas Moedas"
143
144      If SuperGetMV("MV_TRCMOED")
145      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0011 , "Alert("+STR0044+")" , "", "14"} ) // "Troca da Moeda da Venda"
146      EndIf
147
148      EndIf
149
150      // Registro de Midia
151      If AllTrim(Str(SuperGetMv("MV_LJRGMD",,0))) $ "1|2"
152      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0040 , "STIMultiMedia()" , "", "15"} ) //"Registro de Midia"
153      ,""} ) // Midia
154      EndIf

```

```

154
155 If STFProFile(13,,,,,.T.)[1] .AND. !Empty(STFGetStation('GAVETA'))
156   AAdd( aRet , { AllTrim(STR(++nOption)) , STR0016 , "STB0penDrawer()" , "", "16"} ) //"Abrir Gaveta"
157 EndIf
158
159 // Funcoes para ECF
160 If lUseECF
161
162   AAdd( aRet , { AllTrim(STR(++nOption)) , STR0015 , "STB0penECF()" , "", "17"} ) //"Abrir ECF"
163
164   If !lPafEcf .AND. STFProFile(21,,,,,.T.)[1]
165     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0017 , "STBReadingX()" , "", "18"} ) //"Leitura X"
166   EndIf
167
168
169   If !lPafEcf .AND. STFProFile(12,,,,,.T.) [1]
170     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0018 , "STWZReduction(.F.)" , "", "19"} ) //"Fechar ECF (Reducao Z)"
171   EndIf
172
173   AAdd( aRet , { AllTrim(STR(++nOption)) , STR0019 , "STWMemryFisc()" , "", "20"} ) //"Leitura da Memoria Fiscal"
174
175 EndIf
176
177 // Funcoes associadas ao TEF
178 If lUsaTef
179
180   AAdd( aRet , { AllTrim(STR(++nOption)) , STR0021 , "STBFunAdm()" , "", "21"} ) //"TEF - Gerenciais"
181
182   //Se esta habilitado Coreespondente Sitef
183   If STFGetStat( "CBSIT" , .T. ) == "1"
184     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0004 , "STWCorBank()" , "", "22"} ) //"Correspondente bancario"
185   EndIf
186
187   //Se esta habilitado Recarga de celular Sitef
188   If STFGetStat( "RCSIT" , .T. ) == "1"
189     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0022 , "STWRecMob()" , "", "23"} ) //"Recarga de celular"
190   EndIf
191
192 EndIf
193
194 If lPafEcf .OR. lIsPafNfce
195   If SM0->M0_ESTCOB = "DF"
196     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0041 , "STBCotepe3505(.T.)" , "M", "24"} ) //"Ato COTEPE/ICMS 35/2005"
197   EndIf
198
199   If !lOnlyMenuFiscal
200     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0042 , "STBMenFis(.T., .T., .T.)" , "M", "25"} ) //"Menu Fiscal"
201
202     If STFProFile(12,,,,,.T.) [1] .And. !lIsPafNfce
203       AAdd( aRet , { AllTrim(STR(++nOption)) , STR0018 , "STWZReduction(.F.)" , "", "26"} )
204     EndIf
205   Else

```

```

206      AAdd( aRet , { AllTrim(STR(++nOption))      , STR0042 , "STBMenFis(.T., .T., .F.)"      , "M", "27"} ) //"Menu Fiscal"
207      EndIf
208  EndIf
209
210
211  // RFID
212  If SuperGetMv("MV_RFID",, .F.) .AND. ExistFunc("LJRFPesqProd") .AND. SLG->(ColumnPos("LG_RFID") > 0) .AND. STFGetStation("RFID") == "1"
213      AAdd( aRet , { AllTrim(STR(++nOption)) , STR0048 + " - " + STR0049      , "MsgRun(' + STR0050 + "','',{|| LJRFPesqProd() })" , "", "28"} ) // "RFID" - "Leitura de
214      Itens" //"Aguarde!, Realizando Leitura de Itens RFID. "
215  EndIf
216
217  //Lista de Presentes
218  If SuperGetMv("MV_LJLSPRE",, .F.)
219      AAdd( aRet , { AllTrim(STR(++nOption))      , STR0051 , "STIGiftList()" , "", "29"} ) // "Lista de Presentes"
220  EndIf
221
222  If lSTMenEdt
223      aPEMenu := ExecBlock("STMenEdt",.F.,.F.,{aRet})
224      If Len(aPEMenu) > 0
225          aRet := aPEMenu
226      EndIf
227  EndIf
228
229  //Função para validação dos menus
230  STBVldMenus(@aRet)
231 EndIf
232 Return aRet
233
234
235 //-----
236 /*{Protheus.doc} STBMenuMobile
237 Retorna as funcoes de menu disponiveis na versao Mobile do PDV
238
239 @return Ret - Lista de Funcoes
240
241 @obs
242 @sample
243 */
244 //-----
245 Function STBMenuMobile()
246
247 Local aRet      := {}                // Array de retorno das rotinas disponiveis
248 Local nOption   := 0                // Numero do atalho
249 Local lUsaTef   := STFGetCfg("lUsaTef") // Usa TEF
250 Local lNFCETSS  := STFGetCfg("lNFCETSS", .T.) // NFCE COM TSS
251
252
253 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0052 , ; //"Abertura de Caixa"
254      "IIF(STBPdvFree(), STIOpenCash(), Nil)", ""} ) // "Abertura de caixa"
255
256 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0034 , ;

```

```

257         "IIF(STBPdvFree(), STWPOSCloseCash(), Nil)", "") // "Encerramento de caixa"
258
259 //Versao Mobile Demonstrativa
260 //Somente opcoes de abertura e Fechamento de caixa
261 If STFGetCfg("cTypeOperation","") == "DEMONSTRACA0"
262     Return aRet
263 EndIf
264 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0036 , ;
265         "IIF(STBCaixaVld() .AND. STBPdvFree(), STISupplyBleeding('1',.T.), Nil)", "") // "Sangria de caixa" | Somente Dinheiro
266
267 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0037 , ;
268         "IIF(STBCaixaVld() .AND. STBPdvFree(), STISupplyBleeding('2') , Nil)", "") // "Suprimento de caixa"
269
270 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0047 , ;
271         "IIF(STBCaixaVld() , (STI7InfCPF(.F.),STWInfoCNPJ()) , Nil)", "") // "Informar CPF"
272
273 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0038 , ;
274         "IIF(STBCaixaVld() , STISalesmanSelection(), Nil)", "") // "Alterar Vendedor"
275
276
277 //Se estiver em venda -->> Cancela Cupom atual
278 //Se nao estiver em venda -->> Abre opcao para digitar o numero da venda
279 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0006 , ;
280         "IIF( STBCaixaVld() , IIF( STBCaixaVld() .AND. STFSaleTotal() > 0 , STICancelSale() , STIPanSaleCancel() ) , Nil )", "") // "Cancelar Cupom"
281
282 // Funcoes associadas ao TEF
283 If lUsaTef
284     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0021 , ;
285         "IIF(STBCaixaVld() .AND. STBPdvFree(), STBFunAdm(), Nil)", "") // "TEF - Gerenciais"
286 EndIf
287
288 //Subida de Vendas
289 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0053 , "FWMsgRun(,{|| STWUpData( , , STFGetStat('CODIGO')) } ,'+ STR0057 +' " , '+STR0058+' " ) " , "" } ) // "Envio de
    Vendas"#Aguarde'#Executando envio de vendas
290
291
292 //Carga de Dados
293 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0054 , "FWMsgRun(,{|| STWDownData( , , STFGetStat('CODIGO'), 5, .F. ) } , ' " + STR0057 + " ' , ' " + STR0059 + " ' ) " , "" } )
    // "Carga de Dados"#Aguarde'#Executando carga de dados
294
295 lNFCETSS := Valtype(lNFCETSS) = "U" .OR. lNFCETSS
296
297 If !lNFCETSS
298     //configurações NFCE Lib
299     AAdd( aRet , { AllTrim(STR(++nOption)) , STR0055 , "LjDCfgNFCE()", "" } ) // "Configurações NFC-e"
300
301 EndIf
302
303 //Log de mensagens
304 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0056 , "STFLatestMsg(.F.,.F.,.F.)" , "" } ) // "Log de Mensagens"
305
306 //Dados Cadastrais

```

```

307 //retirada da chamada da função STFCadEmp()
308
309 //Configurações Gerais
310 AAdd( aRet , { AllTrim(STR(++nOption)) , STR0064 , "STFMobWizard()" , "" , "configuracoes.svg" } ) // "Configurações"
311
312 Return aRet
313
314
315 //-----
316 /*/{Protheus.doc} STBGetNFCE
317 Valida uso da Nfc-e
318
319 @return lRet - Nfc-e
320
321 @obs
322 @sample
323 */
324 //-----
325 Function STBGetNFCE()
326
327 Local lEmitNfce := LjEmitNfCe() // Sinaliza se utiliza NFC-e
328
329 Return lEmitNfce
330
331
332 //-----
333 /*/{Protheus.doc} STBCaixaVld
334 Valida se o caixa está aberto e exibe mensagens caso nao esteja
335
336 @return lRet - Retorna se o caixa está aberto
337
338 @obs
339 @sample
340 */
341 //-----
342 Function STBCaixaVld(cType,cFunc)
343
344 Local lRet := .F. //Retorno
345
346 Default cType := "POPUP" //Tipo da Mensagem, quando Mobile "POPUP", para apresentar no rodape Desktop "ALERT"
347 Default cFunc := ""
348
349
350 /*Conforme legado FrontLoja, para realizar operacoes no Front deve estar
351 com o caixa aberto(ajusta todas as rotinas apos a primeira(encerramento), inclusive PEs)*/
352 lRet := STB0penCash()
353
354 If !lRet
355     STFMessage(ProcName(),cType,STR0062) //"Realize a Abertura do Caixa para executar esta opção."
356     STFShowMessage(ProcName())
357 EndIf
358

```

```

359 // Se for recebimento de titulos nao permite alterar no menu
360 If STIGetRecTit() .AND. !("STICancelReceb" $ cFunc)
361     lRet := .F.
362     STFMessage(ProcName(),cType,STR0065) //"Recebimento em andamento, finalize ou cancele o recebimento pelo Menu."
363     STFShowMessage(ProcName())
364 EndIf
365
366 Return lRet
367
368
369 //-----
370 /*{Protheus.doc} STBPdvFree
371 Valida se o PDV esta livre. ou seja nao esta em processo de venda.
372 Se não estiver livre exibe mensagem informativa.
373
374 @return lRet - Retorna se o pdv esta livre
375
376 @obs
377 @sample
378 */
379 //-----
380 Function STBPdvFree()
381
382 Local lRet          := .F.                //Retorno
383
384 // Se estiver zerado nao está em operacao de venda, e esta livre.
385 lRet := STFSaleTotal() == 0
386
387 If !lRet
388     STFMessage(ProcName(),"POPUP",STR0061) //"0 PDV já está em operação. Finalize a operação para executar está opção."
389     STFShowMessage(ProcName())
390 EndIf
391
392 Return lRet
393
394 //-----
395 /*{Protheus.doc} STBVldMenus
396 Função generica para validar os menus
397
398 @param aMenu, array, item do menu
399 @return lRet
400 @obs
401 @sample
402 */
403 //-----
404 Function STBVldMenus(aMenu)
405 Local nI      := 0
406 Local cFunc := ""
407
408 For nI:= 1 To Len(aMenu)
409     cFunc := aMenu[nI][3]
410

```

```

411 //Conforme legado, não pode inserir a validação do caixa no primeiro item e
412 //para o PAF não pode impedir o acesso ao menu
413 If nI == 1 .Or. ("STBMENFIS" $ Upper(cFunc))
414   aMenu[nI][3] := "IIF(STBVldTef('POPUP','"+AllTrim(aMenu[nI][2])+"' )," + cFunc + ",Nil)"
415 Else
416   aMenu[nI][3] := "IIF(STBCaixaVld('ALERT','"+StrTran(cFunc,"'", "'")+"' ) .And. "+
417     "STBVldTef('POPUP','"+AllTrim(aMenu[nI][2])+"' )," + cFunc + ",Nil) "
418   EndIf
419 Next aMenu
420
421 Return aMenu
422
423 //-----
424 /*{Protheus.doc} STBVldTef
425 Valida se fez a venda com TEF assim, permitindo o acesso a função
426 do menu
427
428 @param cType, string, tipo da mensagem (POPUP,ALERT)
429 @param cNomeMenu, string, nome do menu da tela
430 @return lRet - Retorna se pode fazer a venda
431 */
432 //-----
433 Function STBVldTef(cType,cNomeMenu)
434 Local lRet      := .T.
435
436 Default cType := "POPUP"    //Tipo da Mensagem
437 Default cNomeMenu := ""
438
439 //Mantem habilitado no menu somente a opção Cancelar,
440 //conforme documentação do param MV_TEFPEND ou
441 //caso não esteja com o fonte atualizado
442 If (Upper("Cancela") $ Upper(cNomeMenu))
443   lRet := .T.
444 Else
445   lRet := STIBlqMnTef()
446   If !lRet
447     STIMBlPTef(cType)
448   EndIf
449 EndIf
450
451 Return lRet
452
453 //-----
454 /*{Protheus.doc} STBVldPD
455 Verifica se o usuário pode acessar um determinado menu caso a rotina tenha campos Pessoais/Sensíveis
456 referente a Implementação de Proteção de Dados (LGPD).
457
458 @type Function
459 @param cNomeMenu, Caracter, Nome de referência do Menu.
460
461 @return Lógico, Retorna se o usuário tem acesso ou não a um determinado menu.
462 */

```

```

463 //-----
464 Function STBVldPD(cNomeMenu)
465 Local lRet      := .T.
466
467 Do Case
468
469     Case cNomeMenu == "CADCLI" //"Cadastro de Clientes"
470
471         If lLjPD Acesso
472             lRet := LjPD Acesso({"A1_NOME", "A1_CGC", "A1_PESSOA", "A1_TIPO", "A1_DTNASC", "A1_CEP", "E1_END", "A1_BAIRRO", "A1_EST", "A1_MUN", "A1_TEL", "A1_EMAIL"}) //Verifica se o
usuário pode acessar a rotina devido a regra de proteção de dados.
473             EndIf
474
475     Case cNomeMenu == "RECTIT" //"Recebimento de Titulo"
476
477         If lLjPD Acesso
478             lRet := LjPD Acesso({"L1_NOMCLI", "L1_CGCCLI"}) //Verifica se o usuário pode acessar a rotina devido a regra de proteção de dados.
479             EndIf
480
481     Case cNomeMenu == "ESTTIT" //"Estorno de Titulo"
482
483         If lLjPD Acesso
484             lRet := LjPD Acesso({"L1_NOMCLI", "L1_CGCCLI"}) //Verifica se o usuário pode acessar a rotina devido a regra de proteção de dados.
485             EndIf
486
487 EndCase
488
489 Return lRet

```

Componentes/Salesmen/STDSalesmanSelection.prw

354 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWMBROWSE.CH"
4 #INCLUDE "FWMVCDEF.CH"
5 #INCLUDE "PROTHEUS.CH"
6 #INCLUDE "APWIZARD.CH"
7 #INCLUDE "FWADAPTEREAI.CH"
8 #INCLUDE "STPOS.CH"
9 #INCLUDE "STDSALESMANSELECTION.CH"
10
11 Static oModelVen := Nil //Model do Vendedor
12 Static oVendStruct := Nil //Estrutura do vendedor
13
14 //-----
15 /*{Protheus.doc} STDSalesmanData
16 Cria estrutura de dados do Model de Vendedores, sem suas validacoes.
17 @param cKey Chave de Pesquisa. Contém Filial, Vendedor e Loja.
18 @param lOffline Pesquisa offline
19 @return xRet - Retorna estrutura de dados do Model
20 @obs
21 @sample
22 */
23 //-----
24 Function STDSalesmanData( cKey , lOffline )
25
26 Local aArea := GetArea() // Guarda area
27 Local oStruct := Nil // Estrutura
28 Local xRet := IIF(lOffline,Nil,{}) // Se a busca for offline, sera retornado um model. Caso contrario, sera retornado um array
29 Local aCampos := {} // Array de campos
30 Local nX := 0 // Contador
31
32 Default cKey := ""
33 Default lOffline := .T.
34
35
36 ParamType 0 Var cKey AS Character Default ""
37 ParamType 1 var lOffline AS Logical Default .F.
38
39 /*
40 Monta estrutura de tabela de Vendedores
41 */
42 oStruct := NoVldStruct()
43
44 aCampos := oStruct:GetFields()
45
46 If lOffline
47 /*
48 Instacia Objeto
49 */
50 xRet := MPFormModel():New( 'SA3', /*bPreValidacao*/, /*bPosValidacao*/, /*bCommit*/, /*bCancel*/ )
51
52 xRet:AddFields("SA3MASTER",/*cOwner*/,oStruct)

```

```

53     xRet:SetDescription("SalesMans")
54     xRet:SetOperation(3)
55     xRet:Activate()
56 EndIf
57
58
59 /*
60     Preenche o Modelo de dados do Vendedor com as informacoes da tabela SA3
61 */
62
63 DbSelectArea("SA3")
64 DbSetOrder(1)//A3_FILIAL+A3_COD
65 If DbSeek ( cKey )
66     For nX := 1 to Len(aCampos)
67         If !offline
68             xRet:SetValue('SA3MASTER', aCampos[nX][MODEL_FIELD_IDFIELD], &("SA3->" + aCampos[nX][MODEL_FIELD_IDFIELD]))
69         Else
70             AAdd(xRet,{aCampos[nX][MODEL_FIELD_IDFIELD],&("SA3->" + aCampos[nX][MODEL_FIELD_IDFIELD])})
71         EndIf
72     Next nX
73 EndIf
74
75 If !offline
76     oModelVen := xRet
77 EndIf
78
79 RestArea(aArea)
80
81 Return xRet
82
83
84 //-----
85 /*{Protheus.doc} NoVldStruct
86 Cria estrutura do Model de Vendedores, sem suas validacoes.
87 @param
88 @return oStruct - retorna estrutura do Model
89 @obs
90 @sample
91 */
92 //-----
93 Static Function NoVldStruct()
94
95 Local aArea      := GetArea()                // Guarda area
96 Local cAlias     := "SA3"                    // Alias que criara estrutura
97 Local cX2Unico   := "A3_FILIAL+A3_COD" //X2_UNICO da tabela SA3
98
99 If ValType(oVendStruct) == '0'
100     oVendStruct:DeActivate()
101     oVendStruct := Nil
102 EndIf
103
104 oVendStruct := FWFormModelStruct():New()      // Estrutura

```

```

105
106 //Carrega informacoes da Tabela
107 SX2->( DbSetOrder( 1 ) )
108 SX2->( DbSeek( cAlias ) )
109
110 If ExistFunc('FWX2Unico')
111     cX2Unico := FWX2Unico(cAlias)
112 EndIf
113
114 oVendStruct:AddTable(
115     FWX2CHAVE(), ; // [01] Alias da tabela
116     StrTokArr( cX2Unico, '+' ), ; // [02] Array com os campos que correspondem a primary key
117     FWX2Nome(cAlias) ) // [03] Descrição da tabela
118
119 //Carrega informacoes de campos
120 DbSelectArea("SX3")
121 SX3->(DbSetOrder(1))
122 SX3->(DbSeek(cAlias))
123
124 While !SX3->(EOF()) .AND. SX3->X3_ARQUIVO == cAlias
125
126     If Upper(AllTrim(SX3->X3_TIPO)) <> "M" .AND. SX3->X3_CONTEXT <> 'V'
127         oVendStruct:AddField(
128             AllTrim( X3Titulo() ) , ; // [01] Titulo do campo
129             AllTrim( X3Descric() ) , ; // [02] ToolTip do campo
130             AllTrim( SX3->X3_CAMPO ) , ; // [03] Id do Field
131             SX3->X3_TIPO , ; // [04] Tipo do campo
132             SX3->X3_TAMANHO , ; // [05] Tamanho do campo
133             SX3->X3_DECIMAL , ; // [06] Decimal do campo
134             Nil , ; // [07] Code-block de validacao do campo
135             Nil , ; // [08] Code-block de validacao When do campo
136             StrTokArr( AllTrim( X3CBox() ), ';' ) , ; // [09] Lista de valores permitido do campo
137             Nil , ; // [10] Indica se o campo tem preenchimento obrigatorio
138             Nil , ; // [11] Code-block de inicializacao do campo
139             NIL , ; // [12] Indica se trata-se de um campo chave
140             NIL , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
141             ( SX3->X3_CONTEXT == 'V' ) ) // [14] Indica se o campo e virtual
142         EndIf
143     SX3->(DbSkip())
144 End
145
146 RestArea(aArea)
147
148 Return oVendStruct
149
150
151
152 //-----
153 /* {Protheus.doc} STDGVenModel
154 Funcao que retorna o Model do Vendedor preenchido.
155
156 @return oModelVen - Model do Vendedor

```

```

157 @obs
158 @sample
159 /*/
160 //-----
161
162 Function STDGVenModel()
163 Return oModelVen
164
165
166 //-----
167 /*/ {Protheus.doc} STDFilPBSalesManData
168 Funcao que retorna o Model do Vendedor preenchido.
169 @param cVendedor   Vendedor
170 @return Nil
171 @obs
172 @sample
173 /*/
174 //-----
175 Function STDFilPBSalesManData( cVendedor )
176
177 Default cVendedor      := ""
178
179 ParamType 0 Var cVendedor      AS Character      Default ""
180
181 STDSPBasket("SL1","L1_VEND" ,oModelVen:GetValue("SA3MASTER","A3_COD"))
182
183 Return Nil
184
185
186 //-----
187 /*/ {Protheus.doc} STDFilPBSalesManData
188 Funcao que retorna o Model do Vendedor preenchido.
189 @param aDados - array de dados do Vendedor
190 @return oModel - Model do Vendedor
191 @obs
192 @sample
193 /*/
194 //-----
195 Function STDFilVenData( aDados )
196
197 Local oStruct      := Nil      // Estrutura
198 Local oModel       := Nil      // Model do Vendedor
199 Local nX           := 0        // Contador
200
201 Default aDados      := {}
202
203 ParamType 0 Var aDados AS Array Default {}
204
205 /*/
206     Monta estrutura de tabela de Vendedores
207 /*/
208 oStruct := NoVldStruct()

```

```

209
210 /*/
211     Instacia Objeto
212 /*/
213 oModel := MPFormModel().New( 'SA3', /*bPreValidacao*/, /*bPosValidacao*/, /*bCommit*/, /*bCancel*/ )
214
215 oModel:AddFields("SA3MASTER",/*cOwner*/,oStruct)
216 oModel:SetDescription("SalesMans")
217 oModel:SetOperation(3)
218 oModel:Activate()
219
220 For nX := 1 to Len(aDados)
221     oModel:LoadValue('SA3MASTER', aDados[nX][FIELD_NAME], aDados[nX][FIELD_VALUE])
222 Next nX
223
224 oModelVen := oModel
225
226 Return oModel
227
228 //-----
229 /*{Protheus.doc} STISalManFilter
230 Responsavel pela criação do filtro que sera passado ao Browse, com base nas informacoes digitadas pelo Vendedor.
231 @param cGetSalesman - String digitada pelo usuario utilizada para pesquisar o Vendedor.
232 @param oGetList      - Objeto de escolha do campo a ser utilizado na busca
233 @return
234 */
235 //-----
236
237 Function STISalManFilter(cGetSalesman,oGetList,aRecno)
238 Local aArea      := GetArea()    // Area Atual
239 Local nSec1      := 0            // Segundos Iniciais
240 Local nSec2      := 0            // Segundos no processo
241 Local nCont      := 0            // Contador
242 Local lMinGetSize := Len(AllTrim(cGetSalesman)) >= 1 // Tamanho minimo do campo
243 Local aSalesmans := {}           // Array com os vendedores achados
244 Local nLimit     := 20           // Limite de informacoes apresentadas na tela
245 Local cFilter    := ""           // Filtro que será utilizado para pesquisar o Vendedor desejado.
246 Local lRet       := .T.          // Variavel de retorno
247
248 nSec1 := Seconds()
249 aRecno := {}
250
251 If !Empty(cGetSalesman)
252     If lMinGetSize
253
254         DbSelectArea("SA3")
255
256         If Val(SubStr(cGetSalesman,1,TamSX3("A3_COD")[1])) == 0
257             cFilter := "SA3->A3_FILIAL == '" + xFilial('SA3') + "' .AND. ('" + Upper(AllTrim(cGetSalesman)) + "' $ Upper(AllTrim(SA3->A3_NOME)))" + ";
258                 " .Or. '" + Upper(AllTrim(cGetSalesman)) + "' $ Upper(AllTrim(SA3->A3_COD)))" + " .AND. SA3->A3_MSBLQL <> '1'"
259
260             SA3->(DbSetOrder())

```

```

261      If SA3->(EOF())
262          SA3->(DbGoTo(SA3->(LastRec()+1))
263      EndIf
264
265      SA3->(DbSetFilter({ || &cFilter }, cFilter))
266      SA3->(DbGoTop())
267
268      While !SA3->(EOF())
269          nCont++
270          aAdd(aRecno,SA3->(Recno()))
271          aAdd(aSalesmans,AllTrim(SA3->A3_COD)+" / "+AllTrim(SA3->A3_NOME)+" / "+AllTrim(SA3->A3_CGC))
272          If nCont == nLimit
273              Exit
274          EndIf
275          SA3->(DbSkip())
276      EndDo
277
278      SA3->(DbClearFilter())
279  Else
280      SA3->(DbSetOrder(1))
281      If DbSeek(xFilial("SA3")+AllTrim(cGetSalesman))
282          If SA3->A3_MSQLQL <> '1'
283              aAdd(aRecno,SA3->(Recno()))
284              aAdd(aSalesmans,AllTrim(SA3->A3_COD)+" / "+AllTrim(SA3->A3_NOME)+" / "+AllTrim(SA3->A3_CGC))
285          Endif
286      EndIf
287  EndIf
288  Else
289      lRet := .F.
290      STFMessage(ProcName(),"STOP",STR0001) //"É necessário digitar pelo menos 3 caracteres."
291      STFShowMessage(ProcName())
292  EndIf
293 EndIf
294
295
296 nSec2 := Seconds()-nSec1
297
298 Conout("-----")
299 ConOut(Str(nSec2))
300 Conout("-----")
301
302 oGetList:SetArray(aSalesmans)
303
304 If Len(aSalesmans) > 0
305     oGetList:GoTop()
306 EndIf
307
308 If lMinGetSize
309     If Len(aSalesmans) == 0
310         STFMessage(ProcName(),"STOP",STR0002) //"Nenhum Vendedor encontrado."
311         STFShowMessage(ProcName())
312         lRet := .F.

```

```

313 Else
314     If Len(aSalesmans) == nLimit
315         STFMessagem(ProcName(),"STOP",STR0003+AllTrim(Str(Len(aSalesmans)))+STR0004) //"0 resultado foi limitado a "##" Vendedores. Refine sua busca se for necessário."
316     ElseIf Len(aSalesmans) == 1
317         STFMessagem(ProcName(),"STOP",STR0005) //"1 Vendedor foi encontrado."
318     Else
319         STFMessagem(ProcName(),"STOP",STR0006+AllTrim(Str(Len(aSalesmans)))+STR0007) //"Foram encontrados "##" Vendedores."
320     EndIf
321     STFShowMessage(ProcName())
322
323     oGetList:SetFocus()
324 EndIf
325 EndIf
326
327 RestArea(aArea)
328
329 Return lRet
330
331 //-----
332 /*{Protheus.doc} STDFindSMan()
333 Pesquisa o vendedor em específico
334
335 @param
336 @return aRet
337 @obs
338 @sample
339 */
340 //-----
341 Function STDFindSMan()
342
343 Local aRet      := {}
344 Local cSalesMan := STDGPBasket("SL1","L1_VEND")
345
346 SA3->(dbSetOrder(1))
347 SA3->(dbSeek(xFilial('SA3')+cSalesMan))
348 If SA3->(!EOF())
349     Aadd(aRet,SA3->A3_NOME)
350
351 EndIf
352 EndIf
353
354 Return aRet

```

Componentes/Salesmen/STDSalesmenInfo.prw

26 linhas após remoção de metadados de autoria/data

```
1 #Include 'Protheus.ch'
2
3 //-----
4 /*{Protheus.doc} STDGComission
5 Busca a comissão do Vendedor
6
7 @param
8 @return Nil
9 @obs
10 @sample
11 */
12 //-----
13
14 Function STDGComission( cSalesman )
15 Local aArea := GetArea()
16 Local nRet := 0
17
18 DbSelectArea("SA3")
19 SA3->(DbSetOrder())
20 If DbSeek(xFilial("SA3")+cSalesman)
21     nRet := SA3->A3_COMIS
22 EndIf
23
24 RestArea(aArea)
25
26 Return nRet
```

Componentes/Salesmen/STWSalesmanSelection.prw

62 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STPOS.CH"
4
5
6 //-----
7 /* {Protheus.doc} STWSalesmanSelection
8 Workflow responsavel pela selecao de Vendedores.
9 @param  cKey      - Chave para busca no Back-Office
10 @return oModelVendedor - Retorna model do Vendedor
11 */
12 //-----
13 Function STWSalesmanSelection(cKey)
14
15 Local lOffline      := .T.           // Por questoes tecnicas, a busca pelo Vendedor sera feita apenas offline, porem a busca online ja esta implementada
16 Local aDadosVen     := {}           // Array de dados do Vendedor
17 Local aParam        := {}           // Array de parametros
18 Local cVendedor     := ""           // Vendedor
19 Local cLojaVen      := ""           // Loja do Vendedor
20 Local oModelVendedor := Nil         // Model de Vendedor
21 Local uResult       := Nil         // Resultado generico
22
23 Default cKey        := ""
24
25 ParamType 0 var  cKey      As Character      Default ""
26
27
28 If lOffline
29   /*
30     Atribuicao do model do Vendedor à variavel de Retorno.
31   */
32
33   oModelVendedor := STDSalesmanData(xFilial("SA3")+cKey,lOffline)
34   cVendedor := oModelVendedor:GetValue("SA3MASTER","A3_COD")
35
36 EndIf
37
38 Return oModelVendedor
39
40 //-----
41 /*{Protheus.doc} STBValidVenPad
42 Funcao que verifica se o Vendedor digitado pelo usuario eh o Vendedor padrao. Caso seja, seta lSearchOffline para True,
43 a fim de forçar todo o processo de selecao de Vendedores para ocorrer em ambiente local.
44 @param  cKey      - Chave para busca no Back-Office
45 @return lRet      - Retorna True caso o Vendedor digitado seja o Vendedor padrao, o que forca a pesquisa em ambiente local.
46 */
47 //-----
48
49 Function STBValidVenPad( cKey )
50
51 Local cVenPad      := SuperGetMv( "MV_VENDPAD" )      //Vendedor Padrao
52 Local lRet         := .F.                            //Retorno

```

```
53
54 Default cKey      := ""
55
56 ParamType 0 var cKey      As Character Default ""
57
58 If (cKey == cVenPad)
59     lRet := .T.
60 EndIf
61
62 Return lRet
```

Componentes/ShopCard/STBShopCard.prw

438 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWEVENTVIEWCONSTS.CH"
4 #INCLUDE "FWADAPTEREAI.CH"
5 #INCLUDE "FWMVCDEF.CH"
6
7 Static aRecSaldo      := {}           //Array que armazenara o saldo inserido no cartao fidelidade
8 Static llJcFid        := SuperGetMv("MV_LJCFID",,.F.) .AND. CrdxInt()//Indica se a recarga de cartao fidelidade esta ativa
9
10
11 //-----
12 /*{Protheus.doc} STBShopCard
13 STBShopCard
14 @return Nil
15 @obs
16 @sample
17 */
18 //-----
19 Function STBShopCard()
20
21 Return Nil
22
23
24 //-----
25 /*{Protheus.doc} ModelDef
26 MVC - Camada de modelo de dados
27 @return Exp01 = oModel
28 @obs
29 @sample
30 */
31 //-----
32 Static Function ModelDef()
33
34 Local oStruMBP := STBModelStruct()      // Estrutura MBP
35 Local oModel   := MPFormModel():New('STBSHOPCARD','*bPreValid*','*bPosValid*','*Commit*') // Model
36
37 oModel:AddFields('MBPDETAIL',Nil, oStruMBP )
38 oModel:SetDescription( "Recarga de cartão fidelidade" )// "Recarga de cartão fidelidade"
39 oModel:GetModel("MBPDETAIL"):SetDescription("Recarga de cartão fidelidade")
40 oModel:SetPrimaryKey({"MBP_FILIAL+MBP_NUMCAR+MBP_ITEM"})
41
42 Return oModel
43
44
45 //-----
46 /*{Protheus.doc} STBModelStruct
47 Monta a estrutura do model
48
49 @return oStruct - Retorno da estrutura
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Static Function STBModelStruct()
55
56 Local oStruct := FWFormModelStruct():New()    // Struct
57
58 oStruct:AddField(   "Num Cartão"               ,      ; // [01] Titulo do campo
59                   "Numero do Cartão"         ,      ; // [02] Desc do campo
60                   "MBP_NUMCAR"                ,      ; // [03] Id do Field
61                   "C"                         ,      ; // [04] Tipo do campo
62                   TamSx3("MBP_NUMCAR")[1]     ,      ; // [05] Tamanho do campo
63                   0                           ,      ; // [06] Decimal do campo
64                   Nil                          ,      ; // [07] Code-block de validacao do campo
65                   Nil                          ,      ; // [08] Code-block de validacao When do campo
66                   Nil                          ,      ; // [09] Lista de valores permitido do campo
67                   .T.                          ,      ; // [10] Indica se o campo tem preenchimento obrigatorio
68                   Nil                          ,      ; // [11] Code-block de inicializacao do campo
69                   NIL                          ,      ; // [12] Indica se trata-se de um campo chave
70                   NIL                          ,      ; // [13] Indica se o campo pode receber valor em uma operacao de update.
71                   .F.                          )      ; // [14] Indica se o campo e virtual
72
73 oStruct:AddField(   "Val.Recarga"               ,      ; // [01] Titulo do campo
74                   "Valor da Recarga"         ,      ; // [02] Desc do campo
75                   "MBP_VALOR"                ,      ; // [03] Id do Field
76                   "N"                         ,      ; // [04] Tipo do campo
77                   TamSx3("MBP_VALOR")[1]     ,      ; // [05] Tamanho do campo
78                   2                           ,      ; // [06] Decimal do campo
79                   Nil                          ,      ; // [07] Code-block de validacao do campo
80                   Nil                          ,      ; // [08] Code-block de validacao When do campo
81                   Nil                          ,      ; // [09] Lista de valores permitido do campo
82                   .T.                          ,      ; // [10] Indica se o campo tem preenchimento obrigatorio
83                   Nil                          ,      ; // [11] Code-block de inicializacao do campo
84                   NIL                          ,      ; // [12] Indica se trata-se de um campo chave
85                   NIL                          ,      ; // [13] Indica se o campo pode receber valor em uma operacao de update.
86                   .F.                          )      ; // [14] Indica se o campo e virtual
87
88 oStruct:AddField(   "Valid.Saldo"               ,      ; // [01] Titulo do campo
89                   "Validade do Saldo"         ,      ; // [02] Desc do campo
90                   "MBP_DTVAL"                ,      ; // [03] Id do Field
91                   "D"                         ,      ; // [04] Tipo do campo
92                   TamSx3("MBP_DTVAL")[1]     ,      ; // [05] Tamanho do campo
93                   0                           ,      ; // [06] Decimal do campo
94                   Nil                          ,      ; // [07] Code-block de validacao do campo
95                   Nil                          ,      ; // [08] Code-block de validacao When do campo
96                   Nil                          ,      ; // [09] Lista de valores permitido do campo
97                   .T.                          ,      ; // [10] Indica se o campo tem preenchimento obrigatorio
98                   Nil                          ,      ; // [11] Code-block de inicializacao do campo
99                   NIL                          ,      ; // [12] Indica se trata-se de um campo chave
100                  NIL                          ,      ; // [13] Indica se o campo pode receber valor em uma operacao de update.
101                  .F.                          )      ; // [14] Indica se o campo e virtual
102
103 Return oStruct
104

```

```

105
106 //-----
107 /* {Protheus.doc} ViewDef
108 MVC - Camada de visualização de dados
109 @return      oView = View
110 @obs
111 @sample
112 */
113 //-----
114
115
116 Static Function ViewDef()
117
118 Local oStruMBP      := STIViewStruct()           //Cria um objeto de Modelo de Dados baseado no ModelDef do fonte informado
119 Local oModel        := FWLoadModel( 'STBSHOPCARD' ) //Model
120 Local oView         := Nil                      //View
121
122 // Cria o objeto de View
123 oView := FWFormView():New()
124 oView:SetModel( oModel )
125
126 //Adiciona no View um controle do tipo FormFields(antiga enchoice)
127 oView:AddField( 'VIEW_MBPDETAIL', oStruMBP, 'MBPDETAIL' )
128
129 // Criar um "box" horizontal para receber algum elemento da view
130 oView:CreateHorizontalBox( 'SUPERIOR' , 100)
131
132 // Relaciona o ID da View com o "box" para exibicao
133 oView:SetOwnerView( 'VIEW_MBPDETAIL' , 'SUPERIOR' )
134 oView:EnableTitleView ( 'VIEW_MBPDETAIL' , "Dados da Recarga" ) // "Dados da Recarga"
135
136 Return oView
137
138
139 //-----
140 /*{Protheus.doc} STIStuView
141 Monta a estrutura do model
142 @param      oStruMst - Parametro que recebe a estrutura dos campos do model
143 @param      cType - Verifica se a estrutura 'e do model master ou grid
144 @return      oStruMst - Retorno da estrutura
145 @obs
146 @sample
147 */
148 //-----
149 Static Function STIViewStruct()
150
151 Local oStruct      := FWFormViewStruct():New() // Struct
152
153 oStruct:AddField( "MBP_NUMCAR" , , ; // [01] cIdField      ID do Field
154                  "1" , , ; // [02] cOrdem      Ordem do campo
155                  "Num Cartão" , , ; // [03] cTitulo      Título do campo
156                  "Numero do Cartão" , , ; // [04] cDescric      Descrição completa do campo

```

```

157      NIL      ,      ; // [05] aHelp      Array com o help dos campos
158      "C"      ,      ; // [06] cType      Tipo
159      PesqPict("MBP","MBP_NUMCAR") ,      ; // [07] cPicture      Picture do Campo
160      Nil      ,      ; // [08] bPictVar      Bloco de Picture var
161      Nil      ,      ; // [09] cLookup      Chave para ser usado no Loopup
162      .T.      ,      ; // [10] lCanChange      Lógico dizendo se o campo pode ser alterado
163      Nil      ,      ; // [11] cFolder      Id da folder onde o Field está
164      NIL      ,      ; // [12] cGroup      Id do Group onde o field está
165      NIL      ,      ; // [13] aCaomboValues Array com os valores do combo
166      NIL      ,      ; // [14] nMaxLenCombo      Tamanho máximo da maior opção do combo
167      NIL      ,      ; // [15] cIniBrow      Inicializador do Browse
168      NIL      ,      ; // [16] lVirtual      Indica se o campo é Virtual
169      NIL      ,      ; // [17] cPictVar      Picture Variável
170      NIL      ) // [18] lInsertLine      Indica pulo de linha após o campo
171
172  oStruct:AddField( "MBP_VALOR"      ,      ; // [01] cIdField      ID do Field
173      "2"      ,      ; // [02] cOrdem      Ordem do campo
174      "Val.Recarga" ,      ; // [03] cTitulo      Título do campo
175      "Valor da Recarga" ,      ; // [04] cDescric      Descrição completa do campo
176      NIL      ,      ; // [05] aHelp      Array com o help dos campos
177      "N"      ,      ; // [06] cType      Tipo
178      PesqPict("MBP","MBP_VALOR") ,      ; // [07] cPicture      Picture do Campo
179      Nil      ,      ; // [08] bPictVar      Bloco de Picture var
180      Nil      ,      ; // [09] cLookup      Chave para ser usado no Loopup
181      .T.      ,      ; // [10] lCanChange      Lógico dizendo se o campo pode ser alterado
182      Nil      ,      ; // [11] cFolder      Id da folder onde o Field está
183      NIL      ,      ; // [12] cGroup      Id do Group onde o field está
184      NIL      ,      ; // [13] aCaomboValues Array com os valores do combo
185      NIL      ,      ; // [14] nMaxLenCombo      Tamanho máximo da maior opção do combo
186      NIL      ,      ; // [15] cIniBrow      Inicializador do Browse
187      NIL      ,      ; // [16] lVirtual      Indica se o campo é Virtual
188      NIL      ,      ; // [17] cPictVar      Picture Variável
189      NIL      ) // [18] lInsertLine      Indica pulo de linha após o campo
190
191  oStruct:AddField( "MBP_DTVAL"      ,      ; // [01] cIdField      ID do Field
192      "3"      ,      ; // [02] cOrdem      Ordem do campo
193      "Valid.Saldo" ,      ; // [03] cTitulo      Título do campo
194      "Validade do Saldo" ,      ; // [04] cDescric      Descrição completa do campo
195      NIL      ,      ; // [05] aHelp      Array com o help dos campos
196      "D"      ,      ; // [06] cType      Tipo
197      PesqPict("MBP","MBP_DTVAL") ,      ; // [07] cPicture      Picture do Campo
198      Nil      ,      ; // [08] bPictVar      Bloco de Picture var
199      Nil      ,      ; // [09] cLookup      Chave para ser usado no Loopup
200      .T.      ,      ; // [10] lCanChange      Lógico dizendo se o campo pode ser alterado
201      Nil      ,      ; // [11] cFolder      Id da folder onde o Field está
202      NIL      ,      ; // [12] cGroup      Id do Group onde o field está
203      NIL      ,      ; // [13] aCaomboValues Array com os valores do combo
204      NIL      ,      ; // [14] nMaxLenCombo      Tamanho máximo da maior opção do combo
205      NIL      ,      ; // [15] cIniBrow      Inicializador do Browse
206      NIL      ,      ; // [16] lVirtual      Indica se o campo é Virtual
207      NIL      ,      ; // [17] cPictVar      Picture Variável
208      NIL      ) // [18] lInsertLine      Indica pulo de linha após o campo

```

```

209
210 Return oStruct
211
212
213 //-----
214 /*{Protheus.doc} STBVldShopCard
215 Chama funcao da retaguarda responsavel pela validacao do ShopCard.
216 @param  cNumCartao      Codigo do cartao Shop Card
217 @param  dDtValid        Data de validade
218 @param  nValor          Valor
219 @return lRet            - Retorna True caso o cartao seja valido e falso caso nao seja.
220 @obs
221 */
222 //-----
223 Function STBVldShopCard( cNumCartao , dDtValid , nValor)
224
225 Local oModel          := FwModelActive() // Model
226 Local lRet            := .T.             // Retorno
227 Local lRetComm        := {}              // Retorno comunicacao
228 Local aParam          := {cNumCartao}    // array de parametros
229 Local uResult         := Nil             // Resultado generico
230
231 Default cNumCartao    := ""
232 Default dDtValid      := Ctod("")
233 Default nValor        := 0
234
235 ParamType 0 Var      cNumCartao      As Character      Default      ""
236 ParamType 1 Var      dDtValid        As Date           Default      Ctod(" / / ")
237 ParamType 2 Var      nValor          As Numeric        Default      0
238
239 If Empty(cNumCartao)
240     lRet := .F.
241     STFMessage(ProcName(),"STOP","Código do cartão não preenchido.")
242     STFShowMessage(ProcName())
243     STFCleanMessage(ProcName())
244 EndIf
245
246 If lRet .AND. dDtValid < dDataBase
247     lRet := .F.
248     STFMessage(ProcName(),"STOP","A data de validade do saldo deve ser maior ou igual a data atual.")
249     STFShowMessage(ProcName())
250     STFCleanMessage(ProcName())
251 EndIf
252
253 If lRet .AND. nValor <= 0
254     lRet := .F.
255     STFMessage(ProcName(),"STOP","0 valor da recarga deve ser maior que zero.")
256     STFShowMessage(ProcName())
257     STFCleanMessage(ProcName())
258 EndIf
259
260 If lRet

```

```

261 If !STBRemoteExecute("Ca280Cart" ,aParam, NIL,.F. ,@uResult )
262
263 // Tratamento do erro de conexao
264 STFMessage(ProcName(), "ALERT", "Por falta de comunicação, não será possível prosseguir com a operação.")
265 STFShowMessage(ProcName())
266 STFCleanMessage(ProcName())
267 Else
268 lRetComm := uResult
269 uResult := Nil
270 lRet := lRetComm
271
272 If !lRet
273 STFMessage(ProcName(),"STOP","Cartão inválido.")
274 STFShowMessage(ProcName())
275 STFCleanMessage(ProcName())
276 EndIf
277 EndIf
278 EndIf
279
280 Return lRet
281
282
283 //-----
284 /*{Protheus.doc} STBAvalShopCard
285 Funcao que avalia se o produto informado é um produto de recarga ShopCard valido.
286 @param cItemCode      Codigo do item
287 @obs
288 @return lRet          - Retorna True caso o cartao seja valido e falso caso nao seja.
289 */
290 //-----
291 Function STBAvalShopCard( cItemCode )
292
293 Local lRet           := .F.                // Retorno
294 Local cProdFid       := SuperGetMv("MV_LJPFID") // Parametro com o codigo do produto tipo recarga de cartao fidelidade
295 Local aProdFid       := StrToKarr(cProdFid , "/" ) // Array com os produtos do parametro MV_LJPFID
296 Local nPosProd       := 0                // Posicao do produto no array informado durante a venda
297 Local oServer        := Nil              // Objeto de comunicacao
298
299 Default cItemCode := ""
300
301 ParamType 0 Var      cItemCode      As Character      Default      ""
302
303 If lljcFid
304
305 /*
306 Busco a posicao do produto no array aProdFid, que eh montado a partir do conteudo do parametro MV_LJPFID.
307 Se nao existir, significa que o produto nao eh um produto de recarga de cartao fidelidade.
308 */
309 nPosProd := Ascan(aProdFid,{|x| AllTrim(x) == AllTrim(cItemCode)})
310
311 If nPosProd > 0
312 lRet := .T.

```

```

313     EndIf
314 EndIf
315
316 Return lRet
317
318
319 //-----
320 /*{Protheus.doc} STBVldSCPament
321 Funcao responsavel por validar o ShopCard a ser utilizado como forma de pagamento. É avaliado se o cartao é valido e;
322 se ele tem saldo superior ao valor da compra.
323 @param  cNumCart      Codigo do ShopCard.
324 @param  nValorPgto    Valor pagamento
325 @param  dDataPag      data pagamento
326 @param  nParcelas     numero de parcelas
327 @return  lRet - Validou saldo
328 @obs
329 @sample
330 */
331 //-----
332 Function STBVldSCPament( cNumCart , nValorPgto , dDataPag , nParcelas )
333
334 Local aParam      := {}      // Array de parametros
335 Local lRet        := .T.      // Retorno
336 Local lRetComm     := .T.      // Retorno comunicacao
337 Local nRetComm     := 0       // numero retorno comunicacao
338 Local uResult     := Nil      // Resultado generico
339
340 Default cNumCart   := ""
341 Default nValorPgto := 0
342 Default dDataPag   := Ctod("")
343 Default nParcelas  := 0
344
345 ParamType 0 Var    cNumCart      As Character   Default   ""
346 ParamType 1 Var    nValorPgto    As Numeric     Default   0
347 ParamType 2 Var    dDataPag      As Date        Default   Ctod(" / / ")
348 ParamType 3 Var    nParcelas     As Numeric     Default   0
349
350 If nValorPgto <= 0
351     lRet := .F.
352     STFMessage(ProcName(),"STOP","0 valor deve ser maior que zero")
353     STFShowMessage(ProcName())
354     STFCleanMessage(ProcName())
355 EndIf
356
357 If lRet .AND. dDataPag < dDataBase
358     lRet := .F.
359     STFMessage(ProcName(),"STOP","A data de pagamento deve"+CRLF+"ser maior ou igual a data atual")
360     STFShowMessage(ProcName())
361     STFCleanMessage(ProcName())
362 EndIf
363
364 If lRet .AND. nParcelas < 1

```

```

365     lRet := .F.
366     STFMessage(ProcName(),"STOP","Deve haver ao menos 1 parcela")
367     STFShowMessage(ProcName())
368     STFCleanMessage(ProcName())
369 EndIf
370
371 If lRet
372     If Empty(cNumCart)
373         lRet := .F.
374         STFMessage(ProcName(),"STOP","Informe o numero do cartao!")
375         STFShowMessage(ProcName())
376         STFCleanMessage(ProcName())
377     Else
378         aParam := {cNumCart}
379
380
381         /*
382         Verifico se o cartao eh valido
383         */
384
385         If !STBRemoteExecute("Ca280Cart" ,aParam, NIL,.F. ,@uResult  )
386
387             // Tratamento do erro de conexao
388             STFMessage(ProcName(), "ALERT", "Por falta de comunicação, não será possível prosseguir com a operação.")
389             STFShowMessage(ProcName())
390             STFCleanMessage(ProcName())
391             lRet := .F.
392         Else
393             lRetComm := uResult
394             uResult := Nil
395
396             If !lRetComm
397                 lRet := .F.
398                 STFMessage(ProcName(), "ALERT", "0 cartao informado nao existe")
399                 STFShowMessage(ProcName())
400                 STFCleanMessage(ProcName())
401             Else
402                 aParam := {cNumCart}
403
404                 /*
405                 Verifico se o cartao tem saldo suficiente para a venda
406                 */
407
408                 If !STBRemoteExecute("Ca280Calc" ,aParam, NIL,.F. ,@uResult  )
409
410                     // Tratamento do erro de conexao
411                     STFMessage(ProcName(), "ALERT", "Por falta de comunicação, não será possível prosseguir com a operação.")
412                     STFShowMessage(ProcName())
413                     STFCleanMessage(ProcName())
414                     lRet := .F.
415                 Else
416                     nRetComm := uResult

```

```
417         uResult      := Nil
418
419         If ValType(nRetComm) == "N"
420             If nValorPgto > nRetComm
421                 lRet := .F.
422                 STFMessage(ProcName(), "ALERT", "Saldo indisponivel"+CRLF+"Este cartão possui R$"+ cvaltochar(nRetComm) +" de saldo")
423                 STFSHOWMESSAGE(ProcName())
424                 STFCLEANMESSAGE(ProcName())
425             EndIf
426         Else
427             lRet := .F.
428             STFMessage(ProcName(), "ALERT", "Falha na obtencao do saldo do cartao")
429             STFSHOWMESSAGE(ProcName())
430             STFCLEANMESSAGE(ProcName())
431         EndIf
432     EndIf
433 EndIf
434 EndIf
435 EndIf
436 EndIf
437
438 Return lRet
```

Componentes/ShopCard/STDShopCard.prw

131 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4
5 Static cNumCartao := "" // Numero do cartao
6 Static dValidade := Nil // Validade
7 Static nValue := 0 // Valor
8 Static cLojaFid := "" // Loja
9
10
11 //-----
12 /* {Protheus.doc} STDShopCardValues
13 Seta valores que serao acessados na finalizacao da venda.
14 @param cNumCar Numero do cartao
15 @param dDtValid Validade
16 @param nValor Valor
17 @param cLoja Loja
18 @return Nil
19 @obs
20 @sample
21 */
22 //-----
23 Function STDShopCardValues( cNumCar , dDtValid , nValor , cLoja )
24
25 Default cNumCar := "" // Numero do cartao
26 Default dDtValid := CtoD(" / / ") // Data validade
27 Default nValor := 0 // Valor
28 Default cLoja := "" // Loja
29
30 ParamType 0 Var cNumCar As Character Default ""
31 ParamType 1 Var dDtValid As Date Default CtoD(" / / ")
32 ParamType 2 Var nValor As Numeric Default 0
33 ParamType 3 Var cLoja As Character Default ""
34
35
36 cNumCartao := cNumCar
37 dValidade := dDtValid
38 nValue := nValor
39 cLojaFid := cLoja
40
41 Return Nil
42
43
44 //-----
45 /*{Protheus.doc} STDIncShopCardFunds
46 Chama funcao da retaguarda responsavel pela inclusao de saldo no ShopCard.
47 @param cItemCode Quantidade do Item
48 @param oDAOProd Modelo de dados contendo as informacoes do Produto
49 @return lRet - Inclui saldo?
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Function STDIncShopCardFunds()
55
56 Local aArea      := GetArea() // Guarda area
57 Local aParam     := {}        // Array de parametros
58 Local uResult    := Nil       // Resultado generico
59 Local lRet       := .F.       // retorno
60 Local cDoc       := STDGPBasket("SL1","L1_DOC") // Numero Documento
61 Local cSerie     := STDGPBasket("SL1","L1_SERIE") // Serie
62
63 aParam := {cNumCartao,dValidade,nValue,cDoc,cSerie,cLojaFid}
64
65 If !STBRemoteExecute("CA280ISLD" ,aParam, NIL,.T. ,@uResult )
66
67     // Tratamento do erro de conexao
68     STFMessage(ProcName(), "ALERT", "Por falta de comunicação, será gravada como contingência.")
69 Else
70     lRet := uResult
71 EndIf
72
73 RestArea(aArea)
74
75 // Limpa variaveis static
76 cNumCartao := ""
77 dValidade  := Ctod("")
78 nValue     := 0
79 cLojaFid   := ""
80
81 Return lRet
82
83
84 //-----
85 /*{Protheus.doc} STDUpdShopCardFunds
86 Chama funcao da retaguarda responsavel por atualizar o saldo do ShopCard.
87 @param oModelVenda - Model da venda
88 @param nItemLine - Numero do item
89 @return lRet - Retorna True caso o ShopCard tenha saldo disponivel para a venda.
90 */
91 //-----
92 Function STDUpdShopCardFunds( oModelVenda , nItemLine )
93
94 Local aArea      := GetArea() // Guarda area
95 Local aParam     := {}        // Array de parametros
96 Local uResult    := Nil       // Resultado generico
97 Local lRet       := .F.       // Retorno
98 Local cNumCar    := ""        // Numero do cartao
99 Local cDoc       := ""        // Documento
100 Local cSerie     := ""        // Serie
101 Local cLoja      := ""        // Loja
102 Local nSaldoAbat := 0         // Saldo
103
104 Default oModelVenda := Nil

```

```

105 Default nItemLine      := 1
106
107 ParamType 0 Var  oModelVenda As Object Default Nil
108 ParamType 1 Var   nItemLine   As Numeric Default 1
109
110 cNumCar := oModelVenda:GetValue("L4_NUMCART",nItemLine)
111 nSaldoAbat := oModelVenda:GetValue("L4_VALOR",nItemLine)
112 cDoc      := STDGPBasket("SL1","L1_DOC",nItemLine)
113 cSerie    := STDGPBasket("SL1","L1_SERIE",nItemLine)
114 cLoja     := STDGPBasket("SL1","L1_LOJA",nItemLine)
115 aParam    := {cNumCar,nSaldoAbat,cDoc,cSerie,cLoja}
116
117 If !STBRemoteExecute( "Ca280ASld" ,aParam, NIL,.F. ,@uResult  )
118
119     // Tratamento do erro de conexao
120     STFMessage(ProcName(), "ALERT", "Por falta de comunicação, não será possível prosseguir com a operação.")
121     STFShowMessage(ProcName())
122     STFCleanMessage(ProcName())
123     lRet := .F.
124 Else
125     lRet := uResult
126     uResult := Nil
127 EndIf
128
129 RestArea(aArea)
130
131 Return lRet

```

Componentes/ShopCard/STWShopCardRecharge.prw

124 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3
4 Static lLjcFid      := SuperGetMv("MV_LJCFID",,.F.) .AND. CrdxInt()//Indica se a recarga de cartao fidelidade esta ativa
5 Static lRecharge    := .F.                                     // Controle de Atualiza do shopcard
6
7
8 //-----
9 /*{Protheus.doc} STWShopCardRecharge
10 Efetua os processamentos para produtos agrupados. Chamado durante o processamento do STWItemReg.
11 @param cItemCode    Codigo do item
12 @return lRet        - Caso o produto seja do tipo Recarga Shop Card, retorna .T.
13 @obs
14 @sample
15 */
16 //-----
17 Function STWShopCardRecharge(cItemCode)
18
19 Local lRet          := .F.                                     // Indica se o cartao eh valido para recarga
20 Local cProdFid      := SuperGetMv("MV_LJPFID")               // Parametro com o codigo do produto tipo recarga de cartao fidelidade
21 Local oModelShopCard := Nil                                   // Model
22
23 Default cItemCode := ""
24
25 ParamType 0 Var cItemCode As Character Default ""
26
27 /*
28 Caso o parametro MV_LJCFID esteja setado como True e o produto esteja preenchido, continua o processamento..
29 */
30
31 If !Empty(cItemCode) .AND. lLjcFid
32
33     lRet := STBAvalShopCard(cItemCode) // Funcao que avalia se a regra de recarga do ShopCard se aplica ao produto informado.
34
35     If lRet
36         If Empty(STDGPBasket("SL2","L2_ITEM",1))
37             STIRecShopCard(cItemCode) //Exibe Interface.
38         Else
39             STFMessage(ProcName(),"STOP","Para efetuar a recarga, nenhum outro item pode estar registrado.")
40             STFShowMessage(ProcName())
41             STFCleanMessage(ProcName())
42         EndIf
43     EndIf
44
45 EndIf
46
47 Return lRet
48
49
50 //-----
51 /* {Protheus.doc} STWUpdShopCardFunds
52 Workflow responsavel por atualizar o saldo do ShopCard no momento da recarga.

```

```

53 @return lRet - atualizar o saldo do ShopCard no momento da recarga
54 @obs
55 @sample
56 /*/
57 //-----
58 Function STWUpdShopCardFunds(cNumCartao,nValor,dDtValid)
59 Local lRet      := .T.                // Retorno
60 Local cDoc      := STDGPBasket("SL1","L1_D0C")    // Documento
61 Local cSerie    := STDGPBasket("SL1","L1_SERIE")  // Serie
62 Local cLoja     := STDGPBasket("SL1","L1_LOJA")   // Loja
63
64 DEFAULT cNumCartao := ""
65 DEFAULT nValor     := 0
66 DEFAULT dDtValid   := Ctod("")
67
68 If lRet
69     lRet := STBVldShopCard(cNumCartao,dDtValid,nValor)
70     If lRet
71         STDSShopCardValues(cNumCartao,dDtValid,nValor,cLoja)
72     EndIf
73 EndIf
74
75 lRecharge := lRet
76
77 Return lRet
78
79
80 //-----
81 /*{Protheus.doc} STDGUpdShopCardFundsResult
82 Funcao que retorna se o cartao foi recarregado ou nao
83 @return lRecharge - retorna se o cartao foi recarregado ou nao
84 @obs
85 @sample
86 /*/
87 //-----
88 Function STDGUpdShopCardFundsResult()
89 Return lRecharge
90
91
92 //-----
93 /*{Protheus.doc} STDRUpdShopCardFundsResult
94 Reinicializa a variavel de controle lRecharge.
95 @return Nil
96 @obs
97 @sample
98 /*/
99 //-----
100 Function STDRUpdShopCardFundsResult()
101
102 lRecharge := .F.
103
104 Return Nil

```

```
105
106
107 //-----
108 /*/ {Protheus.doc} STWPayShopCard
109 Workflow responsavel por atualizar o saldo do ShopCard no momento da recarga.
110 @return lRet - Atualizou saldo
111 @obs
112 @sample
113 /*/
114 //-----
115 Function STWPayShopCard()
116
117 Local lRet := .F. // Retorno
118 Local cFormaPagamento := "FID" // T0 D0: Pegar forma de pagamento da cesta
119
120 If AllTrim(cFormaPagamento) == "FID"
121     //T0 D0: ???
122 EndIf
123
124 Return lRet
```